

# 内容 (一页小抄)

---

## 二进制信号

转换

加减乘除

补码

浮点数

Normalized、denormalized

0/1对应实际电压 (噪声边际)

时延

扇入扇出

## 布尔运算

公理定理, Shannon、De Morgan

布尔表达Vs真值表

最小项、最大项

卡诺图

Cost判定

## 组合电路

Decoder

Encoder

Mux

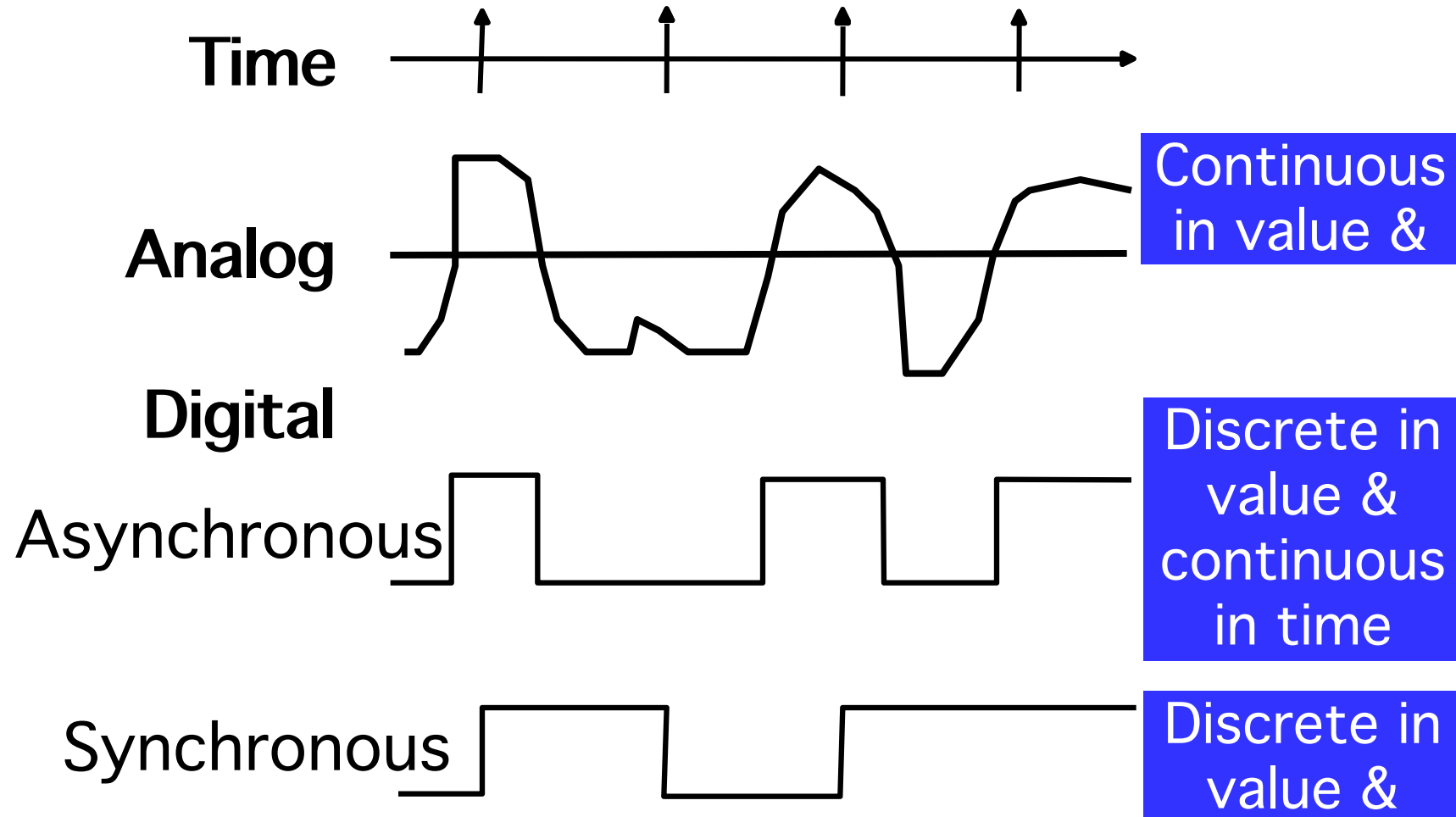
全加器

时序分析

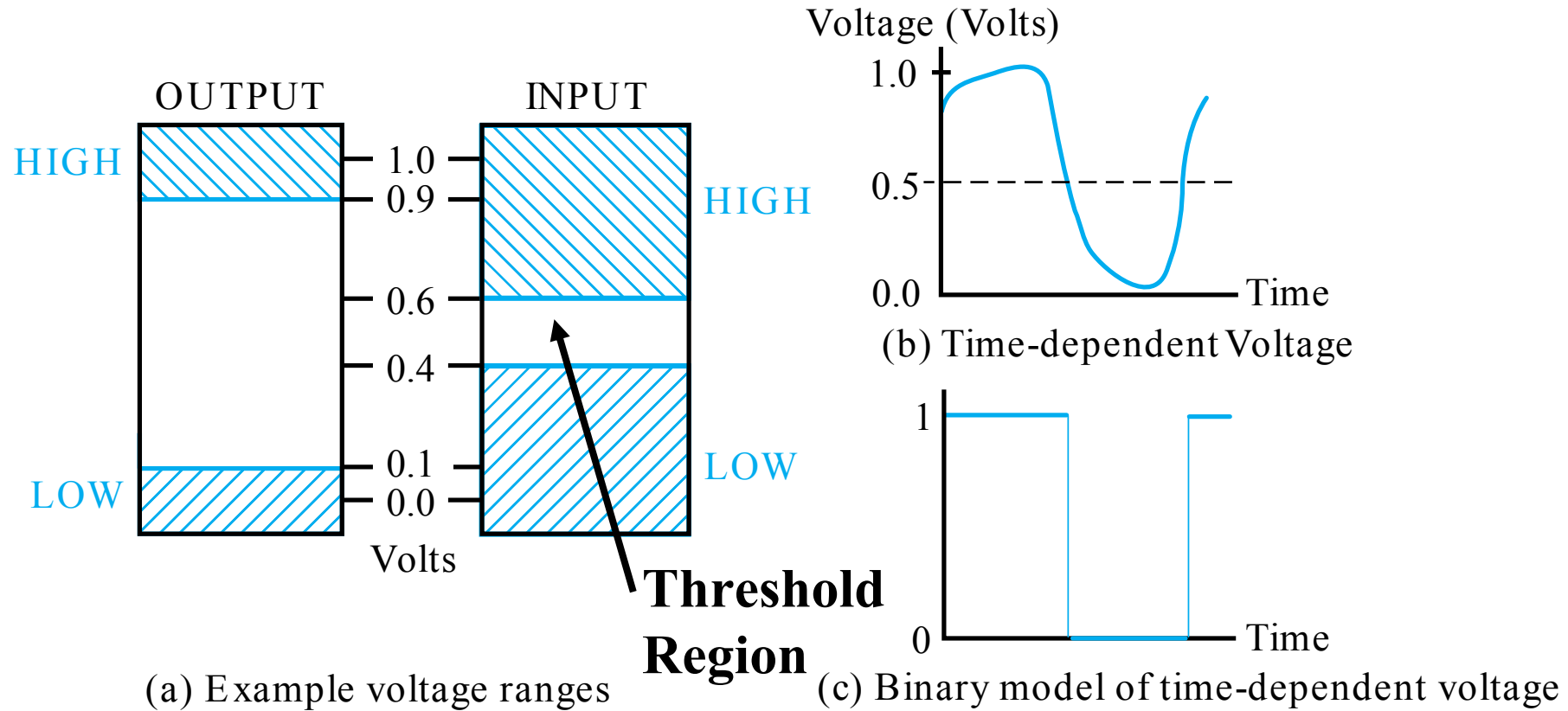
## 时序电路

# Signal Examples Over Time

---



# Signal Example – Physical Quantity: Voltage



# About the Decimal Number

---

**123.456**

$$1*10^2+2*10^1+3*10^0+4*10^{-1}+5*10^{-2}+6*10^{-3}$$

# What About the Binary Number?

---

**$(100101.01)_2$**

$$1*2^5+0*2^4+0*2^3+1*2^2+0*2^1+1*2^0+0*2^{-1}+1*2^{-2}$$

# Example: Convert $135_{10}$ To Base 2

---

$$(135)_{10} = (1000\ 0111)_2$$

2 | 1 3 5  
2 | 6 7 ..... 1  
2 | 3 3 ..... 1  
2 | 1 6 ..... 1  
2 | 8 ..... 0  
2 | 4 ..... 0  
2 | 2 ..... 0  
2 | 1 ..... 0  
2 | 0 ..... 1



# Example: Convert $0.6875_{10}$ To Base 2

---

$$(0.6875)_{10} = (0.1011)_2$$

$$\begin{array}{l} \underline{2 \times 0.6875 \dots\dots\dots} = 1.375 \\ 2 \times 0.375 \dots\dots\dots = 0.75 \\ \underline{2 \times 0.75 \dots\dots\dots} = 1.5 \\ 2 \times 0.5 \dots\dots\dots = 1 \end{array}$$



# Single Bit Binary Addition with Carry

---

Given two binary digits (X,Y), a carry in (Z) we get the following sum (S) and carry (C):

Carry in (Z) of 0:

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| Z   | 0   | 0   | 0   | 0   |
| X   | 0   | 0   | 1   | 1   |
| + Y | + 0 | + 1 | + 0 | + 1 |
| C S | 0 0 | 0 1 | 0 1 | 1 0 |

Carry in (Z) of 1:

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| Z   | 1   | 1   | 1   | 1   |
| X   | 0   | 0   | 1   | 1   |
| + Y | + 0 | + 1 | + 0 | + 1 |
| C S | 0 1 | 1 0 | 1 0 | 1 1 |



# Single Bit Binary Subtraction with Borrow

---

- Given two binary digits (X,Y), a borrow in (Z) we get the following difference (S) and borrow (B):

- Borrow in (Z) of 0:**

|            |                  |                  |                  |                  |
|------------|------------------|------------------|------------------|------------------|
| <b>Z</b>   | <b>0</b>         | <b>0</b>         | <b>0</b>         | <b>0</b>         |
| <b>X</b>   | <b>0</b>         | <b>0</b>         | <b>1</b>         | <b>1</b>         |
| <b>- Y</b> | <b><u>-0</u></b> | <b><u>-1</u></b> | <b><u>-0</u></b> | <b><u>-1</u></b> |
| <b>B S</b> | <b>0 0</b>       | <b>1 1</b>       | <b>0 1</b>       | <b>0 0</b>       |
- Borrow in (Z) of 1:**

|            |                  |                  |                  |                  |
|------------|------------------|------------------|------------------|------------------|
| <b>Z</b>   | <b>1</b>         | <b>1</b>         | <b>1</b>         | <b>1</b>         |
| <b>X</b>   | <b>0</b>         | <b>0</b>         | <b>1</b>         | <b>1</b>         |
| <b>- Y</b> | <b><u>-0</u></b> | <b><u>-1</u></b> | <b><u>-0</u></b> | <b><u>-1</u></b> |
| <b>B S</b> | <b>1 1</b>       | <b>1 0</b>       | <b>0 0</b>       | <b>1 1</b>       |

# Binary Multiplication

---

The binary multiplication table is simple:

$$0 * 0 = 0 \quad | \quad 1 * 0 = 0 \quad | \quad 0 * 1 = 0 \quad | \quad 1 * 1 = 1$$

Extending multiplication to multiple digits:

|                         |                        |
|-------------------------|------------------------|
| <b>Multiplicand</b>     | <b>1011</b>            |
| <b>Multiplier</b>       | <b><u>x 101</u></b>    |
| <b>Partial Products</b> | <b>1011</b>            |
|                         | <b>0000 -</b>          |
|                         | <b><u>1011 - -</u></b> |
| <b>Product</b>          | <b>110111</b>          |

# Binary Division

---

The binary multiplication table is simple:

$0 / 0 \quad | \quad 1 / 0$       **meaningless**

$0 / 1 = 0 \quad | \quad 1 / 1 = 1$

Extending division to multiple digits:

|          |              |
|----------|--------------|
| Dividend | 11001        |
| Divisor  | <u>/ 101</u> |
|          | 101) 11001   |
|          | <u>101</u>   |
|          | 101          |
|          | <u>101</u>   |
| Quotient | 101          |

# Bit-Level Operations in C

---

- Operations `&`, `|`, `~`, `^` Available in C
  - Apply to any “integral” data type
    - long, int, short, char, unsigned
  - View arguments as bit vectors
  - Arguments applied bit-wise
- Examples (Char data type)
  - `~0x41 -> 0xBE`
    - `~010000012 -> 101111102`
  - `~0x00 -> 0xFF`
    - `~000000002 -> 111111112`
  - `0x69 & 0x55 -> 0x41`
    - `011010012 & 010101012 -> 010000012`
  - `0x69 | 0x55 -> 0x7D`
    - `011010012 | 010101012 -> 011111012`

# Contrast: Logic Operations in C

---

- Contrast to Logical Operators

- `&&`, `||`, `!`
  - View 0 as “False”
  - Anything nonzero as “True”
  - Always return 0 or 1
  - Early termination (short cut)

- Examples (char data type)

- `!0x41 -> 0x00`
- `!0x00 -> 0x01`
- `!!0x41 -> 0x01`
  
- `0x69 && 0x55 -> 0x01`
- `0x69 || 0x55 -> 0x01`
- `p && *p` (avoids null pointer access)

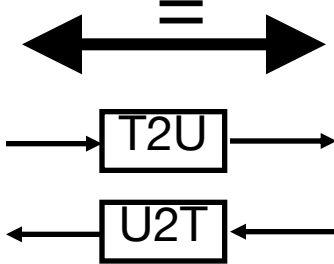
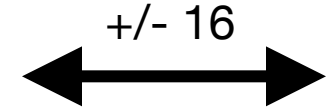
# Shift Operations

- Left Shift:  $x \ll y$ 
  - Shift bit-vector  $x$  left  $y$  positions
    - Throw away extra bits on left
    - Fill with 0's on right
- Right Shift:  $x \gg y$ 
  - Shift bit-vector  $x$  right  $y$  positions
    - Throw away extra bits on right
  - Logical shift
    - Fill with 0's on left
  - Arithmetic shift
    - Replicate most significant bit on right
- Undefined Behavior
  - Shift amount  $< 0$  or  $\geq$  word size

|                |          |
|----------------|----------|
| Argument $x$   | 01100010 |
| $\ll 3$        | 00010000 |
| Log. $\gg 2$   | 00011000 |
| Arith. $\gg 2$ | 00011000 |

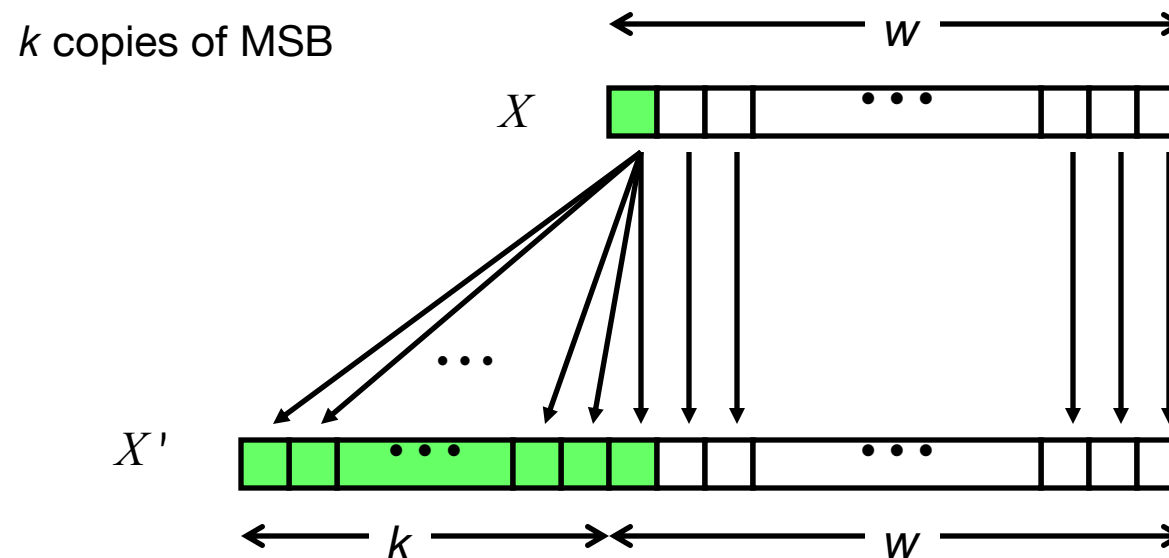
|                |          |
|----------------|----------|
| Argument $x$   | 10100010 |
| $\ll 3$        | 00010000 |
| Log. $\gg 2$   | 00101000 |
| Arith. $\gg 2$ | 11101000 |

# Mapping Signed ↔ Unsigned

| Bits | Signed |                                                                                       | Unsigned |
|------|--------|---------------------------------------------------------------------------------------|----------|
| 0000 | 0      |    | 0        |
| 0001 | 1      |                                                                                       | 1        |
| 0010 | 2      |                                                                                       | 2        |
| 0011 | 3      |                                                                                       | 3        |
| 0100 | 4      |                                                                                       | 4        |
| 0101 | 5      |                                                                                       | 5        |
| 0110 | 6      |                                                                                       | 6        |
| 0111 | 7      |                                                                                       | 7        |
| 1000 | -8     |  | 8        |
| 1001 | -7     |                                                                                       | 9        |
| 1010 | -6     |                                                                                       | 10       |
| 1011 | -5     |                                                                                       | 11       |
| 1100 | -4     |                                                                                       | 12       |
| 1101 | -3     |                                                                                       | 13       |
| 1110 | -2     |                                                                                       | 14       |
| 1111 | -1     |                                                                                       | 15       |

# Sign Extension

- Task:
  - Given  $w$ -bit signed integer  $x$
  - Convert it to  $w+k$ -bit integer with same value
- Rule:
  - Make  $k$  copies of sign bit:
  - $X' = \underbrace{x_{w-1}, \dots, x_{w-1}}_{k \text{ copies of MSB}}, x_{w-1}, x_{w-2}, \dots, x_0$





# Multiplication

$$\begin{array}{r}
 1 \quad 0001 \\
 4 \quad \times 0100 \\
 \hline
 0000 \\
 0000 \\
 0001 \\
 0000 \quad 0 \\
 \hline
 \text{xxxxxxxx} \\
 4 \quad 00000100
 \end{array}$$

$$\begin{array}{r}
 -7 \quad 1001 \\
 4 \quad \times 0100 \\
 \hline
 0000 \\
 0000 \\
 111001 \\
 0000 \quad 0 \\
 \hline
 \text{xxxxxxxx} \\
 -28 \quad 11100100
 \end{array}$$

$$\begin{array}{r}
 1 \quad 0001 \\
 -4 \quad \times 1100 \\
 \hline
 0000 \\
 0000 \\
 0001 \\
 11111 \quad 0 \\
 \hline
 \text{xxxxxxxx} \\
 -4 \quad 11111100
 \end{array}$$

$$\begin{array}{r}
 -7 \quad 1001 \\
 -4 \quad \times 1100 \\
 \hline
 00000000 \\
 0000000 \\
 111001 \\
 00111 \quad 0 \quad (\text{符号位1, 同上}) \\
 \hline
 \text{xxxxxxxx} \\
 28 \quad 100011100
 \end{array}$$

符号位1, 我们取反+1  
 : 0001->1110-  
 >1111,  
 高位补齐, 低位添0  
 -> (1) 1111 (000)

# Normalized Encoding Example

- Value: `Float F = 15213.0;`

- $15213_{10} = 11101101101101_2$   
 $= 1.1101101101101_2 \times 2^{13}$

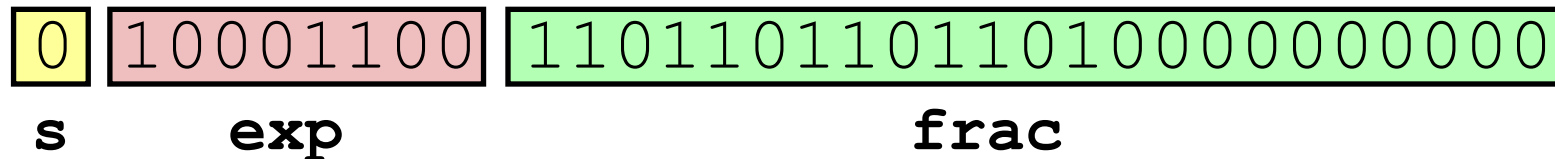
- Significand

$$\begin{array}{lcl} M & = & 1.\underline{1101101101101}_2 \\ \text{frac} & = & \underline{1101101101101}0000000000_2 \end{array}$$

- Exponent

$$\begin{array}{lcl} E & = & 13 \\ \text{Bias} & = & 127 \\ \text{Exp} & = & 140 = 10001100_2 \end{array}$$

- Result:



# Normalized Values

---

- Condition:  $\text{exp} \neq 000\dots 0$  and  $\text{exp} \neq 111\dots 1$ 
  - 指数000...0和111...1是保留给特殊情况
- Exponent coded as biased value:  $E = \text{Exp} - \text{Bias}$ 
  - Exp: unsigned value  $\text{exp}$
  - Bias =  $2^{k-1} - 1$ , where  $k$  is number of exponent bits
    - Single precision: 127 (Exp: 1...254, E: -126...127)
    - Double precision: 1023 (Exp: 1...2046, E: -1022...1023)
- Significand coded with implied leading 1:

$$M = 1.\text{xxx}\dots\text{x}_2$$

注意，内存中M仅存储分数部分。1在每次计算中自动会加。

# Dynamic Range (Positive Only)

|                      | s   | exp  | frac | E   | Value                |                                 |
|----------------------|-----|------|------|-----|----------------------|---------------------------------|
| Denormalized numbers | 0   | 0000 | 000  | -6  | 0                    | closest to zero                 |
|                      | 0   | 0000 | 001  | -6  | $1/8 * 1/64 = 1/512$ |                                 |
|                      | 0   | 0000 | 010  | -6  | $2/8 * 1/64 = 2/512$ |                                 |
|                      | ... |      |      |     |                      |                                 |
|                      | 0   | 0000 | 110  | -6  | $6/8 * 1/64 = 6/512$ | largest denorm<br>smallest norm |
|                      | 0   | 0000 | 111  | -6  | $7/8 * 1/64 = 7/512$ |                                 |
|                      | 0   | 0001 | 000  | -6  | $8/8 * 1/64 = 8/512$ |                                 |
|                      | 0   | 0001 | 001  | -6  | $9/8 * 1/64 = 9/512$ |                                 |
|                      | ... |      |      |     |                      |                                 |
| Normalized numbers   | 0   | 0110 | 110  | -1  | $14/8 * 1/2 = 14/16$ | closest to 1 below              |
|                      | 0   | 0110 | 111  | -1  | $15/8 * 1/2 = 15/16$ |                                 |
|                      | 0   | 0111 | 000  | 0   | $8/8 * 1 = 1$        | closest to 1 above              |
|                      | 0   | 0111 | 001  | 0   | $9/8 * 1 = 9/8$      |                                 |
|                      | 0   | 0111 | 010  | 0   | $10/8 * 1 = 10/8$    |                                 |
|                      | ... |      |      |     |                      |                                 |
|                      | 0   | 1110 | 110  | 7   | $14/8 * 128 = 224$   | largest norm                    |
|                      | 0   | 1110 | 111  | 7   | $15/8 * 128 = 240$   |                                 |
|                      | 0   | 1111 | 000  | n/a | inf                  |                                 |

# Rounding

---

- Rounding Modes (illustrate with \$ rounding)

|   |                          |               |               |               |               |                |
|---|--------------------------|---------------|---------------|---------------|---------------|----------------|
| ■ |                          | <b>\$1.40</b> | <b>\$1.60</b> | <b>\$1.50</b> | <b>\$2.50</b> | <b>−\$1.50</b> |
| • | Towards zero             | \$1           | \$1           | \$1           | \$2           | −\$1           |
| • | Round down ( $-\infty$ ) | \$1           | \$1           | \$1           | \$2           | −\$2           |
| • | Round up ( $+\infty$ )   | \$2           | \$2           | \$2           | \$3           | −\$1           |
| • | Nearest Even (default)   | \$1           | \$2           | \$2           | \$2           | −\$2           |

- What are the advantages of the modes?

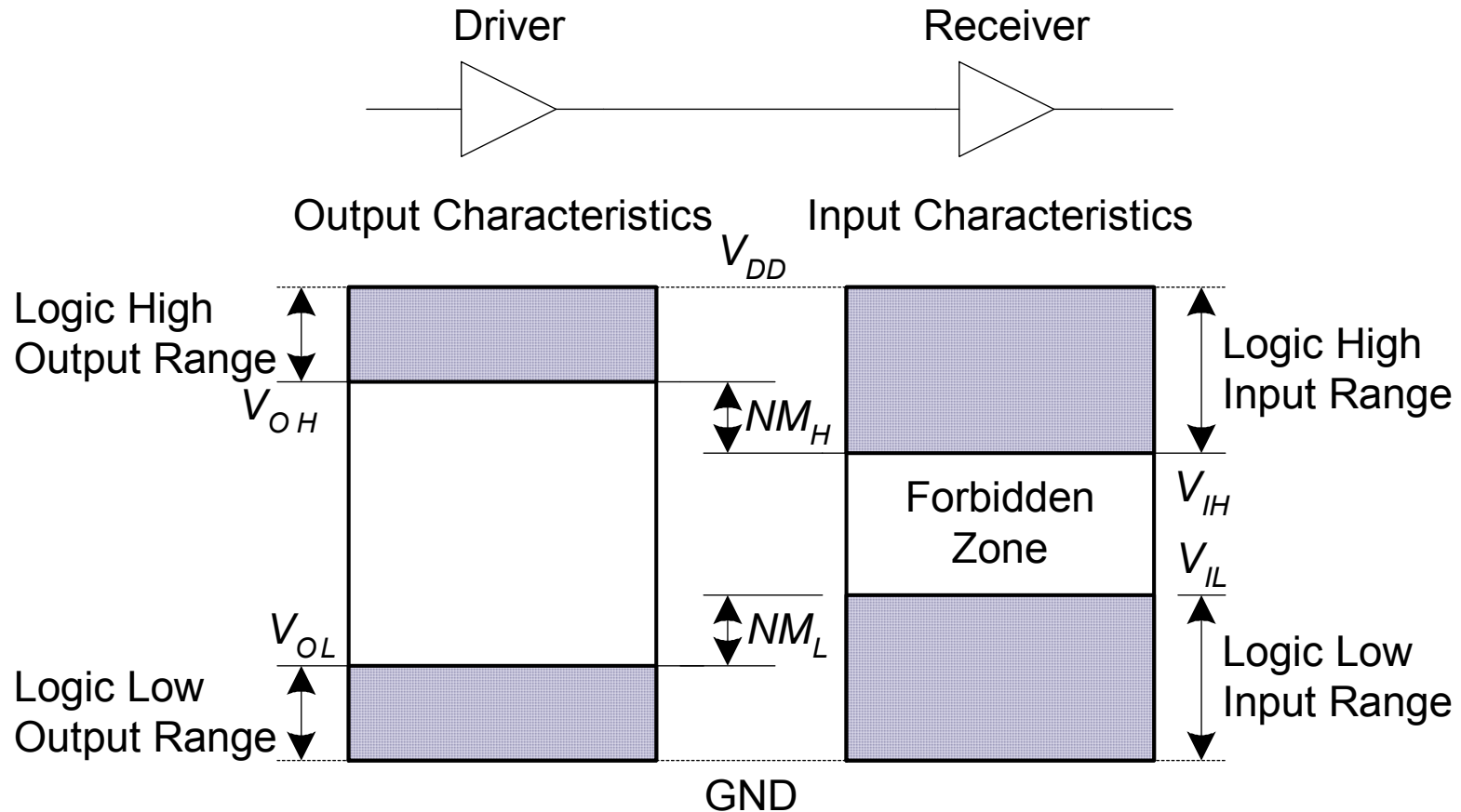
# IEEE 754

- Condition:  $\text{exp} \neq 000\dots 0$  and  $\text{exp} \neq 111\dots 1$ 
  - 指数000...0和111...1是保留给特殊情况
- Exponent coded as biased value:  $E = \text{Exp} - \text{Bias}$ 
  - Exp: unsigned value  $\text{exp}$
  - Bias =  $2^{k-1} - 1$ , where  $k$  is number of exponent bits
    - Single precision: 127 (Exp: 1...254, E: -126...127)
    - Double precision: 1023 (Exp: 1...2046, E: -1022...1023)
- Significand coded with implied leading 1:

$$M = 1.\text{xxx}\dots\text{x}_2$$

注意，内存中M仅存储分数部分。1在每次计算中自动会加。

# Noise Margins

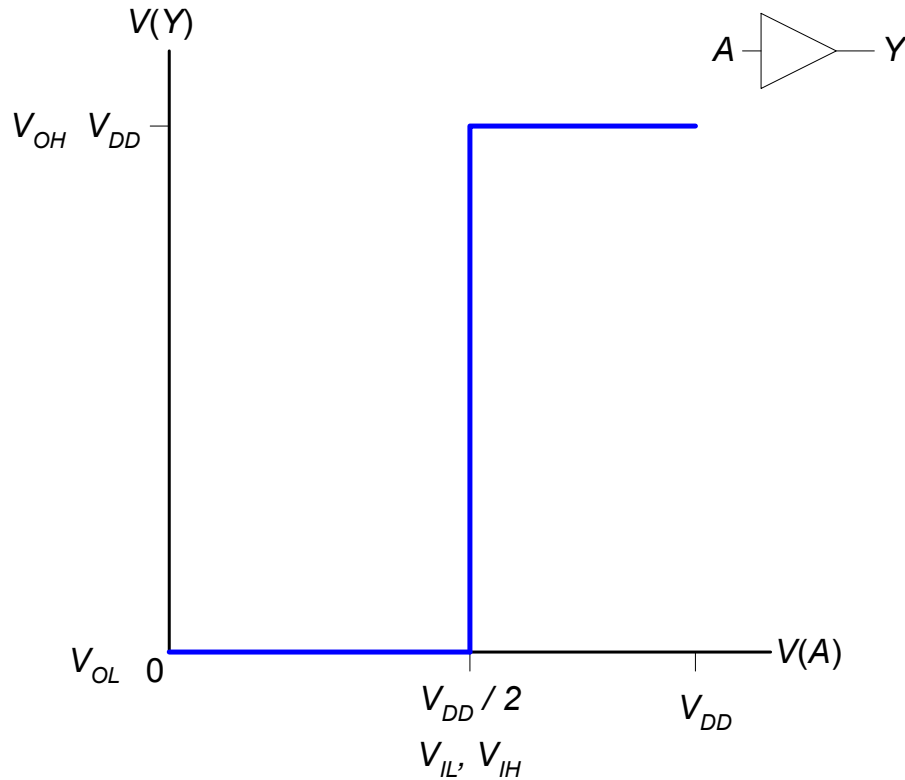


$$NM_H = V_{OH} - V_{IH}$$

$$NM_L = V_{IL} - V_{OL}$$

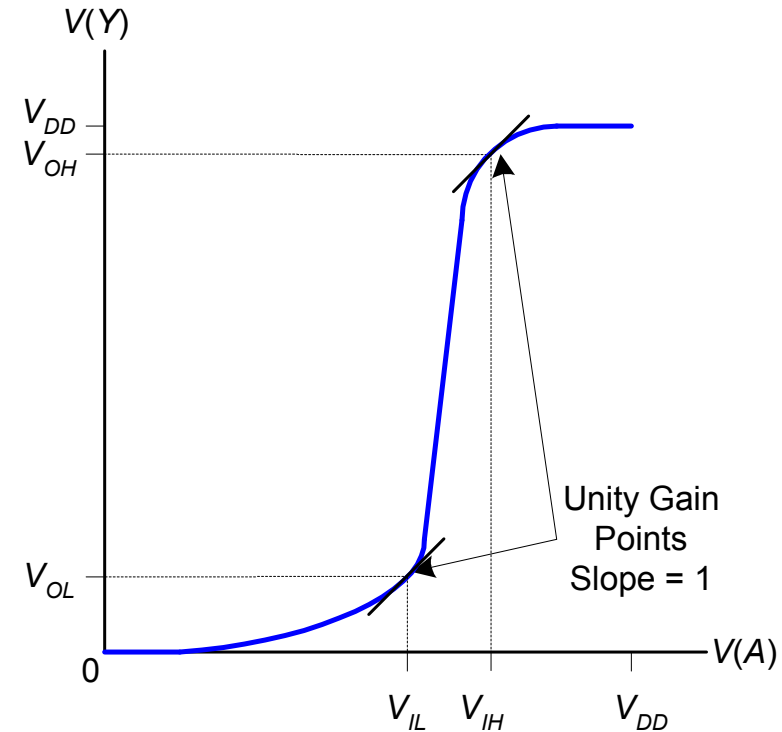
# DC Transfer Characteristics

Ideal Buffer:



$$NM_H = NM_L = V_{DD} / 2$$

Real Buffer:

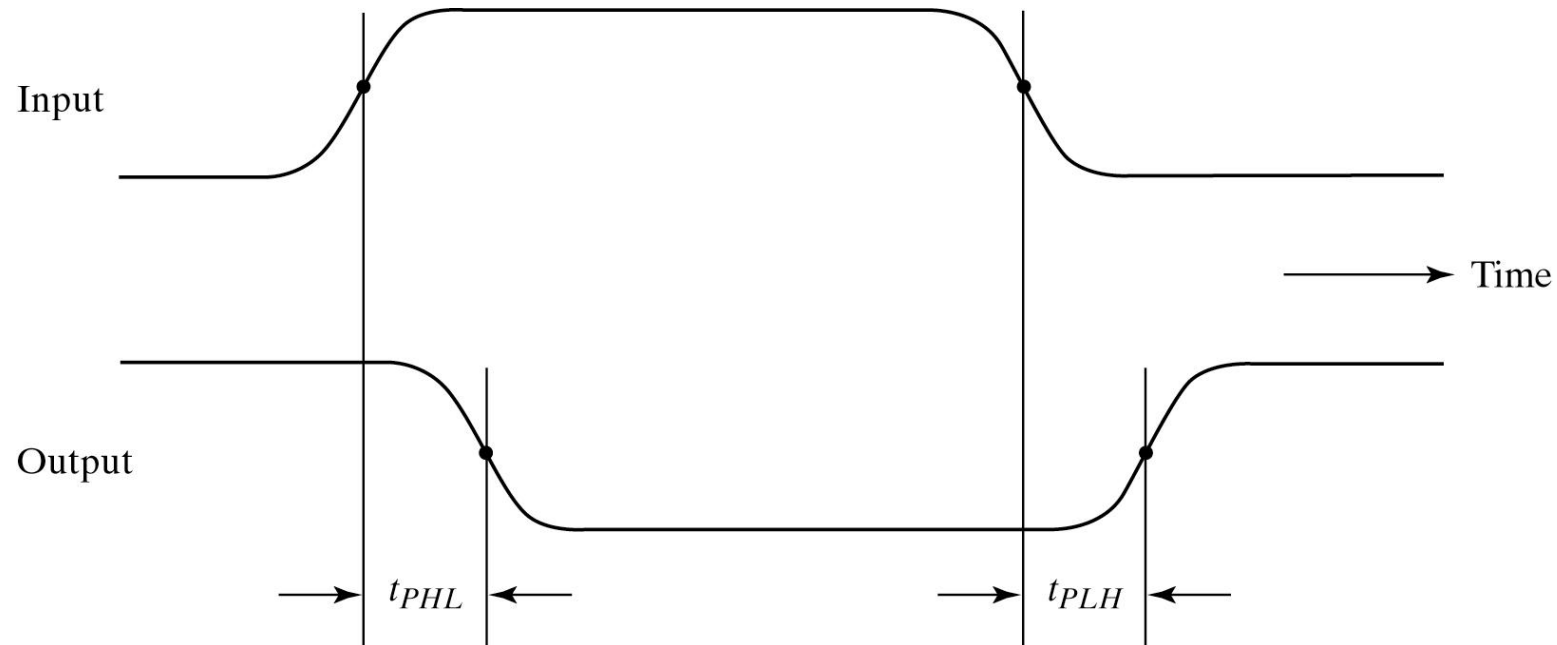


$$NM_H, NM_L < V_{DD} / 2$$



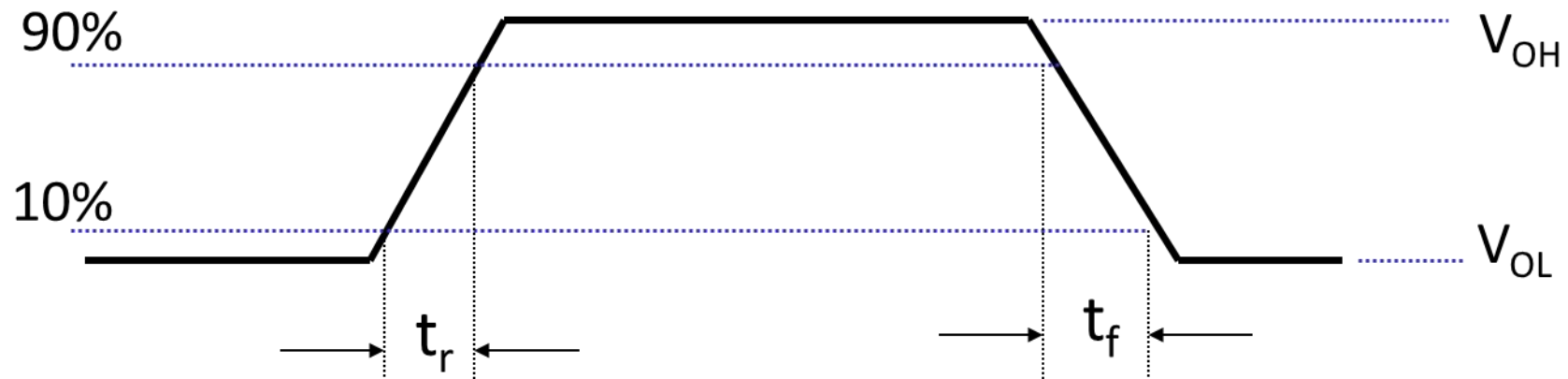
# Propagation Delay ( $T_{pd}$ , $T_{plh}$ , $T_{phl}$ )

- Outputs don't instantaneously reflect input changes
- Propagation delay is a measure of this time
- Delay from H to L can be different than from L to H



# Transition Time (Rise & Fall)

- Takes time for signal to reach its output voltage
- Will be affected by capacitance, fan-out
- Also known as slew rate
- Typically measured from 10%/90% of  $V_{OL}/V_{OH}$  swing



# Power Dissipation

---

- Power “dissipation” and “consumption”
- Static (“quiescent”) power dissipation
  - When device outputs not changing
  - Caused by the quiescent supply current,  $I_{DD}$  (also called the leakage current):  $P_S = I_{DD} \cdot V_{CC}$
  - Very low for CMOS
- Dynamic power dissipation

$$P_D = (C_{PD} + C_L) \cdot V_{CC}^2 \cdot f$$

$C_{PD}$  = Power dissipation capacitance (constant for logic family)

$C_L$  = Capacitive load on output (driving other devices)

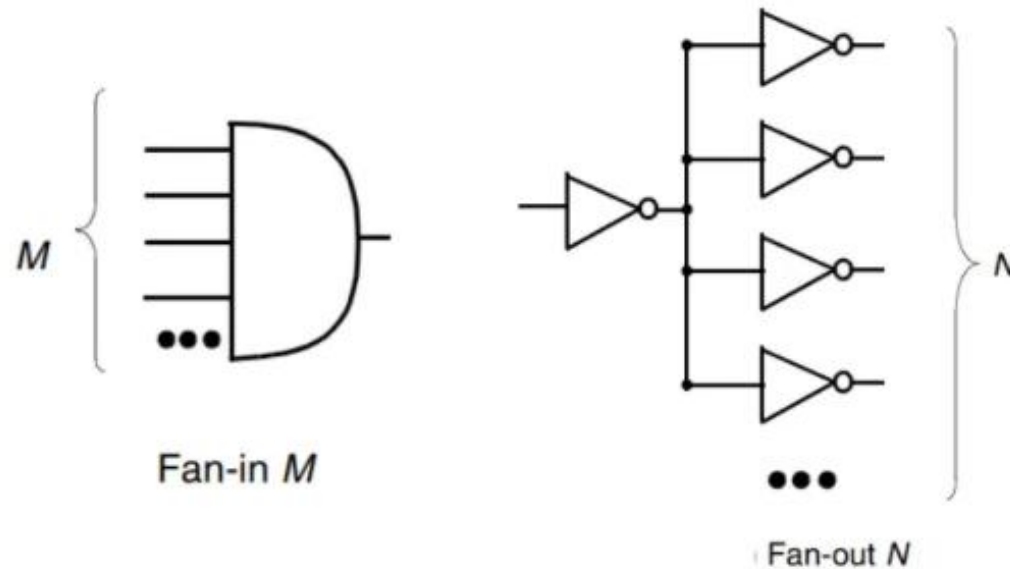
$V_{CC}$  = Supply voltage

$f$  = Transition frequency (0.5 x number of output transitions/second)

# Fan-in and Fan-out

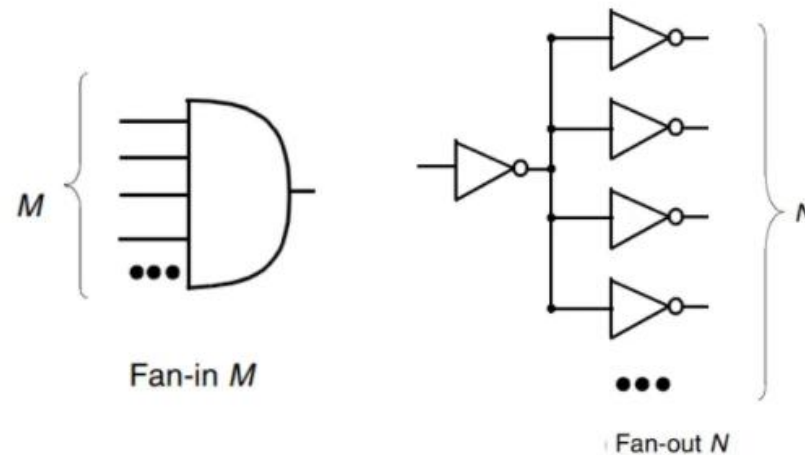
---

- Fan-in refers to the maximum number of input signals that feed the input equations of a logic cell.
- Fan-out refers to the maximum number of output signals that are fed by the output equations of a logic cell.



# Fan-in and Fan-out

- Fan-out refers to the maximum number of output signals that are fed by the output equations of a logic cell.
  - Fan Out is calculated from the amount of current available in the output of a gate and the amount of current needed in each input of the connecting gate. It is specified by manufacturer and is provided in the data sheet. Exceeding the specified maximum load may cause a malfunction because the circuit will not be able supply the demanded power.



# Theorems: Single Variable

---

- (T1)  $X + 1 = 1$
- (T2)  $X + 0 = X$
- (T3)  $X + X = X$
- (T4)  $\overline{\overline{X}} = X$
- (T5)  $X + \overline{X} = 1$
- (T1D)  $X \cdot 0 = 0$
- (T2D)  $X \cdot 1 = X$
- (T3D)  $X \cdot X = X$
- (T5D)  $X \cdot \overline{X} = 0$

# Theorems: 2-3 Variables

---

- (T6)  $X + Y = Y + X$
- (T7)  $(X + Y) + Z = X + (Y + Z)$
- (T8)  $X \cdot (Y + Z) = X \cdot Y + X \cdot Z$
- (T9)  $X + X \cdot Y = X$
- (T10)  $X \cdot Y + X \cdot \bar{Y} = X$
- (T11)  $X \cdot Y + \bar{X} \cdot Z + Y \cdot Z = X \cdot Y + \bar{X} \cdot Z$
- (T6D)  $X \cdot Y = Y \cdot X$
- (T7D)  $(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$
- (T8D)  $X + Y \cdot Z = (X + Y) \cdot (X + Z)$
- (T9D)  $X \cdot (X + Y) = X$
- (T10D)  $(X + Y) \cdot (X + \bar{Y}) = X$
- (T11D)  $(X + Y) \cdot (\bar{X} + Z) \cdot (Y + Z) = (X + Y) \cdot (\bar{X} + Z)$

# Theorems: n Variables

---

## ■ Generalized Idempotent Theorem

- (T12)  $X + X + \dots + X = X$
- (T12D)  $X \cdot X \cdot \dots \cdot X = X$

## ■ De Morgan's Theorem

- (T13)  $\overline{X_1 \cdot X_2 \cdot \dots \cdot X_n} = \overline{X_1} + \overline{X_2} + \dots + \overline{X_n}$
- (T13D)  $\overline{\overline{X_1} + \overline{X_2} + \dots + \overline{X_n}} = \overline{X_1} \cdot \overline{X_2} \cdot \dots \cdot \overline{X_n}$
- (T14)  $\overline{F(X_1, X_2, \dots, X_n, +, \cdot)} = F(\overline{X_1}, \overline{X_2}, \dots, \overline{X_n}, \cdot, +)$

## ■ Shannon's Theorem

- (T15)  $F(X_1, X_2, \dots, X_n) = X_1 \cdot F(1, X_2, \dots, X_n) + \overline{X_1} \cdot F(0, X_2, \dots, X_n)$
- (T15D)  $F(X_1, X_2, \dots, X_n) = [X_1 + F(0, X_2, \dots, X_n)] \cdot [\overline{X_1} + F(1, X_2, \dots, X_n)]$



# Boolean expressions-NOT unique

- Unlike truth tables, expressions representing a Boolean function are NOT unique.
- Example:
- $F(x,y,z) = \bar{x}\bar{y}\bar{z} + \bar{x}y\bar{z} + xy\bar{z}$
- $G(x,y,z) = \bar{x}\bar{y}\bar{z} + y\bar{z}$
- The corresponding truth tables for F() and G() are to the right. They are identical.
- Thus,  $F() = G()$

| x | y | z |  | F | G |
|---|---|---|--|---|---|
| 0 | 0 | 0 |  | 1 | 1 |
| 0 | 0 | 1 |  | 0 | 0 |
| 0 | 1 | 0 |  | 1 | 1 |
| 0 | 1 | 1 |  | 0 | 0 |
| 1 | 0 | 0 |  | 0 | 0 |
| 1 | 0 | 1 |  | 0 | 0 |
| 1 | 1 | 0 |  | 1 | 1 |
| 1 | 1 | 1 |  | 0 | 0 |

# Truth Table notation for Minterms and Maxterms

- Minterms and Maxterms are easy to denote using a truth table.
- Example:  
Assume 3 variables x,y,z  
(order is fixed)

| No. | x | y | z | Minterm                       | Maxterm                         |
|-----|---|---|---|-------------------------------|---------------------------------|
| 0   | 0 | 0 | 0 | $\bar{x}\bar{y}\bar{z} = m_0$ | $x+y+z = M_0$                   |
| 1   | 0 | 0 | 1 | $\bar{x}\bar{y}z = m_1$       | $x+y+\bar{z} = M_1$             |
| 2   | 0 | 1 | 0 | $\bar{x}y\bar{z} = m_2$       | $x+\bar{y}+z = M_2$             |
| 3   | 0 | 1 | 1 | $\bar{x}yz = m_3$             | $x+\bar{y}+\bar{z} = M_3$       |
| 4   | 1 | 0 | 0 | $x\bar{y}\bar{z} = m_4$       | $\bar{x}+y+z = M_4$             |
| 5   | 1 | 0 | 1 | $x\bar{y}z = m_5$             | $\bar{x}+y+\bar{z} = M_5$       |
| 6   | 1 | 1 | 0 | $xy\bar{z} = m_6$             | $\bar{x}+\bar{y}+z = M_6$       |
| 7   | 1 | 1 | 1 | $xyz = m_7$                   | $\bar{x}+\bar{y}+\bar{z} = M_7$ |

# Canonical Forms (Unique)

---

- Any Boolean function  $F()$  can be expressed as a unique **sum of minterms** and a unique **product of maxterms** (under a fixed variable ordering).
- In other words, every function  $F()$  has two canonical forms:
  - Canonical Sum-Of-Products (sum of minterms)
  - Canonical Product-Of-Sums (product of maxterms)

# Standard Forms (NOT Unique)

---

- Standard forms are “like” canonical forms, except that not all variables need appear in the individual product (SOP) or sum (POS) terms.
- Example
  - $f1(x,y,z) = \bar{x}\bar{y}z + y\bar{z} + x\bar{z}$  is a standard sum-of-products form
  - $f1(x,y,z) = (x+y+z) \cdot (\bar{y}+\bar{z}) \cdot (\bar{x}+\bar{z})$  is a standard product-of-sums form
- Conversion of SOP from standard to canonical form
  - Expand non-canonical terms by inserting equivalent of 1 in each missing variable  $x$ :  $(x + \bar{x}) = 1$
  - Remove duplicate minterms
- Conversion of POS from standard to canonical form
  - Expand noncanonical terms by adding 0 in terms of missing variables (e.g.,  $x\bar{x} = 0$ ) and using the distributive law
  - Remove duplicate maxterms

# An Example of 4-Variable K-Map

| <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> | <i>Y</i> |
|----------|----------|----------|----------|----------|
| 0        | 0        | 0        | 0        | 1        |
| 0        | 0        | 0        | 1        | 0        |
| 0        | 0        | 1        | 0        | 1        |
| 0        | 0        | 1        | 1        | 1        |
| 0        | 1        | 0        | 0        | 0        |
| 0        | 1        | 0        | 1        | 1        |
| 0        | 1        | 1        | 0        | 1        |
| 0        | 1        | 1        | 1        | 1        |
| 1        | 0        | 0        | 0        | 1        |
| 1        | 0        | 0        | 1        | 1        |
| 1        | 0        | 1        | 0        | 1        |
| 1        | 0        | 1        | 1        | 0        |
| 1        | 1        | 0        | 0        | 0        |
| 1        | 1        | 0        | 1        | 0        |
| 1        | 1        | 1        | 0        | 0        |
| 1        | 1        | 1        | 1        | 0        |

| Y<br>CD \ AB |   |    |    |    |    |
|--------------|---|----|----|----|----|
|              |   | 00 | 01 | 11 | 10 |
| 00           | 1 | 0  | 0  | 1  |    |
| 01           | 0 | 1  | 0  | 1  |    |
| 11           | 1 | 1  | 0  | 0  |    |
| 10           | 1 | 1  | 0  | 1  |    |

$$Y = \overline{A}C + \overline{A}BD + A\overline{B}\overline{C} + \overline{B}\overline{D}$$

# K-Map Definitions

---

- **Implicant**

- Product of literals  $A\bar{B}C, \bar{A}C, BC$

- **Prime implicant**

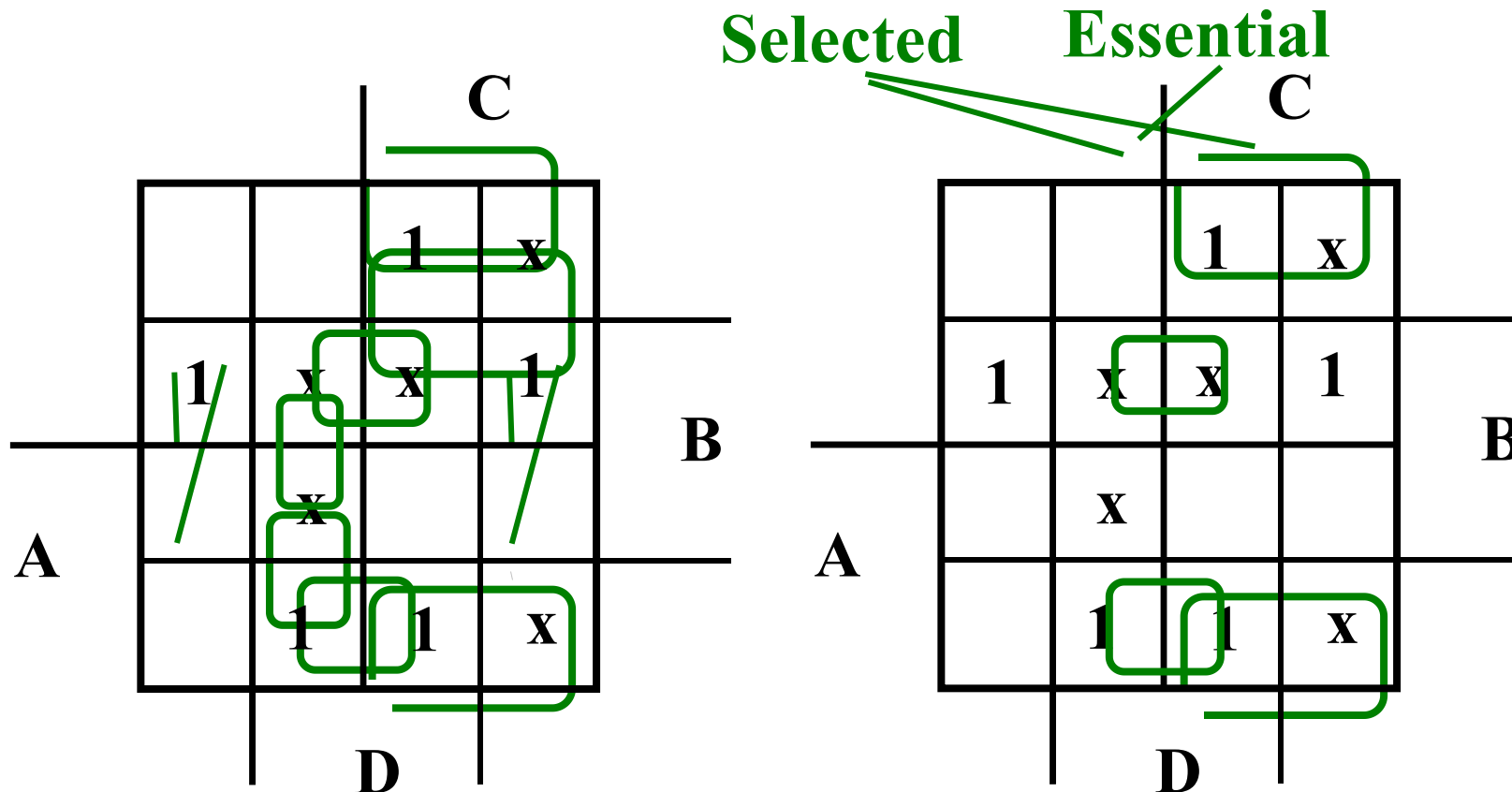
- Implicant corresponding to the largest circle in a K-map, i.e., is a product term obtained by combining the maximum possible number of adjacent squares in the map into a rectangle with the number of squares a power of 2.

- **Essential prime implicant**

- A prime implicant is called an Essential Prime Implicant if it is the only prime implicant that covers (includes) one or more minterms.

# Selection Rule Example with Don't Cares

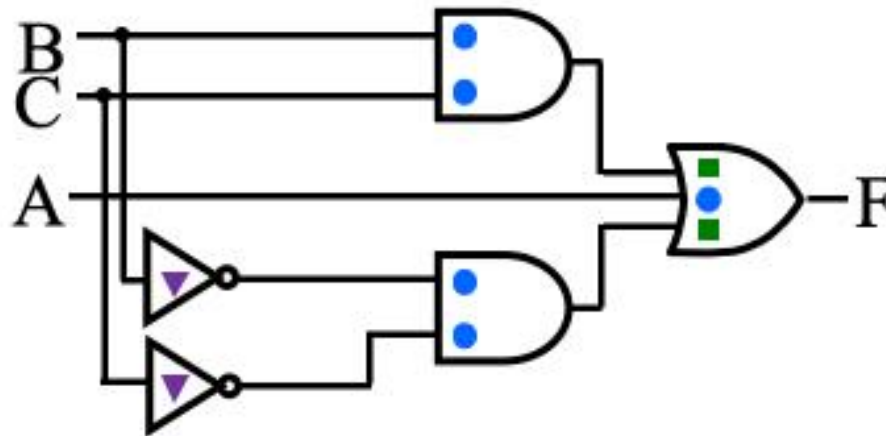
- Simplify  $F(A, B, C, D)$  given on the K-map



Minterms covered by essential prime implicants

## Cost Criteria (continued)

- **Example 1:**  $GN = G + 2 = 9$
- $F = A + BC + \overline{B}\overline{C}$   $L = 5$
- $G = L + 2 = 7$



- L (literal count) counts the AND inputs and the single literal OR input
- G (gate input count) adds the remaining OR gate inputs
- GN (gate input count with NOTs) adds the inverter inputs



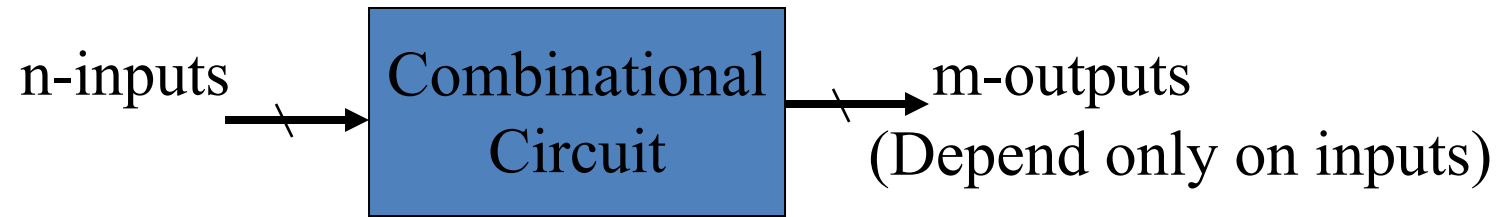
# HDL-based Design Flow

---

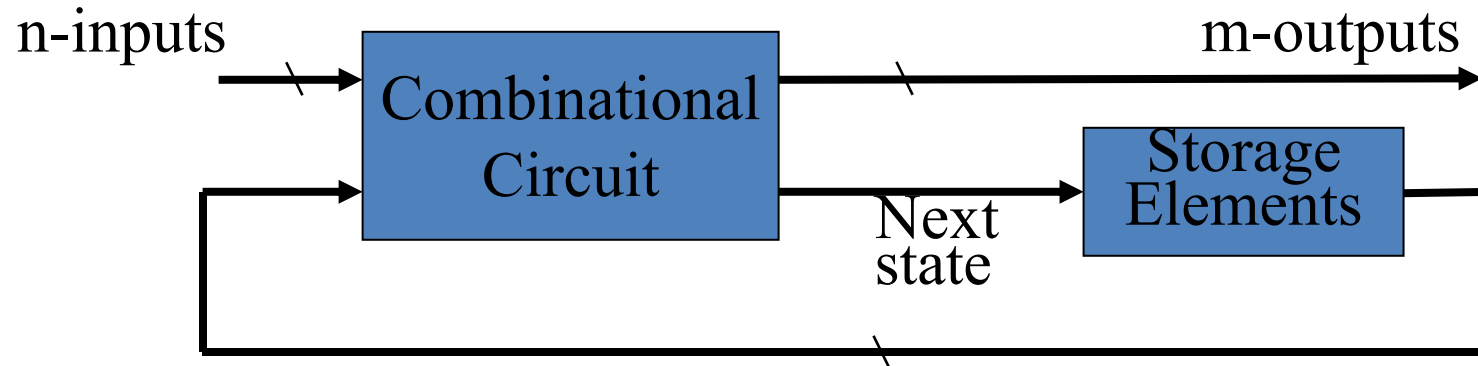
- Logic design with HDL
- Simulation
- Synthesis
- Physical Design
- Final step

# Combinational vs. Sequential Circuits

---



**Combinational Circuit**



**Sequential Circuit**

# How to Design Combinational Logic Circuits

---

- Design procedure
  - Determine inputs, outputs and their logic relationship
    - Truth table
  - Determine logic functions and perform optimization
    - Algebra or K-map
  - Draw logic circuit diagram and timing diagram
    - Timing analysis

# Some Classic/Basic Designs

---

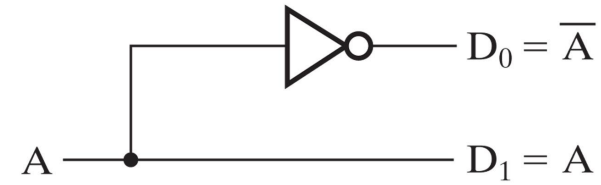
- Encoder and decoder
- Multiplexer and demultiplexer
- Half adder and full adder

# Decoder Examples

- 1-to-2-Line Decoder

| A | D <sub>0</sub> | D <sub>1</sub> |
|---|----------------|----------------|
| 0 | 1              | 0              |
| 1 | 0              | 1              |

(a)



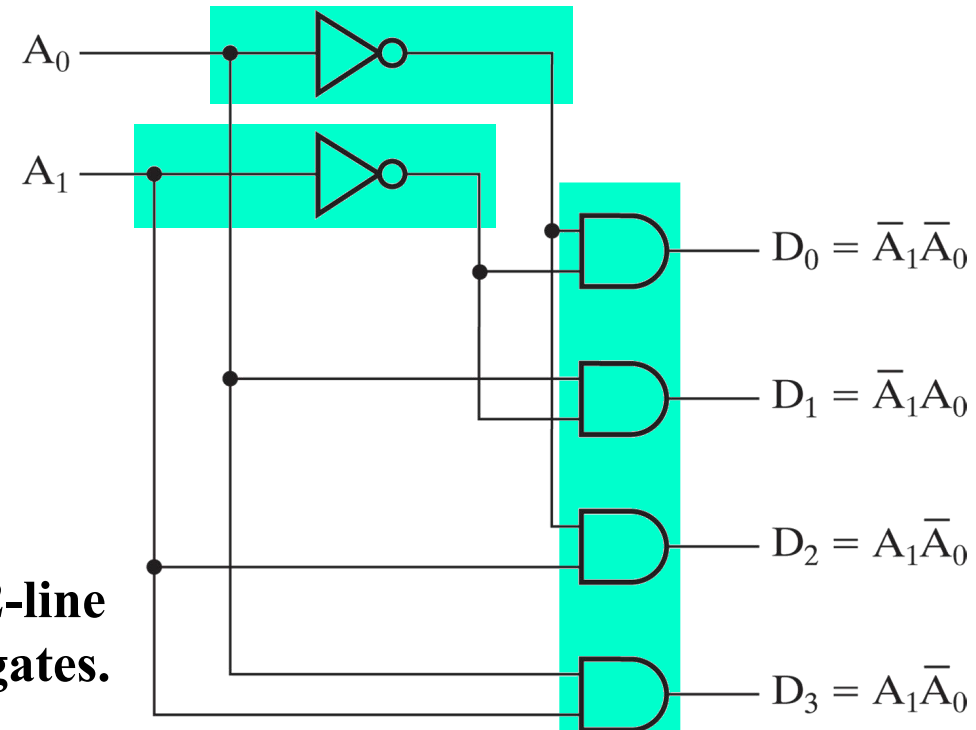
(b)

Copyright ©2016 Pearson Education, All Rights Reserved

- 2-to-4-Line Decoder

| A <sub>1</sub> | A <sub>0</sub> | D <sub>0</sub> | D <sub>1</sub> | D <sub>2</sub> | D <sub>3</sub> |
|----------------|----------------|----------------|----------------|----------------|----------------|
| 0              | 0              | 1              | 0              | 0              | 0              |
| 0              | 1              | 0              | 1              | 0              | 0              |
| 1              | 0              | 0              | 0              | 1              | 0              |
| 1              | 1              | 0              | 0              | 0              | 1              |

(a)

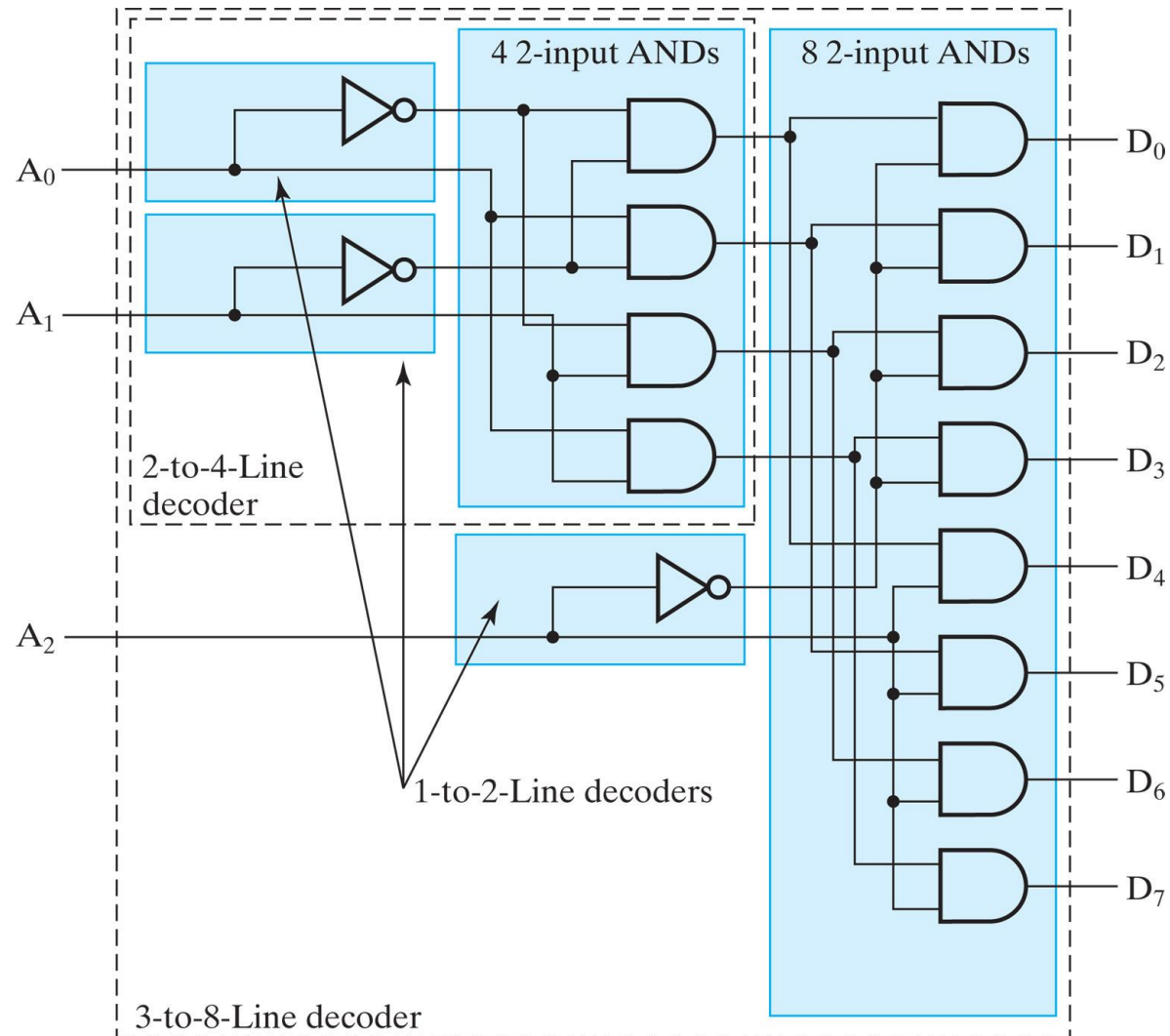


(b)

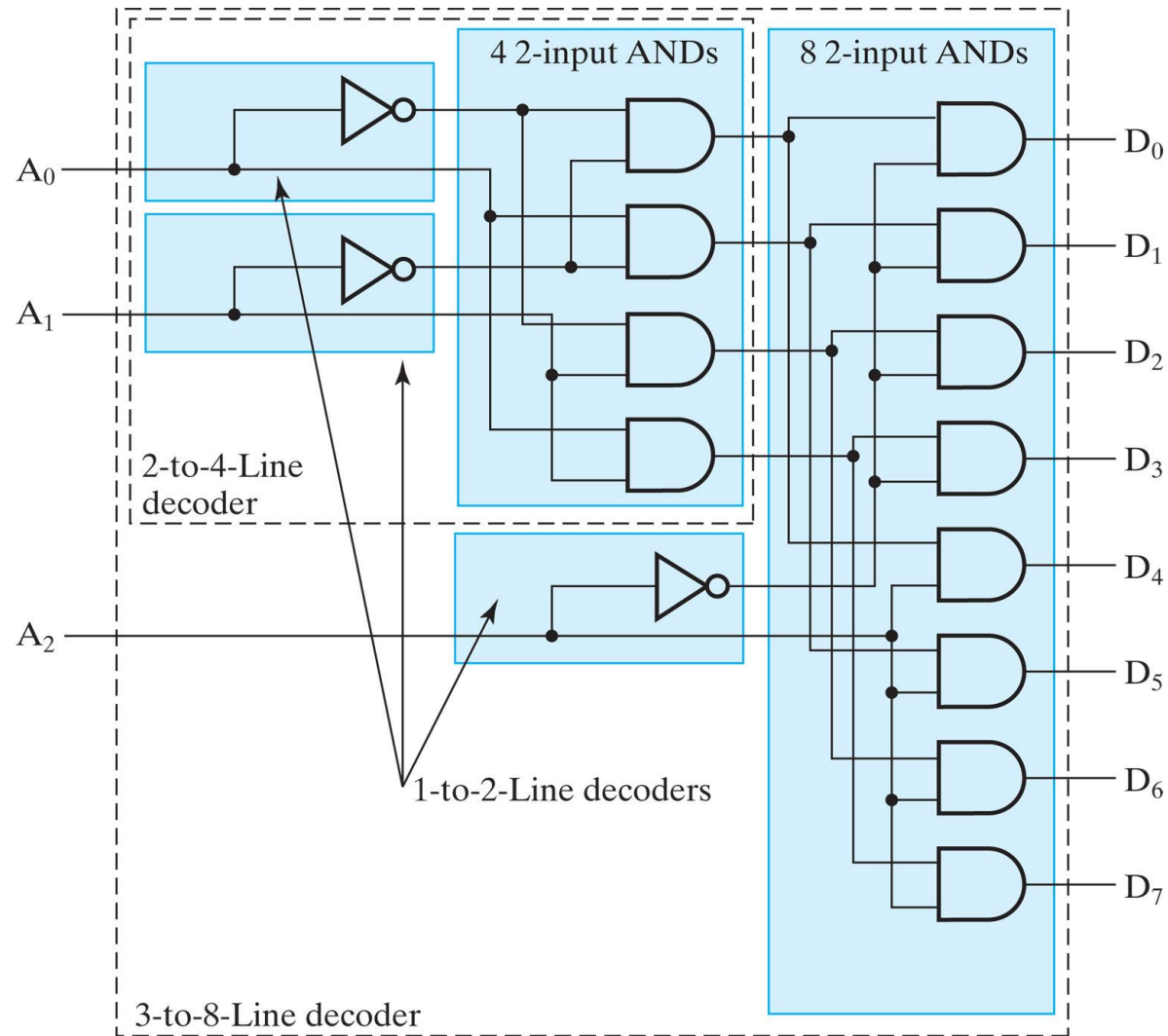
The 2-4-line made up of 2 1-to-2-line decoders and 4 (**2-inputs**) AND gates.

Copyright ©2016 Pearson Education, All Rights Reserved

# Decoder Expansion: 3-8 Decoder (cont'd)

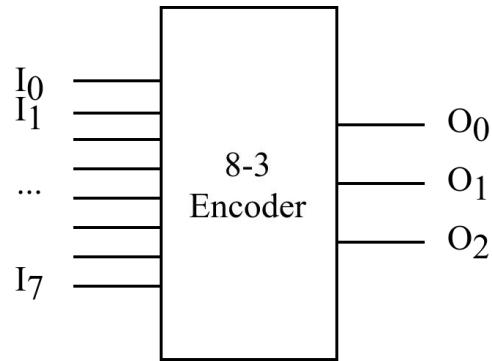


# Decoder Expansion: 3-8 Decoder (cont'd)



# Classic Designs: Encoder

## ■ 8-to-3-line encoder



$$O_0 = I_4 + I_5 + I_6 + I_7$$

$$O_1 = I_2 + I_3 + I_6 + I_7$$

$$O_2 = I_1 + I_2 + I_5 + I_7$$

↓  
OR gates!

| Input |       |       |       |       |       |       |       | Output |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|--------|-------|-------|
| $I_0$ | $I_1$ | $I_2$ | $I_3$ | $I_4$ | $I_5$ | $I_6$ | $I_7$ | $O_0$  | $O_1$ | $O_2$ |
| 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 0      | 0     | 0     |
| 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0     | 0      | 0     | 1     |
| 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0     | 0      | 1     | 0     |
| 0     | 0     | 0     | 1     | 0     | 0     | 0     | 0     | 0      | 1     | 1     |
| 0     | 0     | 0     | 0     | 1     | 0     | 0     | 0     | 1      | 0     | 0     |
| 0     | 0     | 0     | 0     | 0     | 1     | 0     | 0     | 1      | 0     | 1     |
| 0     | 0     | 0     | 0     | 0     | 0     | 1     | 0     | 1      | 1     | 0     |
| 0     | 0     | 0     | 0     | 0     | 0     | 0     | 1     | 1      | 1     | 1     |

**However, there exists two issues that may make the encoder just designed does not work:**

1. If more than one input value is 1 (e.g.,  $I_3$  &  $I_6$  are both 1)
2. All inputs are 0 vs.  $I_0 = 1$



# Priority Encoder Example

- Priority encoder with 4 inputs ( $D_3, D_2, D_1, D_0$ ) - highest priority to most significant 1 present - Code outputs A1, A0 and V where V indicates at least one 1 present.

| No. of Min-terms/Row | Inputs |    |    |    | Outputs |    |   |
|----------------------|--------|----|----|----|---------|----|---|
|                      | D3     | D2 | D1 | D0 | A1      | A0 | V |
| 1                    | 0      | 0  | 0  | 0  | X       | X  | 0 |
| 1                    | 0      | 0  | 0  | 1  | 0       | 0  | 1 |
| 2                    | 0      | 0  | 1  | X  | 0       | 1  | 1 |
| 4                    | 0      | 1  | X  | X  | 1       | 0  | 1 |
| 8                    | 1      | X  | X  | X  | 1       | 1  | 1 |

- Xs in input part of table represent 0 or 1; thus table entries correspond to product terms instead of minterms. The column on the left shows that all 16 minterms are present in the product terms in the table

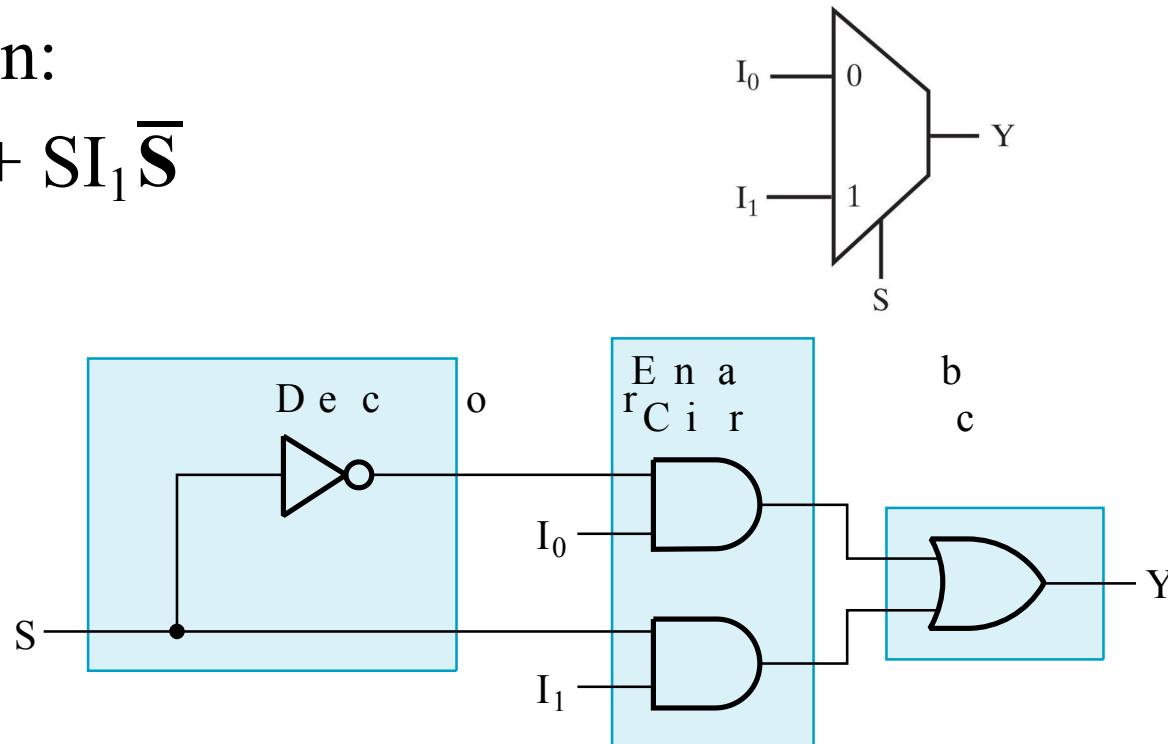
# 2-to-1-Line Multiplexer

- Since  $2 = 2^1$ ,  $n = 1$
- The single selection variable  $S$  has two values:
  - $S = 0$  selects input  $I_0$
  - $S = 1$  selects input  $I_1$

- The equation:

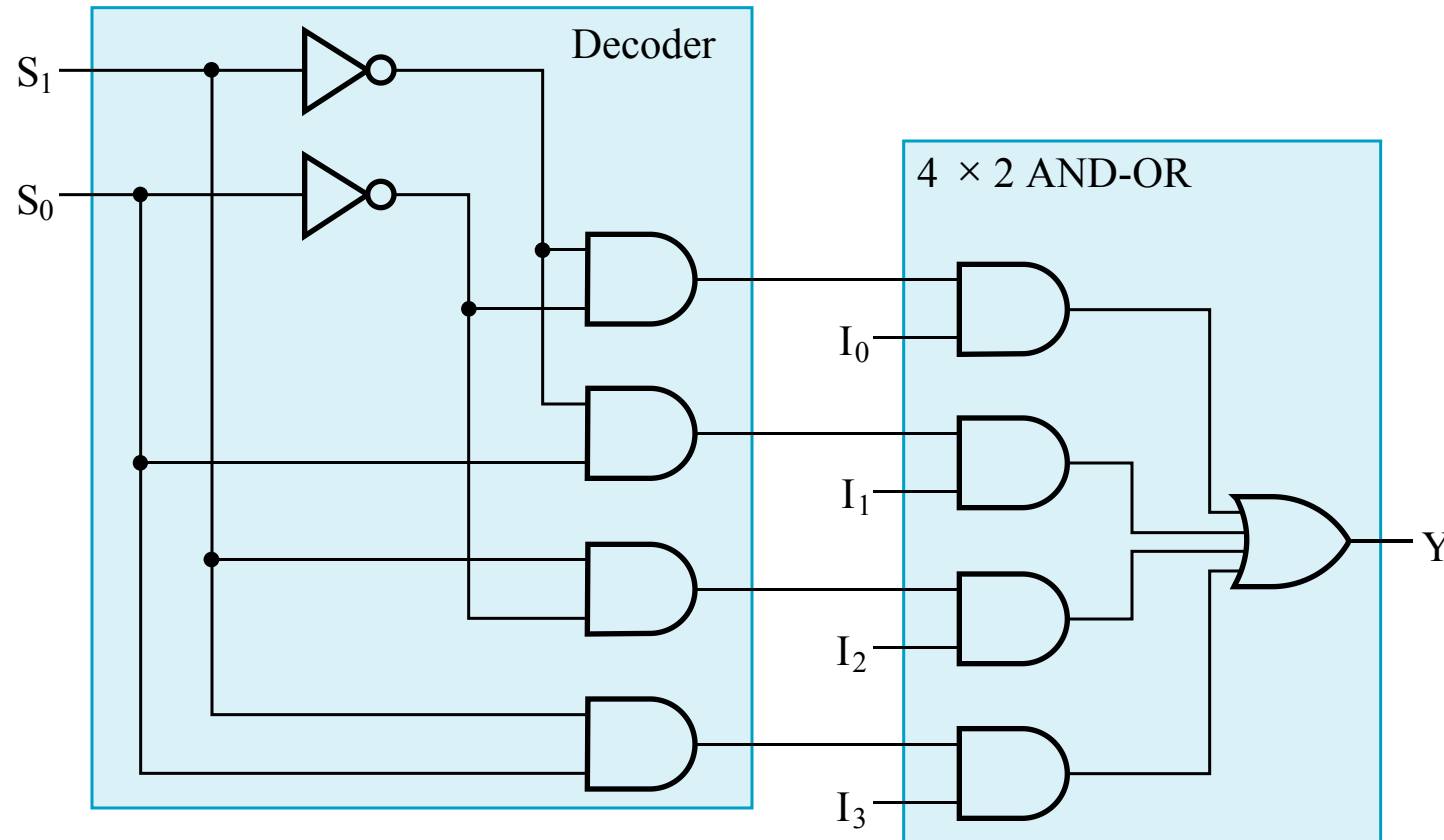
$$Y = I_0 + SI_1\bar{S}$$

- The circuit:



# 4-to-1-line Multiplexer (cont'd)

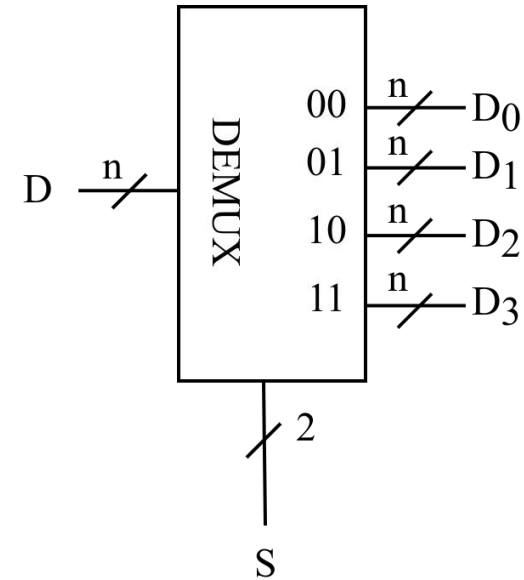
- 2-to- $2^2$ -line decoder
- $2^2 \times 2$  AND-OR



# Classic Designs: Demultiplexer

- Demultiplexer (DEMUX, DMUX)

| Input |       | Output |       |       |       |
|-------|-------|--------|-------|-------|-------|
| $S_0$ | $S_1$ | $D_0$  | $D_1$ | $D_2$ | $D_3$ |
| 0     | 0     | D      | 0     | 0     | 0     |
| 0     | 1     | 0      | D     | 0     | 0     |
| 1     | 0     | 0      | 0     | D     | 0     |
| 1     | 1     | 0      | 0     | 0     | D     |



# Classic Designs: Full Adder

- A full adder is similar to a half adder, but includes a carry-in bit from lower stages.
- Like the half-adder, it computes a sum bit **F** and a carry bit

**C<sub>in</sub>**

- For a carry-in (**C<sub>in</sub>**) of 0, it is the same as the half-adder:

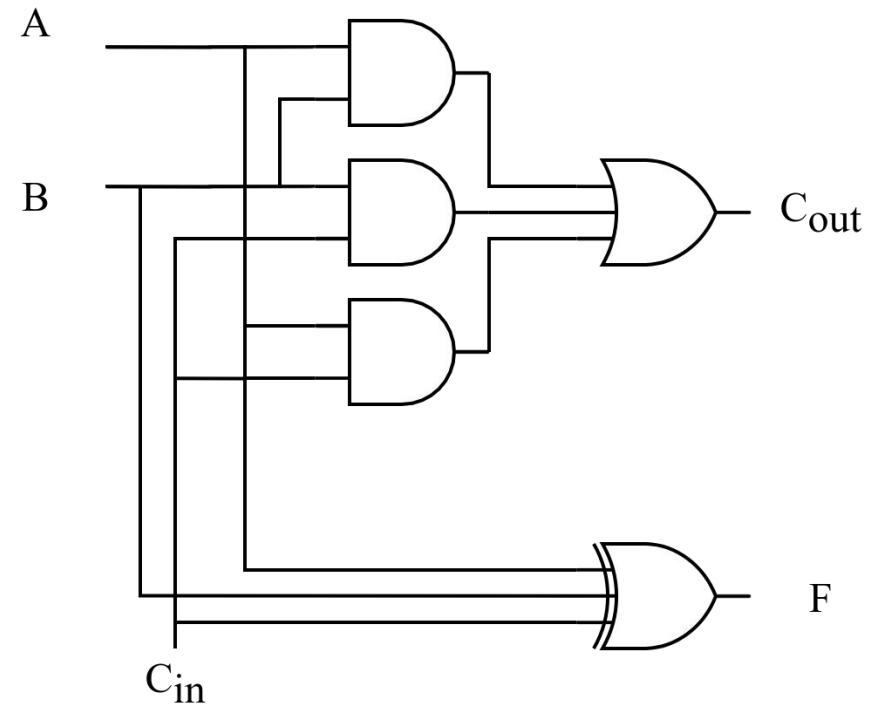
|                          |            |            |            |            |
|--------------------------|------------|------------|------------|------------|
| <b>C<sub>in</sub></b>    | <b>0</b>   | <b>0</b>   | <b>0</b>   | <b>0</b>   |
| <b>A</b>                 | <b>0</b>   | <b>0</b>   | <b>1</b>   | <b>1</b>   |
| <b>+ B</b>               | <b>+ 0</b> | <b>+ 1</b> | <b>+ 0</b> | <b>+ 1</b> |
| <b>C<sub>out</sub> F</b> | <b>0 0</b> | <b>0 1</b> | <b>0 1</b> | <b>1 0</b> |

- For a carry-in (**C<sub>in</sub>**) of 1:

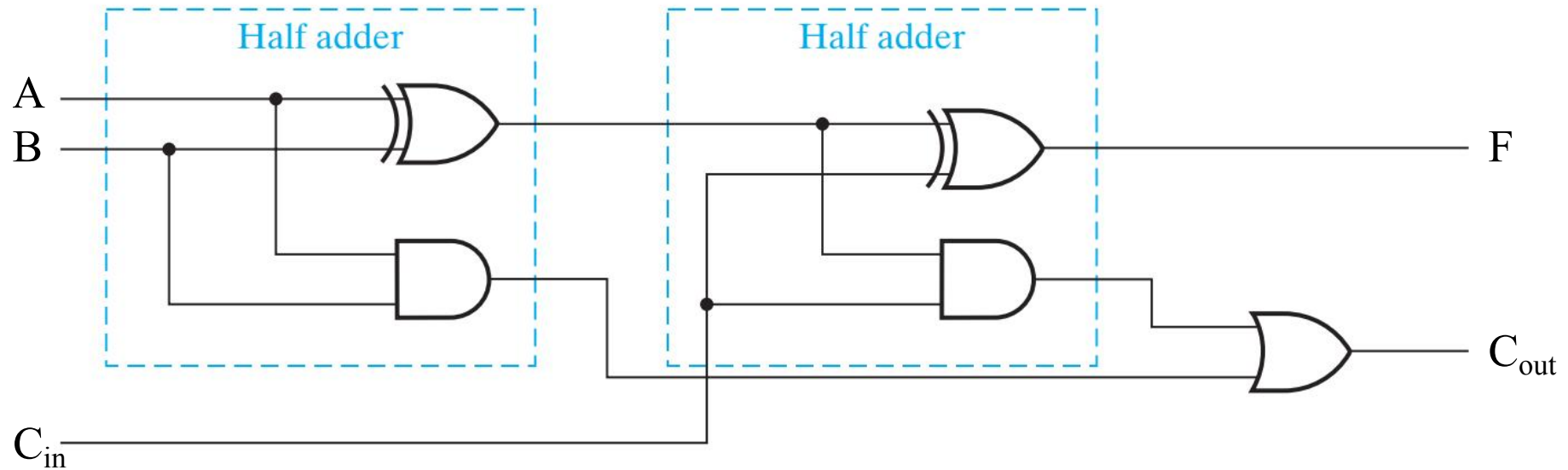
|                          |            |            |            |            |
|--------------------------|------------|------------|------------|------------|
| <b>C<sub>in</sub></b>    | <b>1</b>   | <b>1</b>   | <b>1</b>   | <b>1</b>   |
| <b>A</b>                 | <b>0</b>   | <b>0</b>   | <b>1</b>   | <b>1</b>   |
| <b>+ B</b>               | <b>+ 0</b> | <b>+ 1</b> | <b>+ 0</b> | <b>+ 1</b> |
| <b>C<sub>out</sub> F</b> | <b>0 1</b> | <b>1 0</b> | <b>1 0</b> | <b>1 1</b> |

# Classic Designs: Full Adder (cont'd)

| Input |   |                 | Output |                  |
|-------|---|-----------------|--------|------------------|
| A     | B | C <sub>in</sub> | F      | C <sub>out</sub> |
| 0     | 0 | 0               | 0      | 0                |
| 0     | 0 | 1               | 1      | 0                |
| 0     | 1 | 0               | 1      | 0                |
| 0     | 1 | 1               | 0      | 1                |
| 1     | 0 | 0               | 1      | 0                |
| 1     | 0 | 1               | 0      | 1                |
| 1     | 1 | 0               | 0      | 1                |
| 1     | 1 | 1               | 1      | 1                |

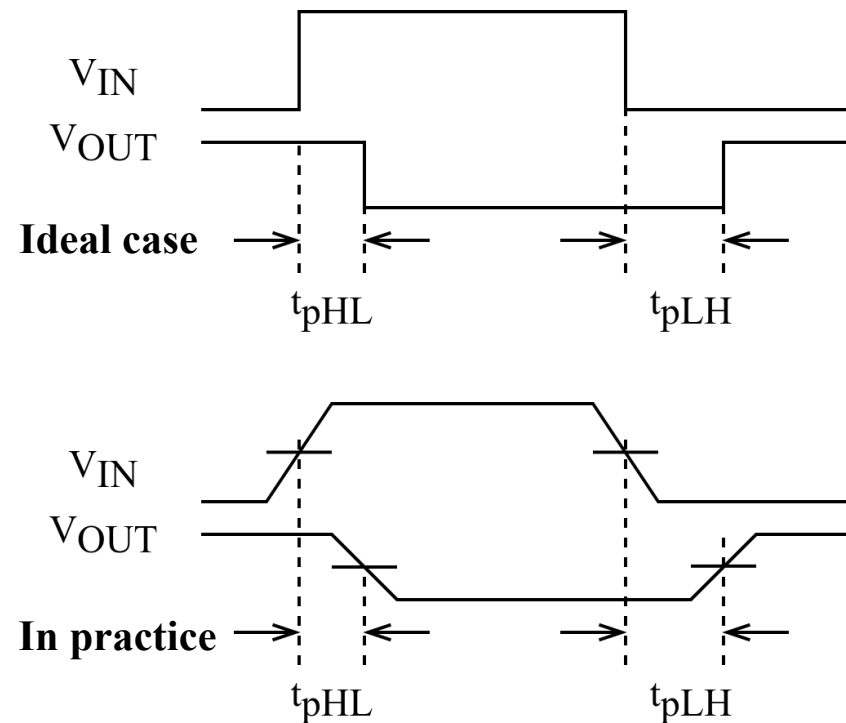


# Classic Designs: Full Adder (cont'd)

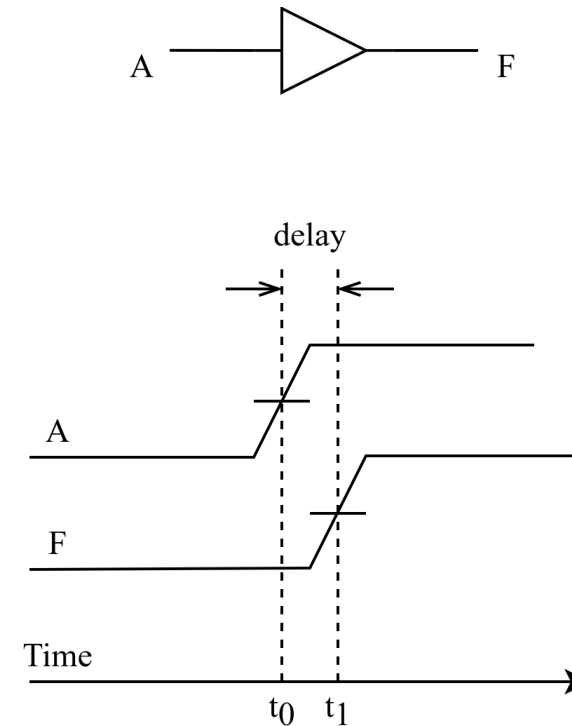


# Timing Analysis

- Circuit delay



Delay of the inverter

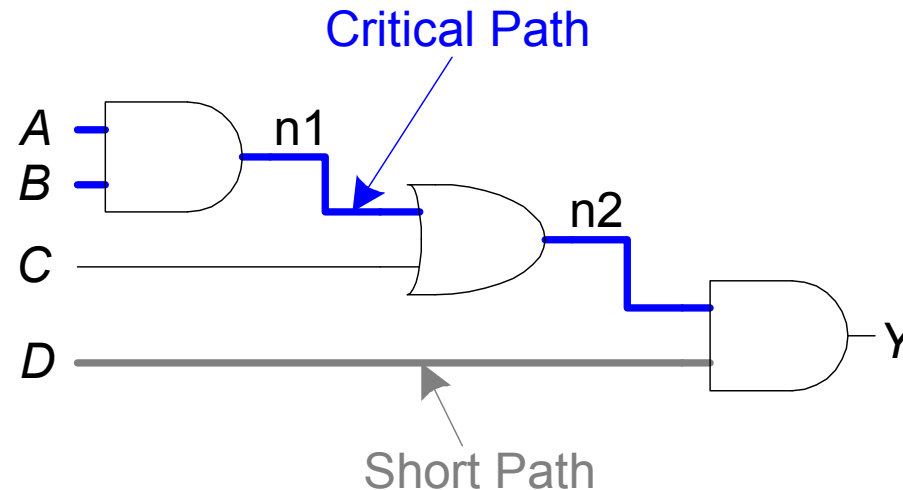


Rise-time delay of the transmission gate



# Timing Analysis (cont'd)

- The critical (longest) path
  - $T_{pd}$  of the circuit =  $\sum$  all  $T_{pd}$  of circuit elements along the critical path
- The shortest path
  - $T_{cd}$  of the circuit =  $\sum$  all  $T_{cd}$  of circuit elements along the shortest path

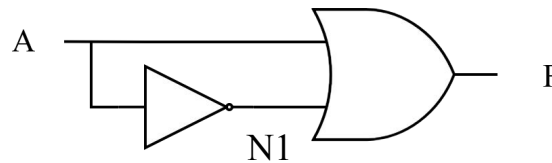


$$T_{pd} = 2T_{pd\_AND} + T_{pd\_OR}$$

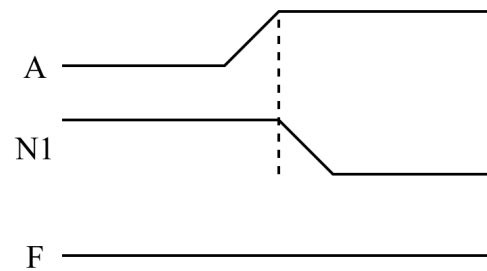
$$T_{cd} = T_{cd\_AND}$$

# Timing Analysis (cont'd)

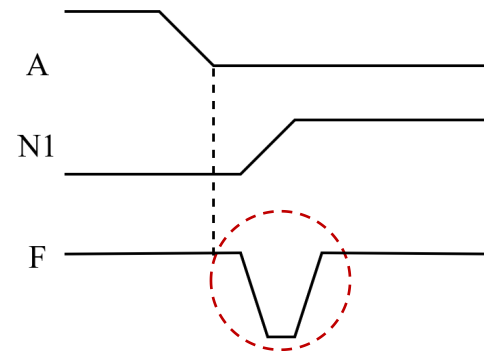
- Race hazard
  - Glitch: when a single input change causes multiple output changes



**Race hazard in a circuit**



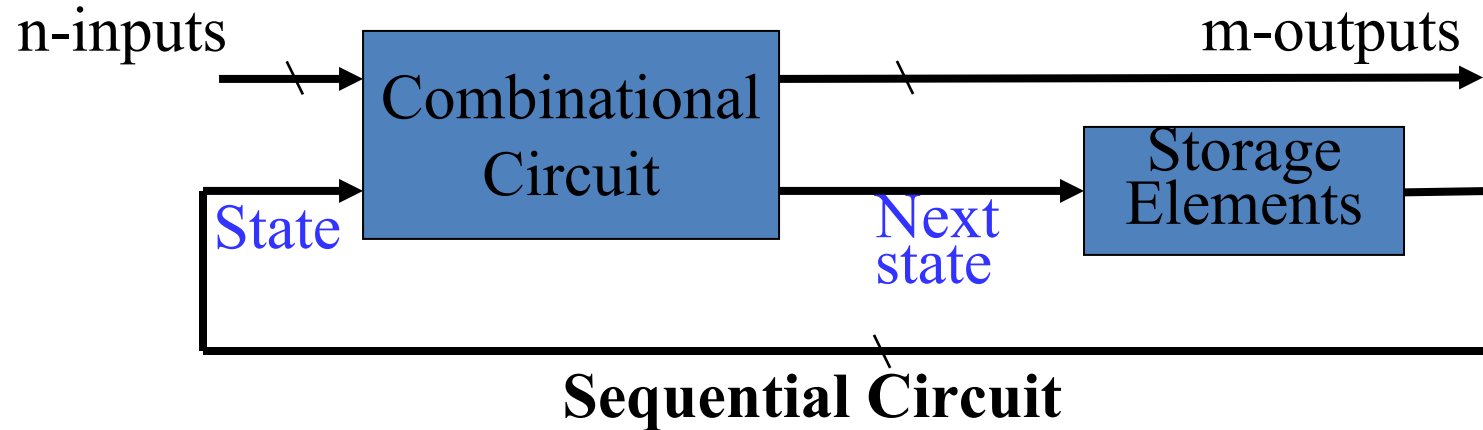
**No glitch**



**Glitch**

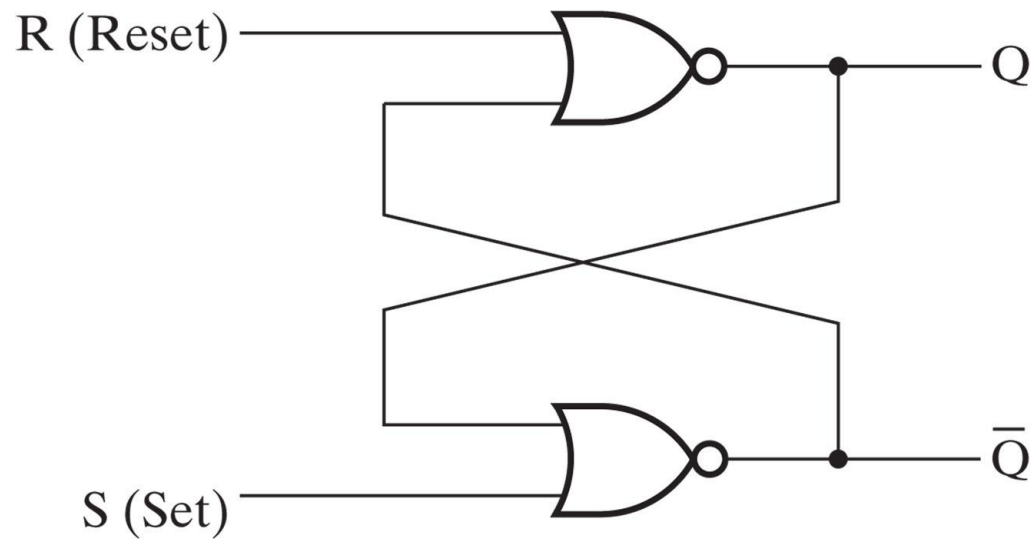
# Sequential Circuits: Storage Elements

---



- Bistable circuits
  - Latches
  - Flip-flops

# SR Latch with NOR Gates



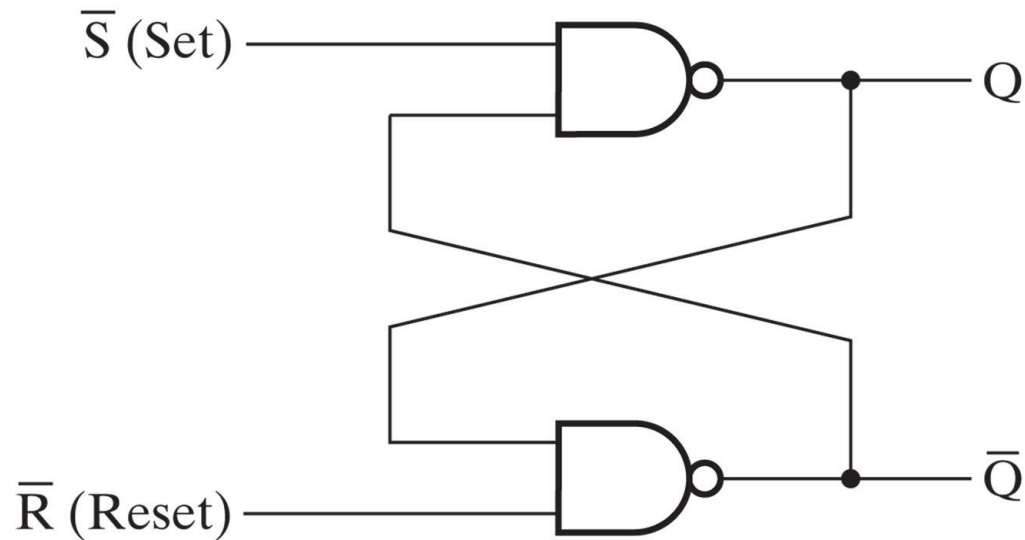
(a) Logic diagram

| S | R | Q | $\bar{Q}$ |             |
|---|---|---|-----------|-------------|
| 1 | 0 | 1 | 0         | Set state   |
| 0 | 0 | 1 | 0         |             |
| 0 | 1 | 0 | 1         | Reset state |
| 0 | 0 | 0 | 1         |             |
| 1 | 1 | 0 | 0         | Undefined   |

(b) Function table

Copyright ©2016 Pearson Education, All Rights Reserved

# $\bar{S}\bar{R}$ Latch with NAND Gates



(a) Logic diagram

| $\bar{S}$ | $\bar{R}$ | $Q$ | $\bar{Q}$ |             |
|-----------|-----------|-----|-----------|-------------|
| 0         | 1         | 1   | 0         | Set state   |
| 1         | 1         | 1   | 0         |             |
| 1         | 0         | 0   | 1         | Reset state |
| 1         | 1         | 0   | 1         |             |
| 0         | 0         | 1   | 1         | Undefined   |

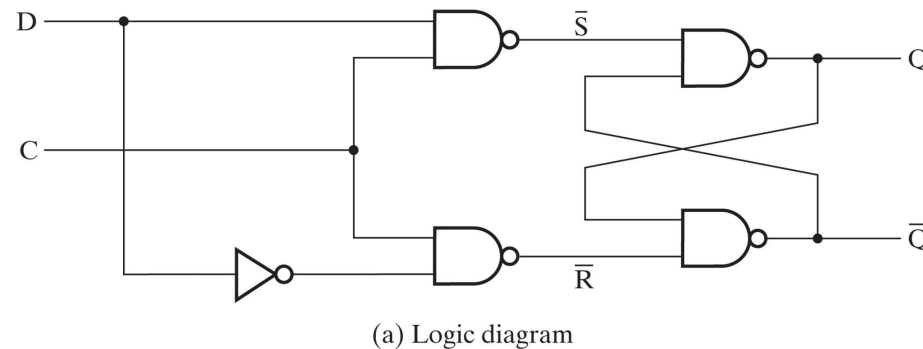
(b) Function table

Copyright ©2016 Pearson Education, All Rights Reserved

# D Latch

- D (Data) latch

- Adding an inverter to the SR latch, gives the D latch.



| C | D | Next state of Q    |
|---|---|--------------------|
| 0 | X | No change          |
| 1 | 0 | Q = 0; Reset state |
| 1 | 1 | Q = 1; Set state   |

(b) Function table

Copyright ©2016 Pearson Education, All Rights Reserved

- Two inputs: **C, D**

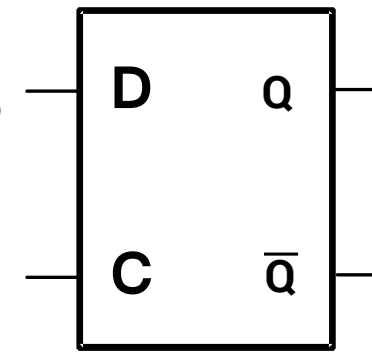
- C: controls when the output changes
- D (the data input): controls what the output changes to

- Function

- When **C = 1**, D passes through to Q (*transparent*)
- When **C = 0**, Q holds its previous value (*opaque*)

- Avoids invalid case when

- $Q \neq \bar{Q}$

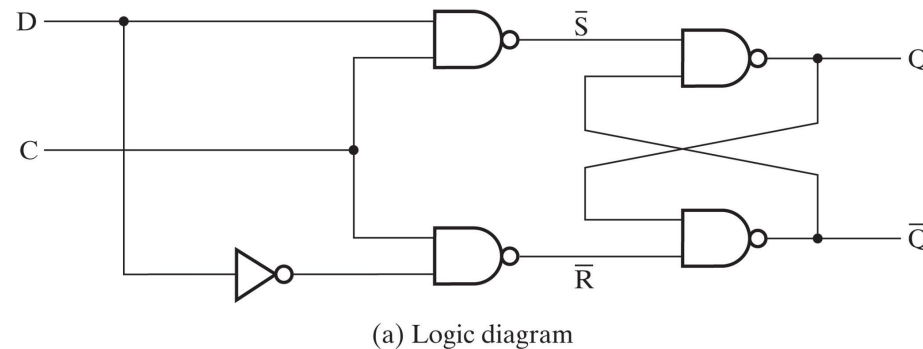


The graphic symbol

# D Latch

- D (Data) latch

- Adding an inverter to the SR latch, gives the D latch.



| C | D | Next state of Q    |
|---|---|--------------------|
| 0 | X | No change          |
| 1 | 0 | Q = 0; Reset state |
| 1 | 1 | Q = 1; Set state   |

(b) Function table

Copyright ©2016 Pearson Education, All Rights Reserved

- Two inputs: **C, D**

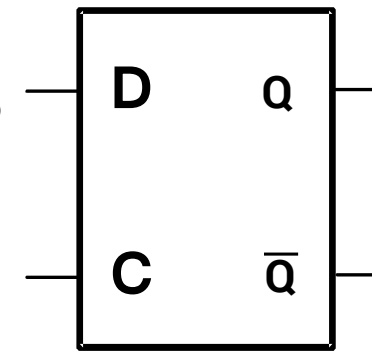
- C: controls when the output changes
- D (the data input): controls what the output changes to

- Function

- When **C = 1**, D passes through to Q (*transparent*)
- When **C = 0**, Q holds its previous value (*opaque*)

- Avoids invalid case when

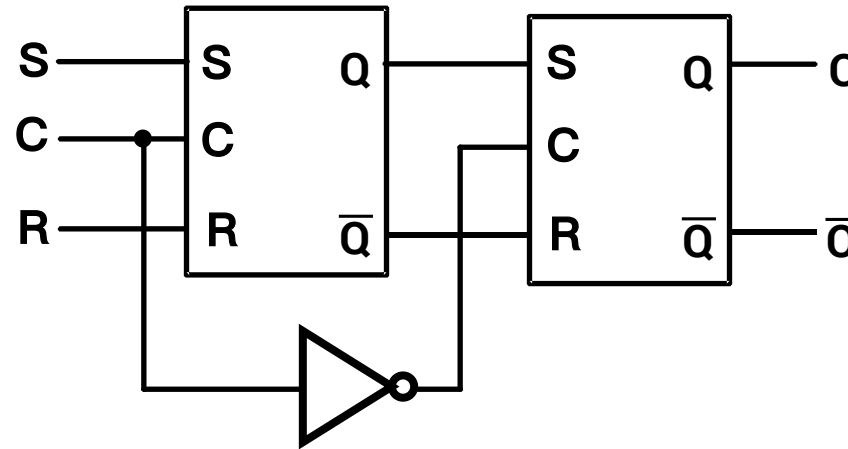
- $Q \neq \bar{Q}$



The graphic symbol

# Pulse-Triggered Flip-Flop

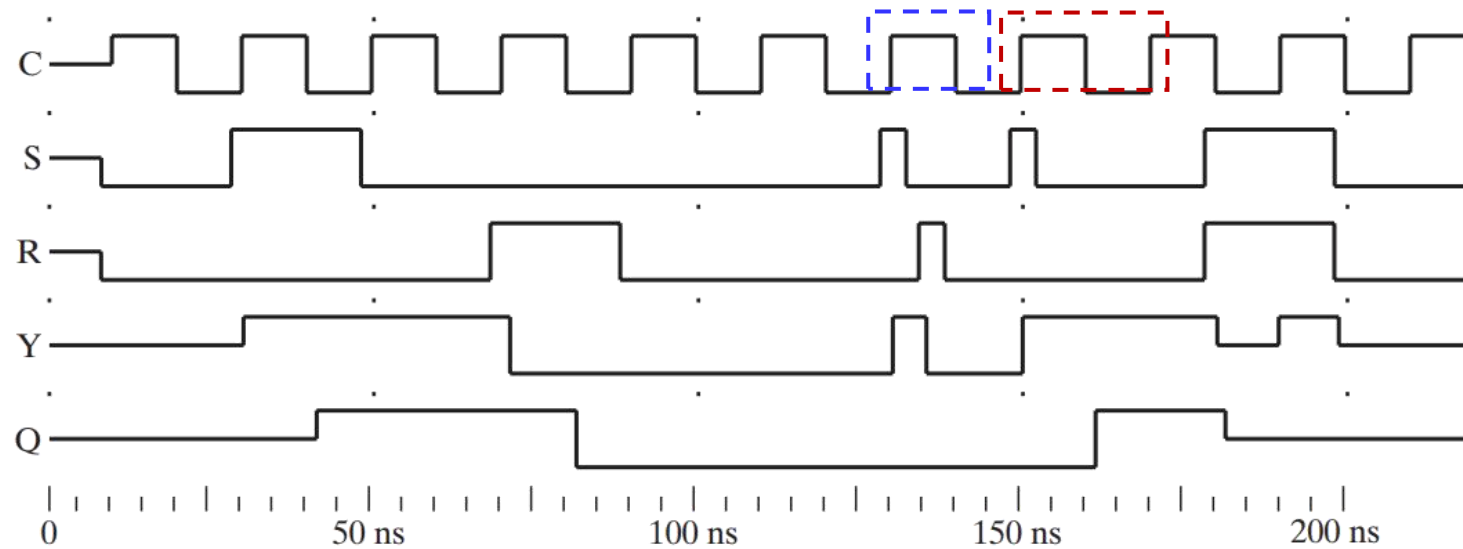
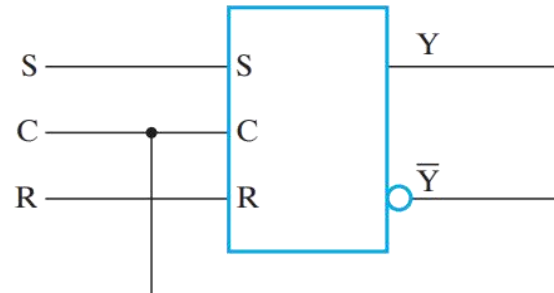
- SR flip-flop
  - Consists of two clocked SR latches in series with the clock on the second latch inverted



- The input is observed by the first latch with  $C = 1$
- The output is changed by the second latch with  $C = 0$
- The path from input to output is broken by the difference in clocking values ( $C = 1$  and  $C = 0$ ).
- The behavior demonstrated by the example with D driven by Y given previously is prevented since the clock must change from 1 to 0 before a change in Y based on D can occur.

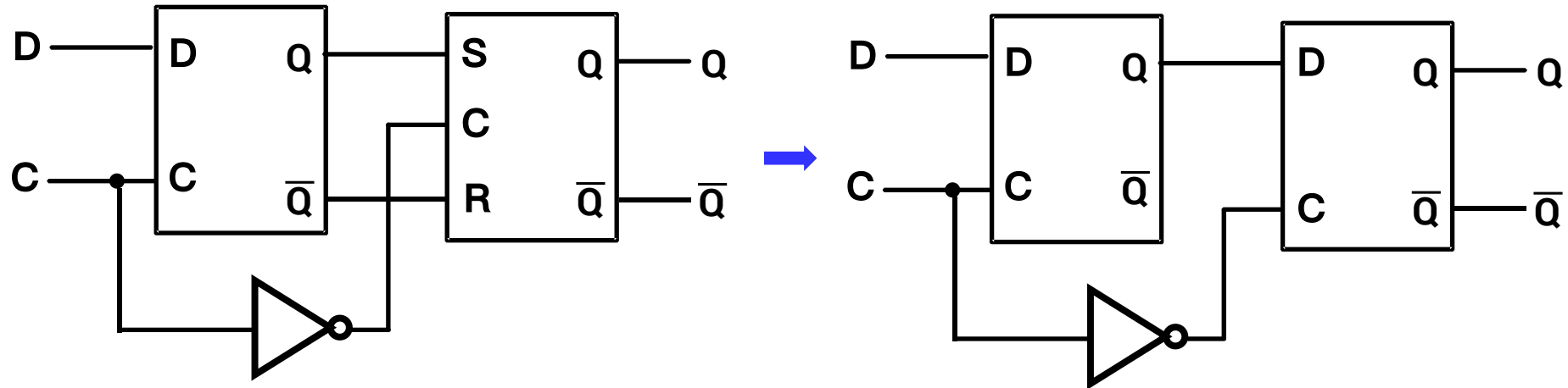


# 1's Catching



# Edge-Triggered D Flip-Flop

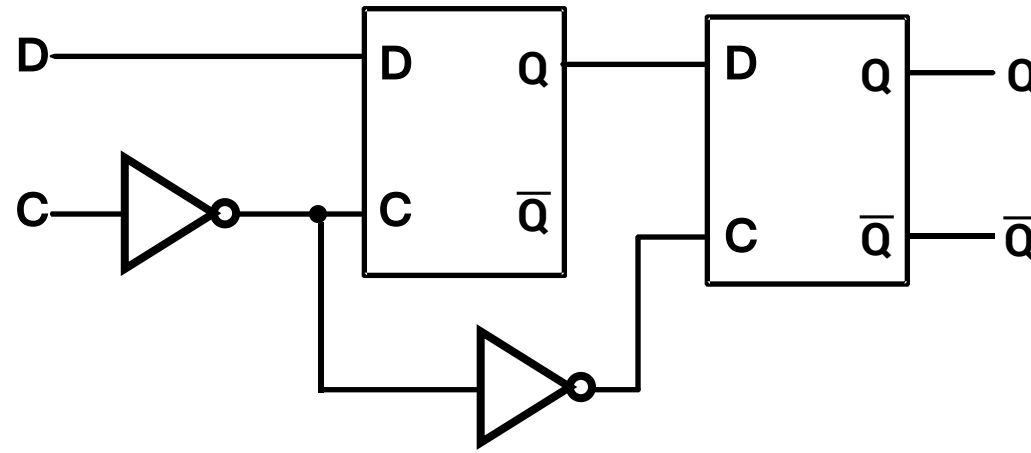
- The edge-triggered D flip-flop can be formed by:
  - Replacing the first clocked SR latch with a clocked D latch or
  - Adding a D input and inverter to a master-slave SR flip-flop



- The delay of the SR master-slave flip-flop can be avoided since the 1's-catching behavior is not present with D replacing S and R inputs
- The change of the D flip-flop output is associated with the negative edge at the end of the pulse, i.e., ***negative-edge triggered*** flip-flop

# Positive-Edge Triggered D Flip-Flop

- Formed by adding inverter to clock input

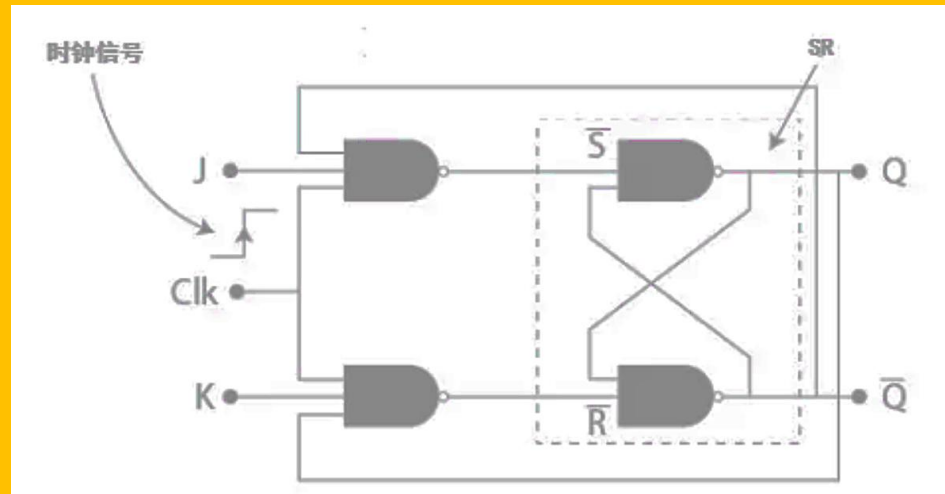


- Q changes to the value on D applied at the positive clock edge within timing constraints to be specified
- Our choice as the **standard flip-flop** for most sequential circuits

# JK (Jack Kilby) Flip-Flop

## ■ Behavior

- Same as SR flip-flop with J analogous to S and K analogous to R
- **Except that  $J = K = 1$  is allowed**, and
- For  $J = K = 1$ , the flip-flop changes to *the opposite state*
- Has the same “1s catching” behavior as SR flip-flop
- If the master changes to the wrong state, that state will be passed to the slave
- E.g., if master falsely set by  $J = 1, K = 1$  cannot reset it during the current clock cycle



# T Flip-Flop

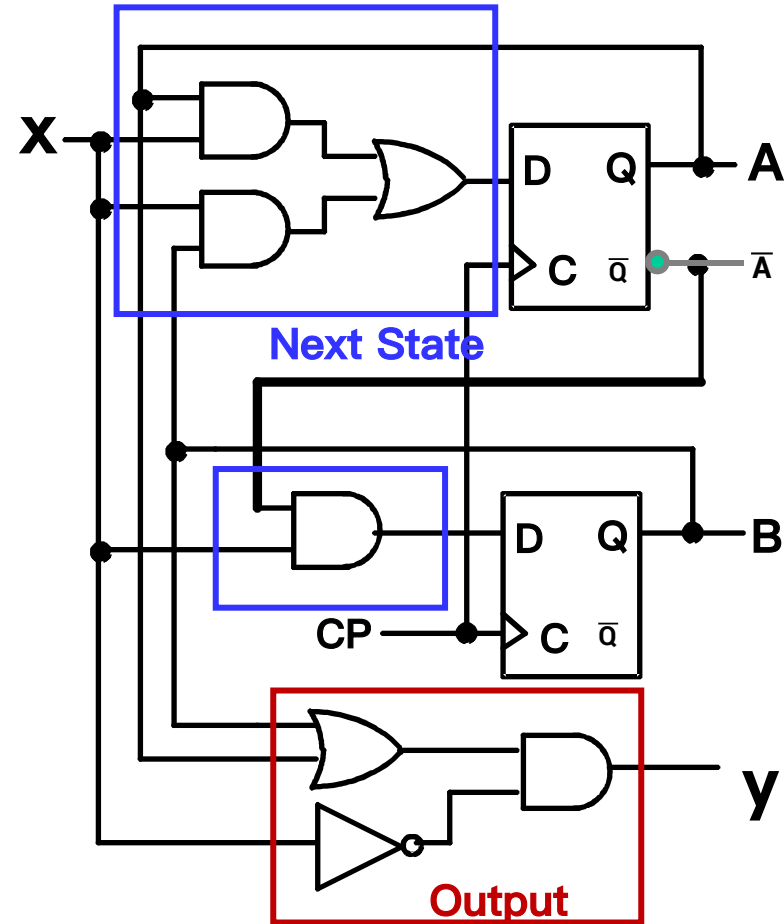
---

- Behavior
  - Has a single input T
    - For  $T = 0$ , no change to state
    - For  $T = 1$ , changes to opposite state
- Same as a JK flip-flop with  $J = K = T$
- Has the same “1’s catching” behavior as JK flip-flop
- Cannot be initialized to a known state using the T input
  - Reset (asynchronous or synchronous) essential

# Example: Boolean Equation

- Boolean equations for the functions:

- $A(t+1) = A(t)x(t) + B(t)x(t)$
- $B(t+1) = \bar{A}(t)x(t)$
- $y(t) = \bar{x}(t)(B(t) + A(t))$



# Example: State Table

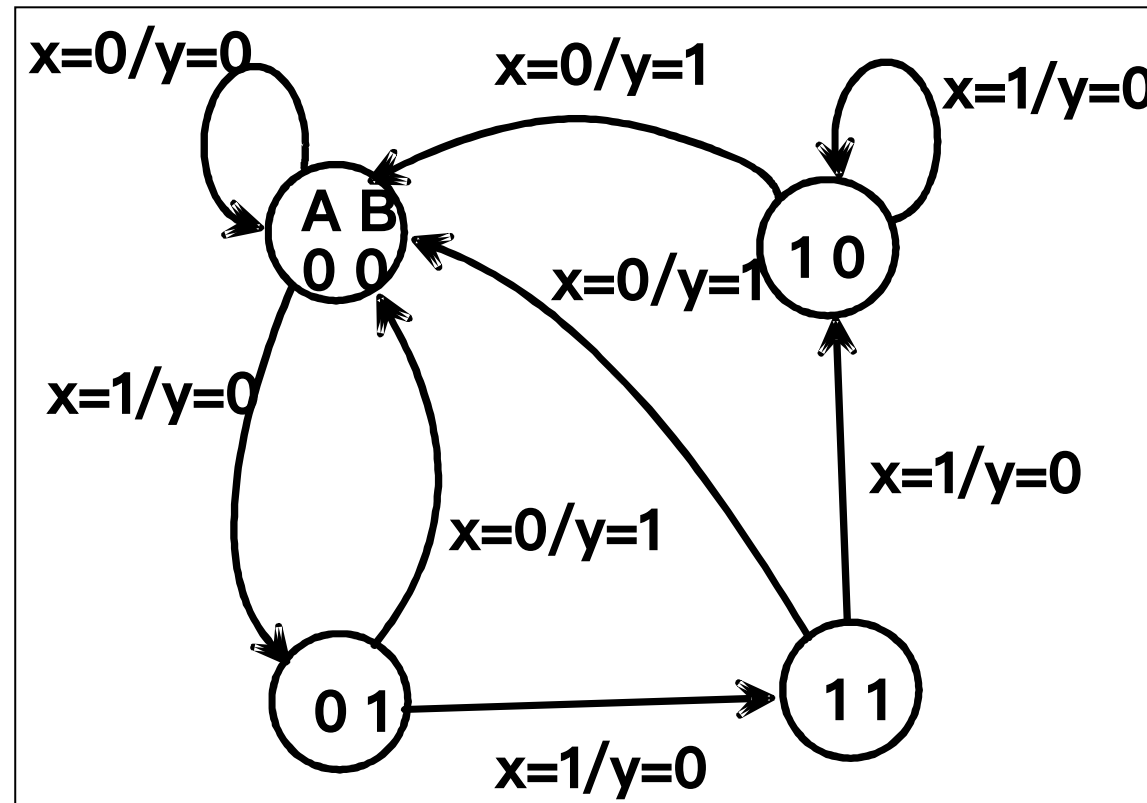
- Boolean equations for the functions:

- $A(t+1) = A(t)x(t) + B(t)x(t)$
- $B(t+1) = \bar{A}(t)x(t)$
- $y(t) = \bar{x}(t)(B(t) + A(t))$

| Present State |      | Input | Next State |        | Output |
|---------------|------|-------|------------|--------|--------|
| A(t)          | B(t) | x(t)  | A(t+1)     | B(t+1) | y(t)   |
| 0             | 0    | 0     | 0          | 0      | 0      |
| 0             | 0    | 1     | 0          | 1      | 0      |
| 0             | 1    | 0     | 0          | 0      | 1      |
| 0             | 1    | 1     | 1          | 1      | 0      |
| 1             | 0    | 0     | 0          | 0      | 1      |
| 1             | 0    | 1     | 1          | 0      | 0      |
| 1             | 1    | 0     | 0          | 0      | 1      |
| 1             | 1    | 1     | 1          | 0      | 0      |

# Example: State Diagram

- Diagram gets confusing for large circuits
- For small circuits, usually easier to understand than the state table





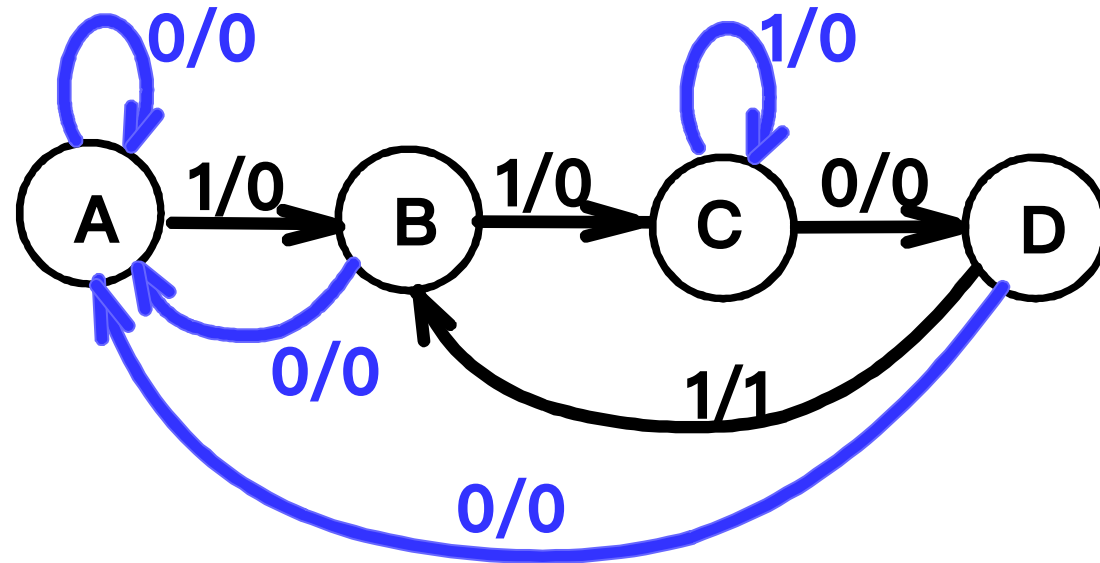
# Sequence Recognizer Procedure (cont'd)

---

- Example: Recognize the sequence 1101
  - Note that the sequence 1111101 contains 1101 and "11" is a proper sub-sequence of 1101.
- Thus, the sequential machine must remember that the first two one's have occurred as it receives another symbol.
- Also, the sequence 1101101 contains 1101 as both an initial subsequence and a final subsequence with some overlap, i. e., 1101101 or 1101101.
- And, the 1 in the middle, 1101101, is in both subsequences.
- The sequence 1101 must be recognized each time it occurs in the input sequence.

## Example: Recognize 1101 (cont'd)

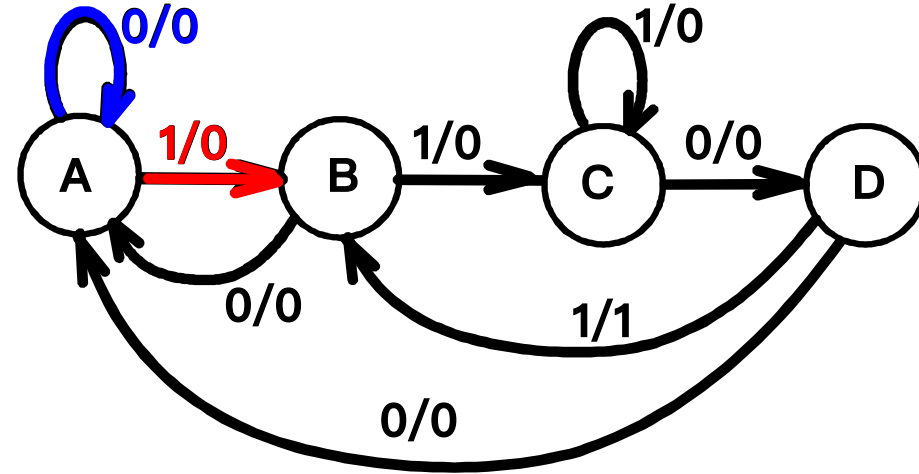
- State transition arcs must represent the fact that an input subsequence has occurred. Thus, we get:



- Note that the 1 arc from state C to state C implies that State C means *two or more 1's have occurred*.

# Formulation: Find State Table

- From the state diagram, we can fill in the state table.
- There are 4 states, one input, and one output. We will choose the form with four rows, one for each current state.
- From State A, the 0 and 1 input transitions have been filled in along with the outputs.



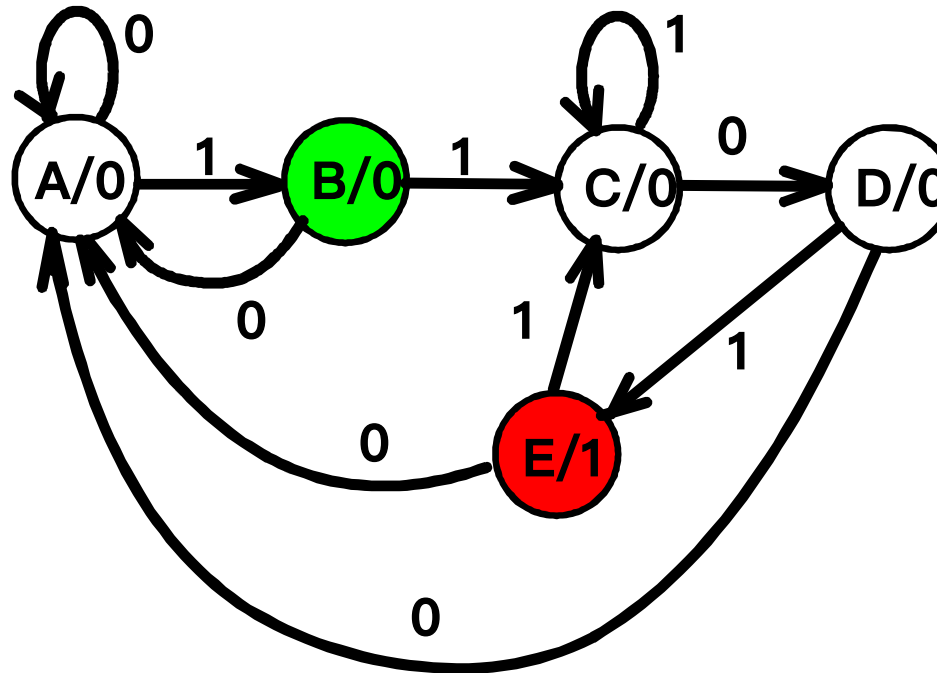
| Present State | Next State |     | Output |     |
|---------------|------------|-----|--------|-----|
|               | x=0        | x=1 | x=0    | x=1 |
| A             | A          | B   | 0      | 0   |
| B             | A          | C   | 0      | 0   |
| C             | D          | C   | 0      | 0   |
| D             | A          | B   | 0      | 1   |

**Note that state table can also be used to merge equivalent states.**

**What would the state diagram and state table look like for the Moore model?**

## Example: Moore Model (cont'd)

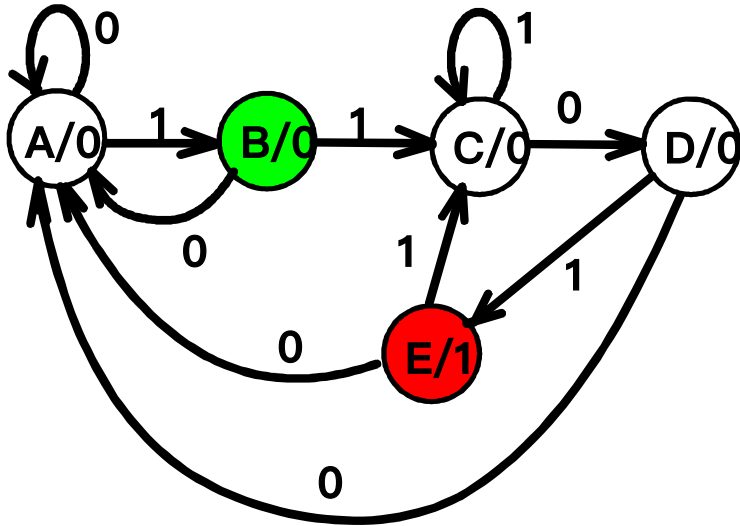
- We mark outputs on states for Moore model
- Arcs now show only state transitions
- Add a new state E to produce the output 1



- Note that the new state, E produces the same behavior in the future as state B. But it gives a different output at the present time. Thus, these states do represent a *different abstraction* of the input history.

## Example: Moore Model (cont'd)

- Fill in the state table according to the stable diagram
- Memory aid re more state in the Moore model: “Moore is More.”



| Present State | Next State |     | Output y |
|---------------|------------|-----|----------|
|               | x=0        | x=1 |          |
| A             | A          | B   | 0        |
| B             | A          | C   | 0        |
| C             | D          | C   | 0        |
| D             | A          | E   | 0        |
| E             | A          | C   | 1        |

# Optimization: Counting Order Assignment

- Performing two-level optimization:

|                      |   |   |  |                |
|----------------------|---|---|--|----------------|
| <b>D<sub>1</sub></b> |   |   |  |                |
|                      |   | X |  |                |
|                      | 0 | 0 |  |                |
|                      | 0 | 1 |  |                |
|                      | 0 | 0 |  | Y <sub>2</sub> |
| Y <sub>1</sub>       | 1 | 1 |  |                |

|                      |   |   |  |                |
|----------------------|---|---|--|----------------|
| <b>D<sub>2</sub></b> |   |   |  |                |
|                      |   | X |  |                |
|                      | 0 | 1 |  |                |
|                      | 0 | 0 |  |                |
|                      | 0 | 1 |  | Y <sub>2</sub> |
| Y <sub>1</sub>       | 1 | 0 |  |                |

|                |   |   |  |                |
|----------------|---|---|--|----------------|
| <b>Z</b>       |   |   |  |                |
|                |   | X |  |                |
|                | 0 | 0 |  |                |
|                | 0 | 0 |  |                |
|                | 0 | 1 |  | Y <sub>2</sub> |
| Y <sub>1</sub> | 0 | 0 |  |                |

- $$D_1 = Y_1 \bar{Y}_2 + X \bar{Y}_1 Y_2$$

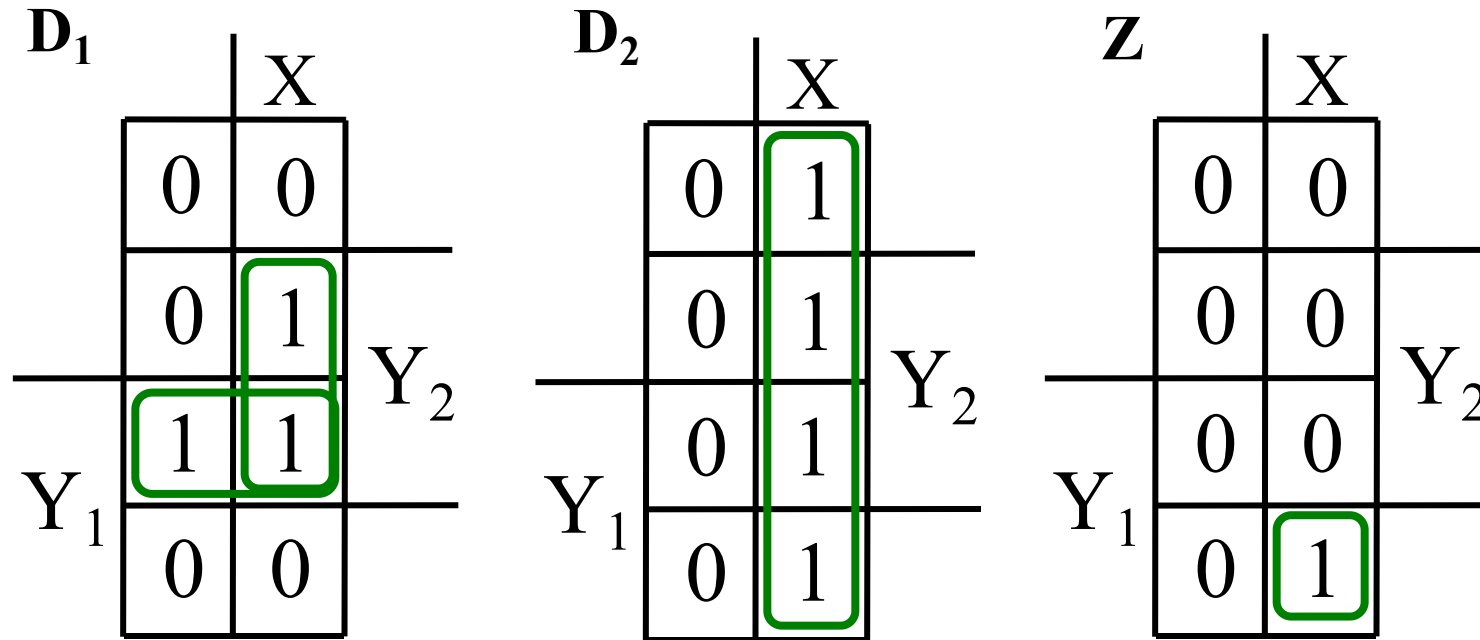
$$D_2 = \bar{X} Y_1 \bar{Y}_2 + X \bar{Y}_1 \bar{Y}_2 + X Y_1 Y_2$$

$$Z = X Y_1 Y_2$$

Gate Input Cost = ?

# Optimization: Gray Code Assignment

- Performing two-level optimization:



- $D_1 = Y_1 Y_2 + X Y_2$   
 $D_2 = X$   
 $Z = X Y_1 \bar{Y}_2$

Gate Input Cost = ?

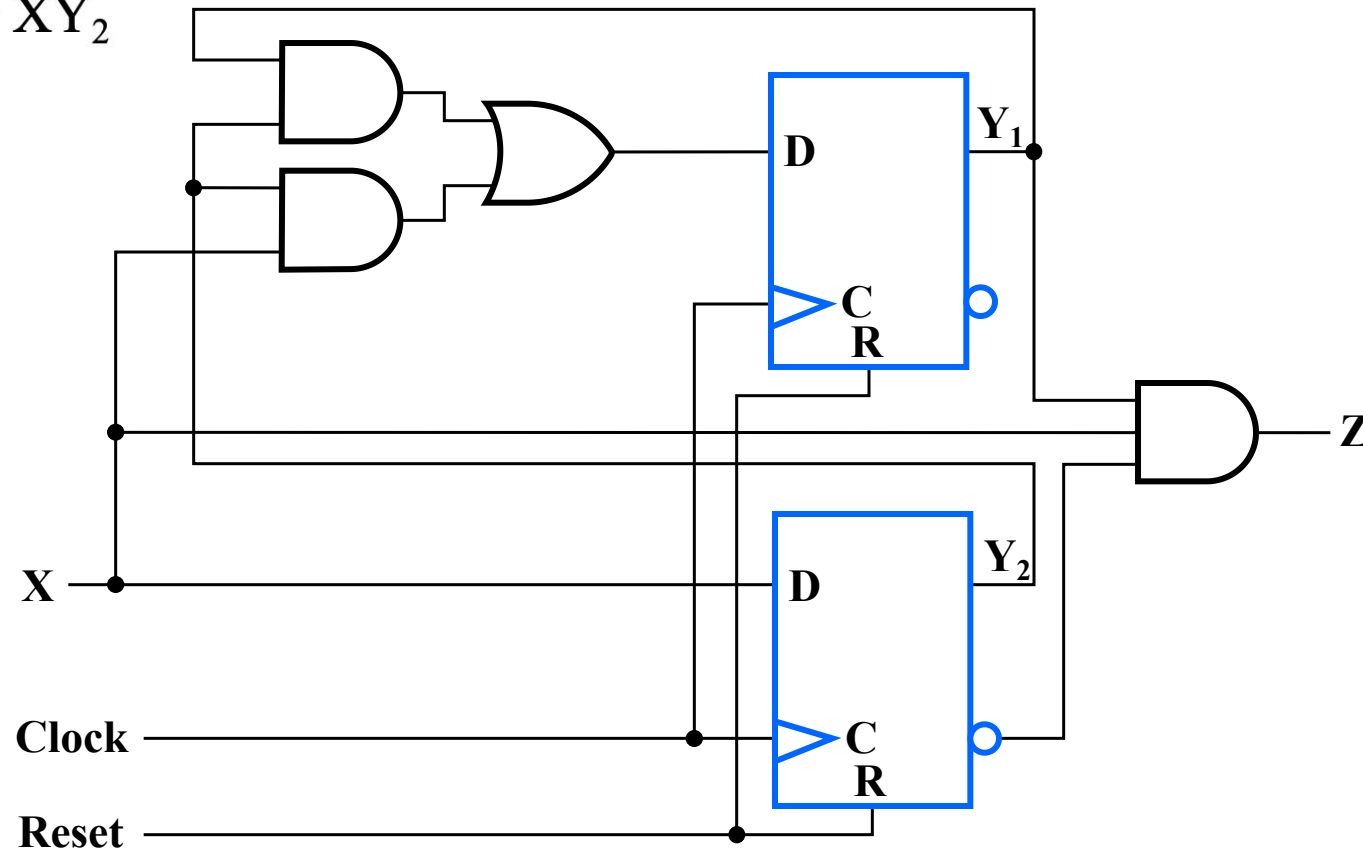
# Map Technology: Gray Code Assignment

- D Flip-flops with Reset (not inverted)

$$D_1 = Y_1 Y_2 + X Y_2$$

$$D_2 = X$$

$$Z = X Y_1 \bar{Y}_2$$

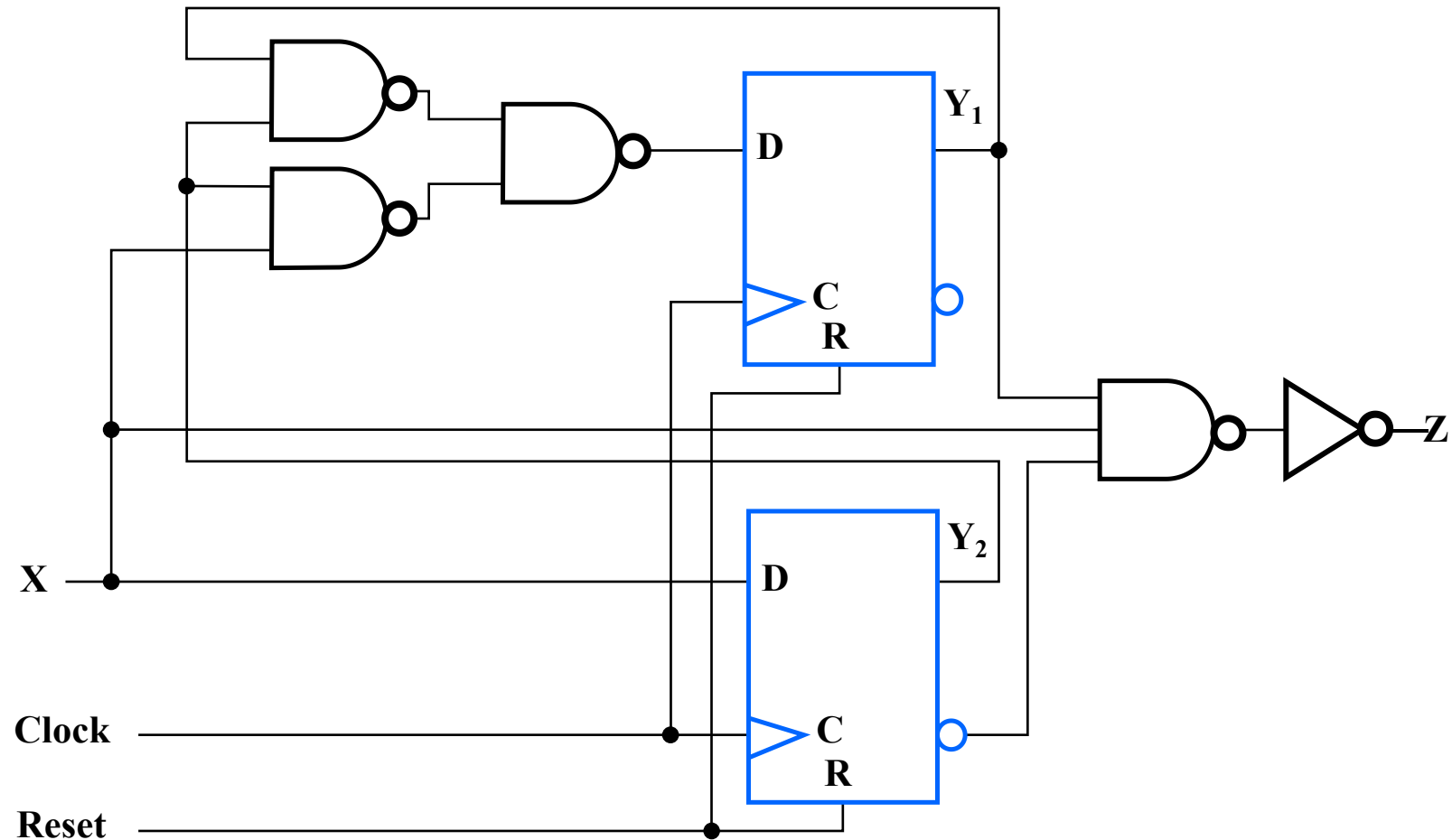


Initial circuit



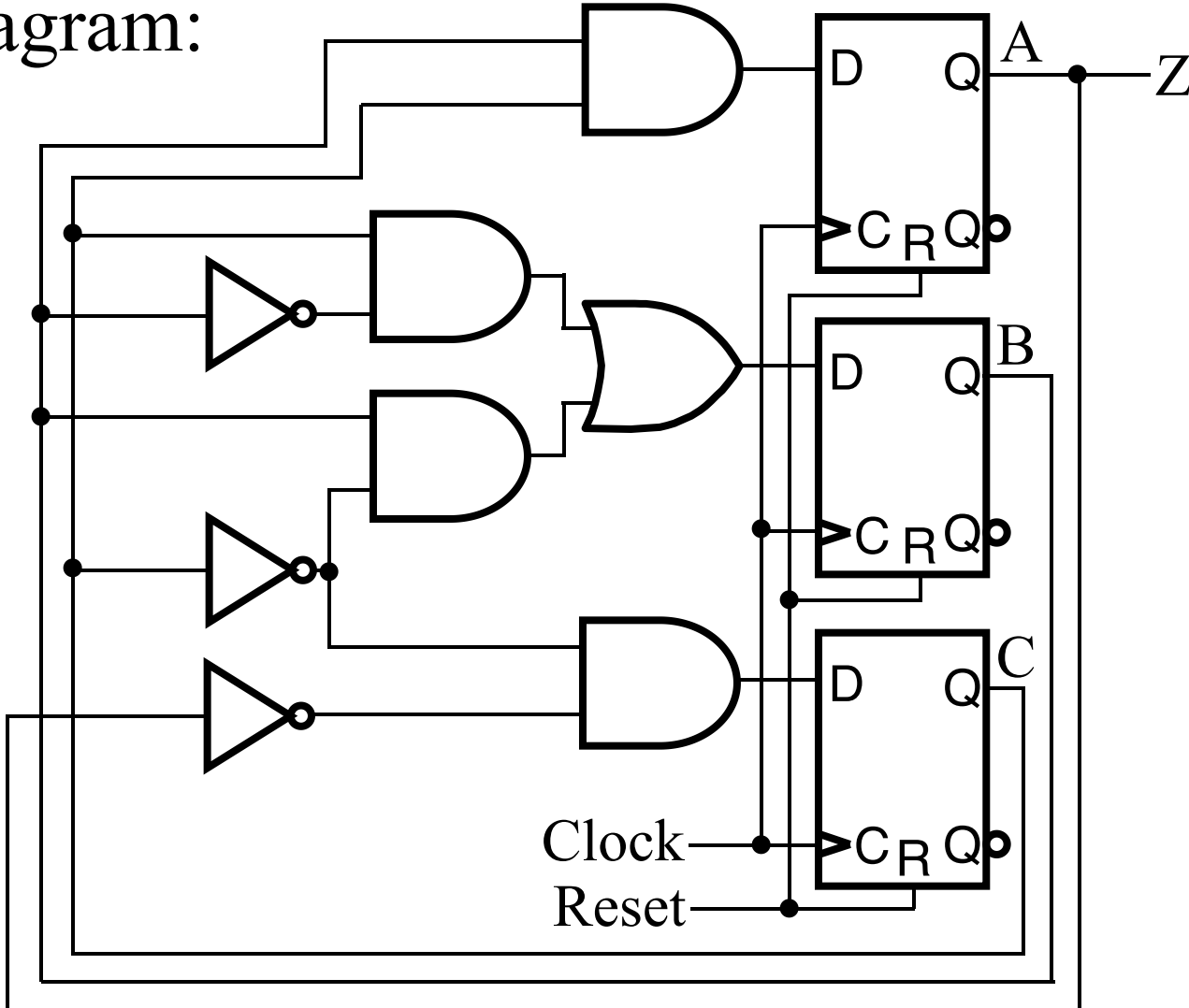
# Mapped Circuit

- NAND gates with up to 4 inputs and inverters



Final circuit

- Logic diagram:



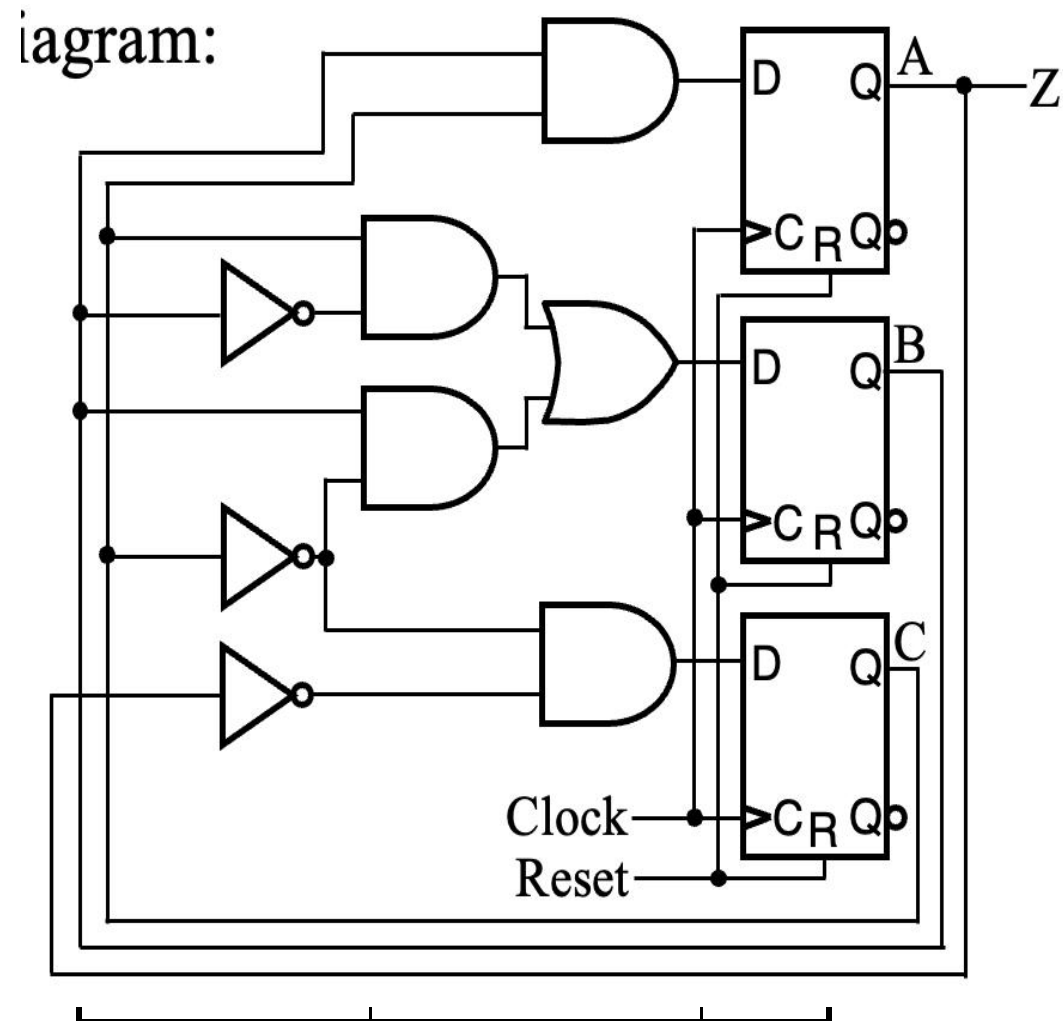
# Flip-Flop Input Equations

---

- Variables
  - Inputs: None
  - Outputs: Z
  - State Variables: A, B, C
- Initialization: Reset to (0,0,0)
- Equations
  - $A(t+1) =$   $Z =$
  - $B(t+1) =$
  - $C(t+1) =$

# State Table

- $A(t+1) = BC$
- $B(t+1) = \bar{B}C + B\bar{C}$
- $C(t+1) = \bar{A}\bar{C}$
- $Z = A$
- Let  $Y = X(t+1)$

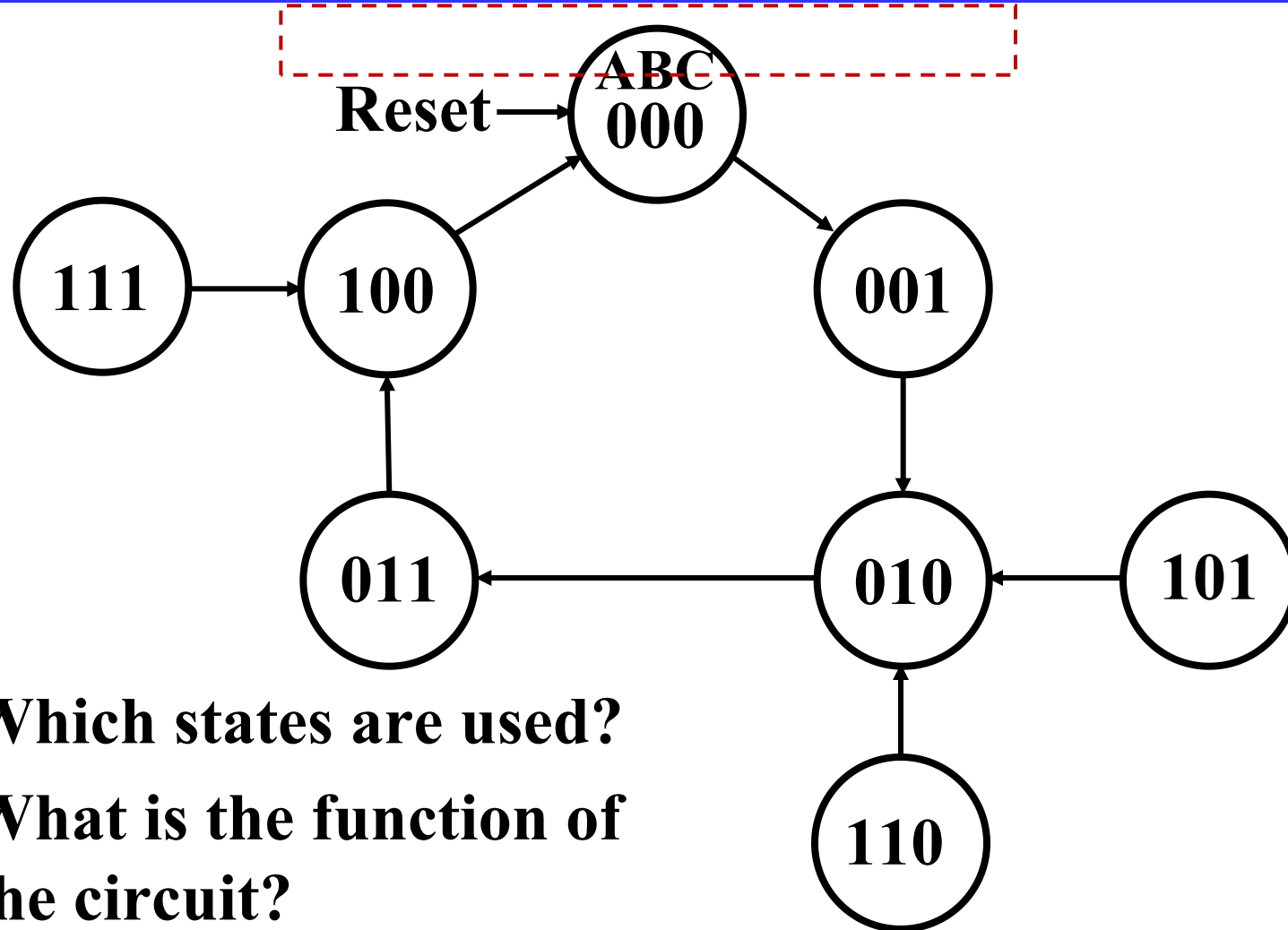


# State Table

- $A(t+1) = BC$
- $B(t+1) = \bar{B}C + B\bar{C}$
- $C(t+1) = \bar{A}\bar{C}$
- $Z = A$
- Let  $Y = X(t+1)$

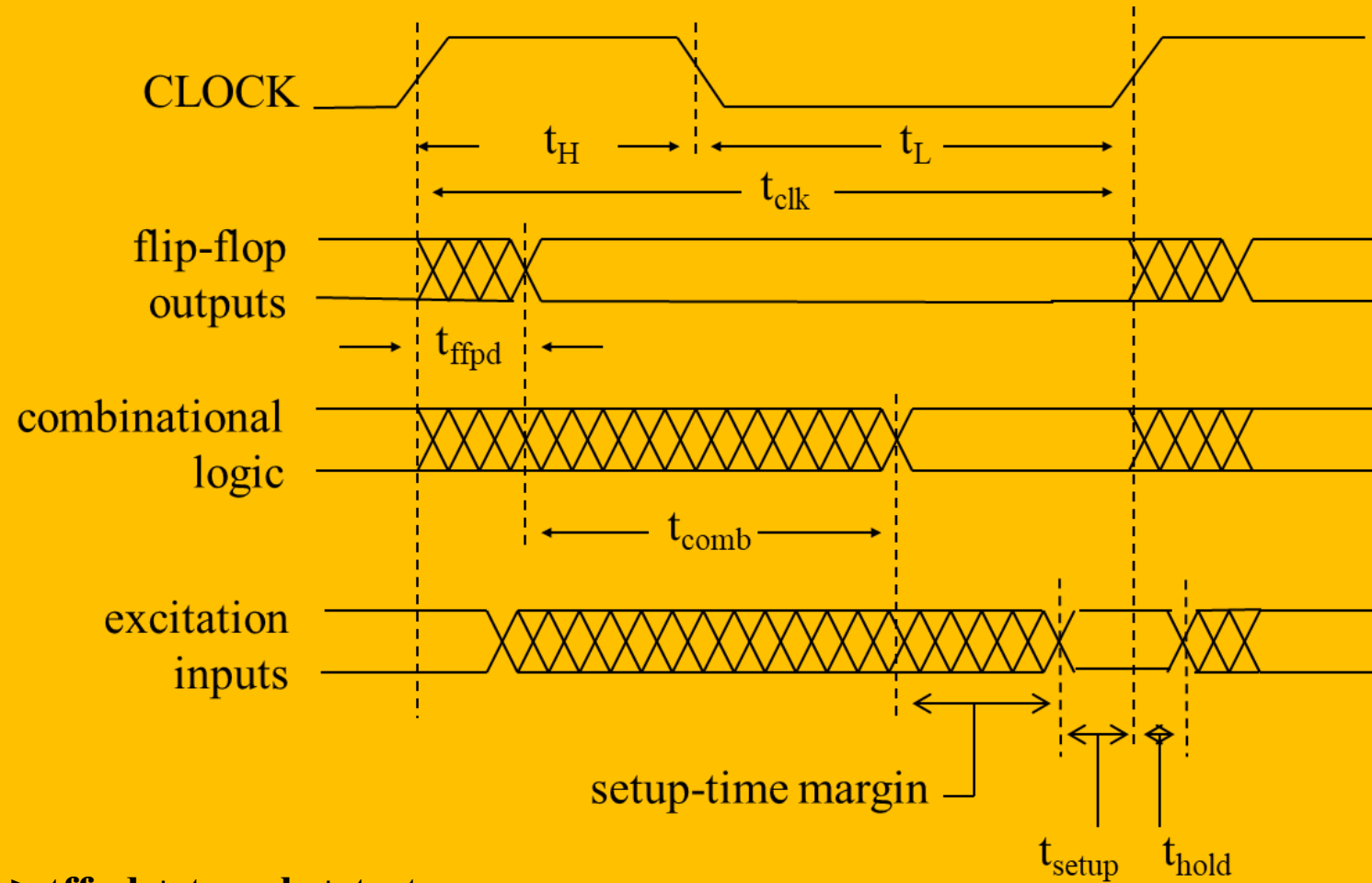
| <b>A B C</b> | <b><math>Y_A Y_B Y_C</math></b> | <b>Z</b> |
|--------------|---------------------------------|----------|
| <b>0 0 0</b> | <b>0 0 1</b>                    | <b>0</b> |
| <b>0 0 1</b> | <b>0 1 0</b>                    | <b>0</b> |
| <b>0 1 0</b> | <b>0 1 1</b>                    | <b>0</b> |
| <b>0 1 1</b> | <b>1 0 0</b>                    | <b>0</b> |
| <b>1 0 0</b> | <b>0 0 0</b>                    | <b>1</b> |
| <b>1 0 1</b> | <b>0 1 0</b>                    | <b>1</b> |
| <b>1 1 0</b> | <b>0 1 0</b>                    | <b>1</b> |
| <b>1 1 1</b> | <b>1 0 0</b>                    | <b>1</b> |

# State Diagram



- Which states are used?
- What is the function of the circuit?

# Timing Diagram of Sequential Circuits

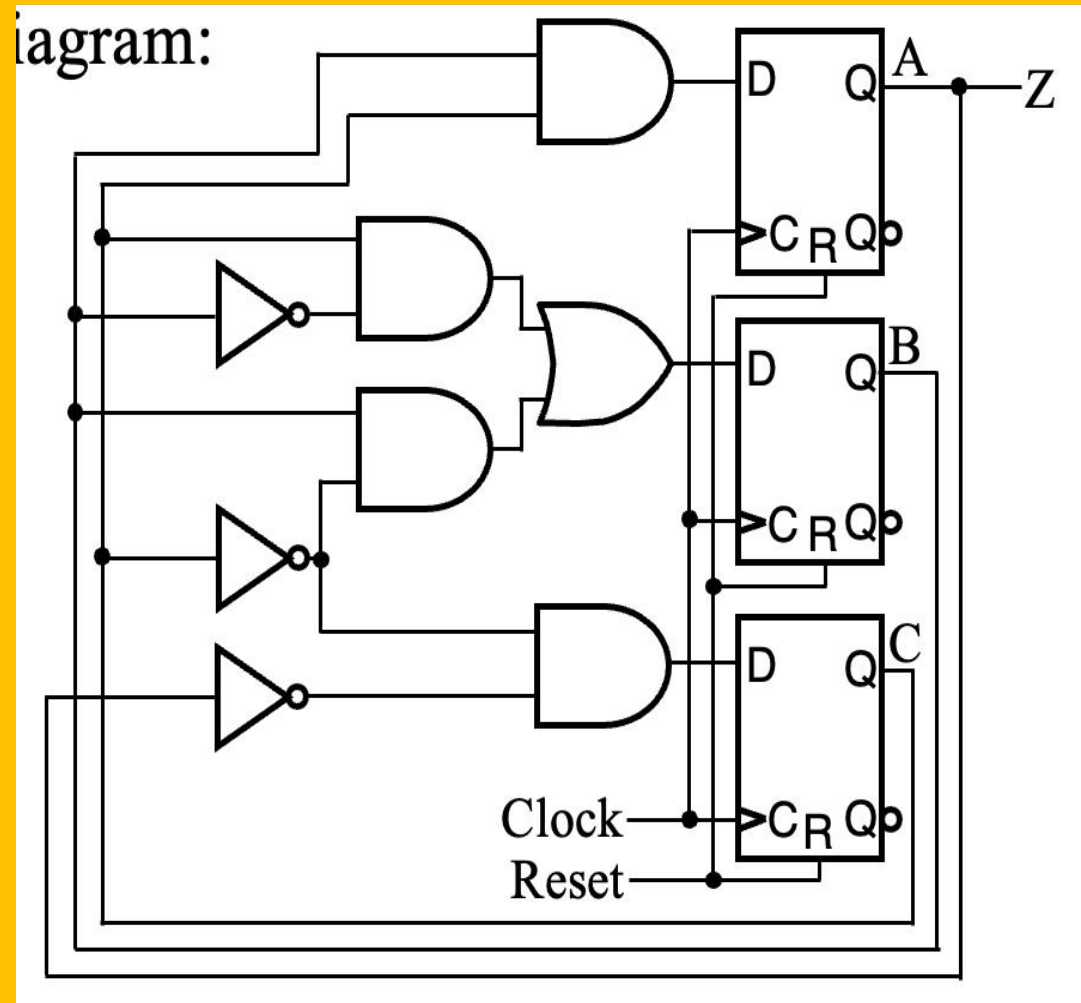


$$t_{clk} \geq t_{ffpd} + t_{comb} + t_{setup}$$

$$t_{hold} \leq t_{ffpd} + t_{comb}$$

# State Table

- $A(t+1) = BC$
- $B(t+1) = \bar{B}C + B\bar{C}$
- $C(t+1) = \bar{A}\bar{C}$
- $Z = A$
- Let  $Y = X(t+1)$





# State Table: Two Alternative Forms

| Present State |          | Input | Next State |          | Output |
|---------------|----------|-------|------------|----------|--------|
| <i>A</i>      | <i>B</i> |       | <i>A</i>   | <i>B</i> |        |
| 0             | 0        | 0     | 0          | 0        | 0      |
| 0             | 0        | 1     | 0          | 1        | 0      |
| 0             | 1        | 0     | 0          | 0        | 1      |
| 0             | 1        | 1     | 1          | 1        | 0      |
| 1             | 0        | 0     | 0          | 0        | 1      |
| 1             | 0        | 1     | 1          | 0        | 0      |
| 1             | 1        | 0     | 0          | 0        | 1      |
| 1             | 1        | 1     | 1          | 0        | 0      |

| Present State | Next State   |              | Output       |              |
|---------------|--------------|--------------|--------------|--------------|
|               | <i>x</i> = 0 | <i>x</i> = 1 | <i>x</i> = 0 | <i>x</i> = 1 |
| <i>AB</i>     | <i>AB</i>    | <i>AB</i>    | <i>y</i>     | <i>y</i>     |
| 00            | 00           | 01           | 0            | 0            |
| 01            | 00           | 11           | 1            | 0            |
| 10            | 00           | 10           | 1            | 0            |
| 11            | 00           | 10           | 1            | 0            |

# S-R Flip-Flop Descriptors

- Characteristic Table

| S | R | Q(t+1) | Operation |
|---|---|--------|-----------|
| 0 | 0 | $Q(t)$ | No change |
| 0 | 1 | 0      | Reset     |
| 1 | 0 | 1      | Set       |
| 1 | 1 | ?      | Undefined |

- Characteristic Equation

$$Q(t+1) = S + \overline{R} \cdot Q(t), S \cdot R = 0$$

- Excitation Table

| Q(t) | Q(t+1) | S | R | Operation |
|------|--------|---|---|-----------|
| 0    | 0      | 0 | X | No change |
| 0    | 1      | 1 | 0 | Set       |
| 1    | 0      | 0 | 1 | Reset     |
| 1    | 1      | X | 0 | No change |

# D Flip-Flop Descriptors

---

- Characteristic Table

| D | Q(t+1) | Operation |
|---|--------|-----------|
| 0 | 0      | Reset     |
| 1 | 1      | Set       |

- Characteristic Equation

$$Q(t+1) = D$$

- Excitation Table

| Q(t +1) | D | Operation |
|---------|---|-----------|
| 0       | 0 | Reset     |
| 1       | 1 | Set       |

# J-K Flip-Flop Descriptors

- Characteristic Table

| J | K | Q(t+1)            | Operation  |
|---|---|-------------------|------------|
| 0 | 0 | $Q(t)$            | No change  |
| 0 | 1 | 0                 | Reset      |
| 1 | 0 | 1                 | Set        |
| 1 | 1 | $\overline{Q}(t)$ | Complement |

- Characteristic Equation

$$Q(t+1) = J \overline{Q}_{(t)} + \overline{K} Q_{(t)}$$

- Excitation Table

| Q(t) | Q(t+1) | J | K | Operation |
|------|--------|---|---|-----------|
| 0    | 0      | 0 | X | No change |
| 0    | 1      | 1 | X | Set       |
| 1    | 0      | X | 1 | Reset     |
| 1    | 1      | X | 0 | No Change |

# T Flip-Flop Descriptors

---

- Characteristic Table

| T | Q(t+1)            | Operation  |
|---|-------------------|------------|
| 0 | $Q(t)$            | No change  |
| 1 | $\overline{Q}(t)$ | Complement |

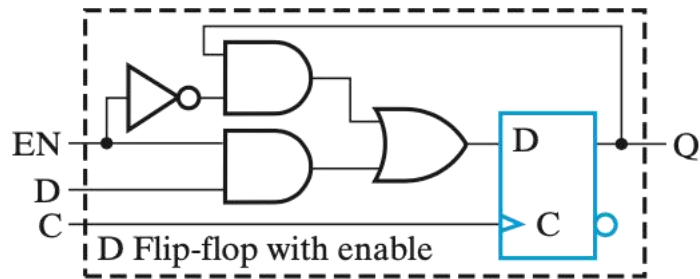
- Characteristic Equation

$$Q(t+1) = T \oplus Q$$

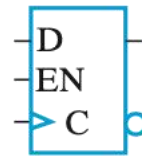
- Excitation Table

| Q(t+1)            | T | Operation  |
|-------------------|---|------------|
| $Q(t)$            | 0 | No change  |
| $\overline{Q}(t)$ | 1 | Complement |

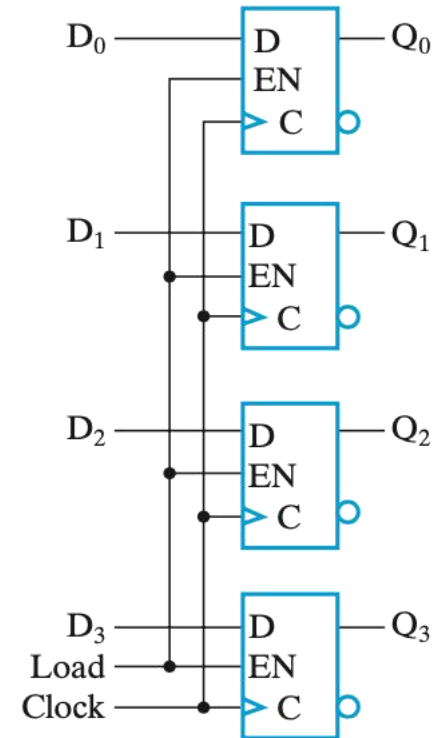
# Flip-Flops With EN for Register



(a)

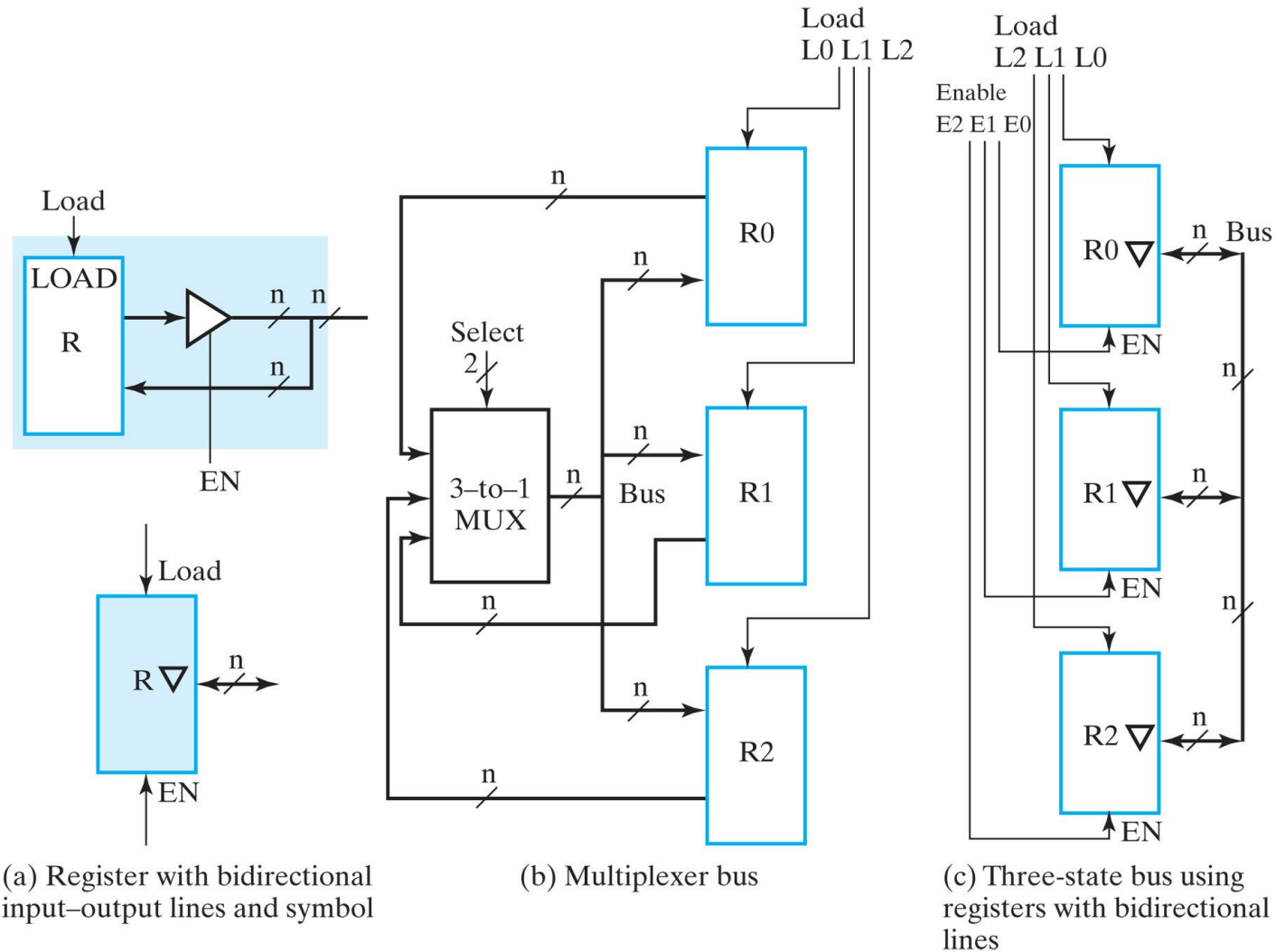


(b)



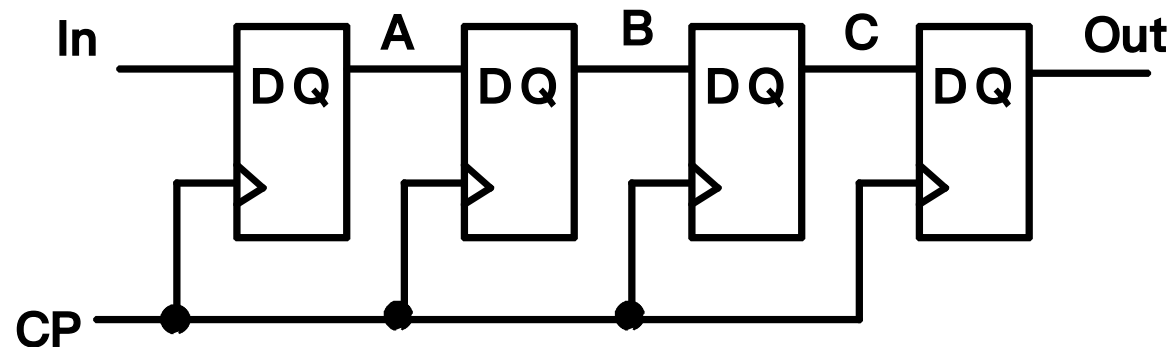
(c)

# Multiplexer Bus vs. 3-State Bus



# Shift Registers

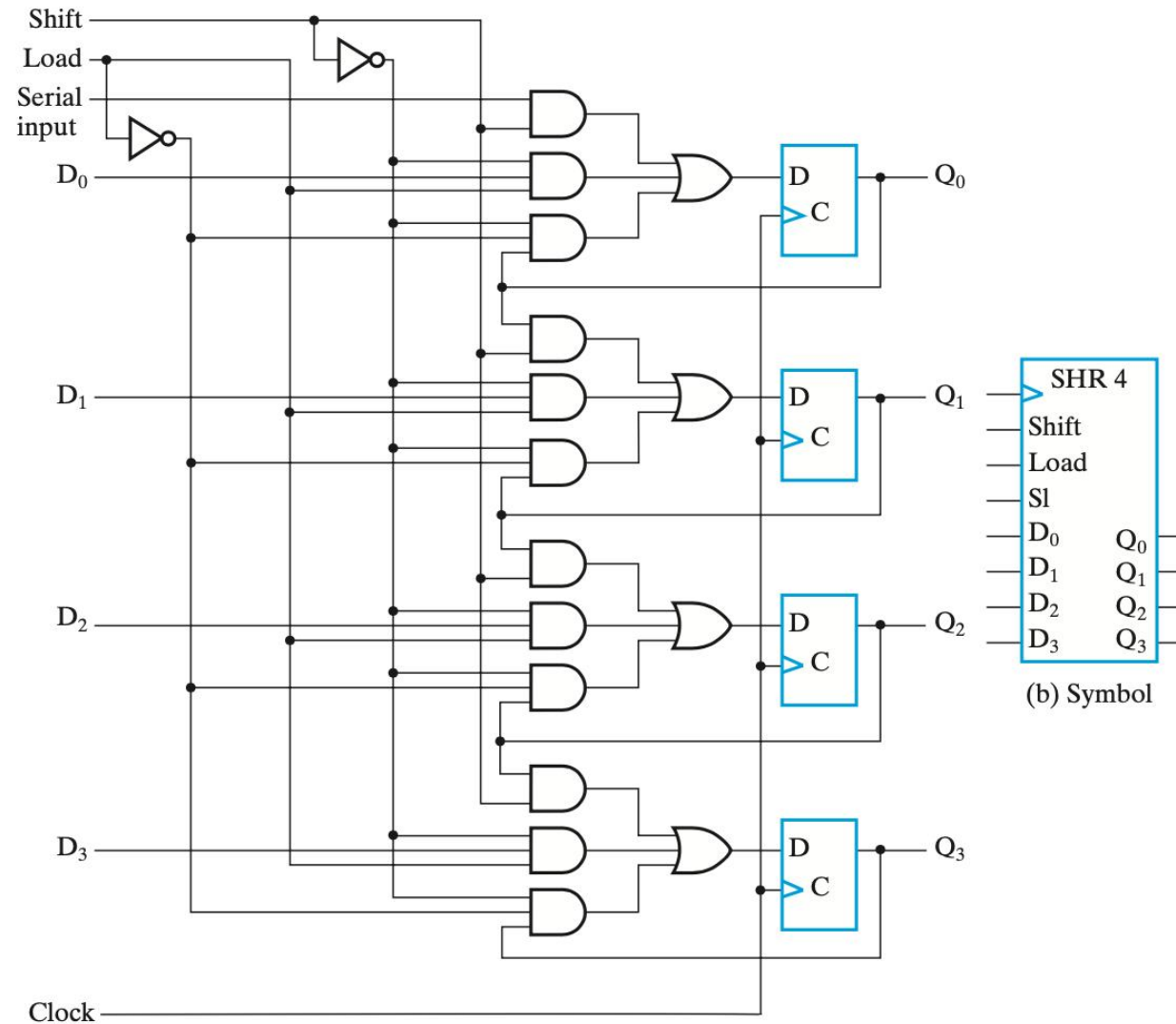
- A register capable of shifting its stored bits laterally in one or both directions is called a shift register
- Shift Registers move data laterally within the register toward its MSB or LSB position
- In the simplest case, the shift register is simply a set of D flip-flops connected in a row like this:



- Data input, *In*, is called a serial input or the shift right input
- Data output, *Out*, is often called the serial output
- The vector (*A*, *B*, *C*, *Out*) is called the parallel output if they can be accessed in parallel

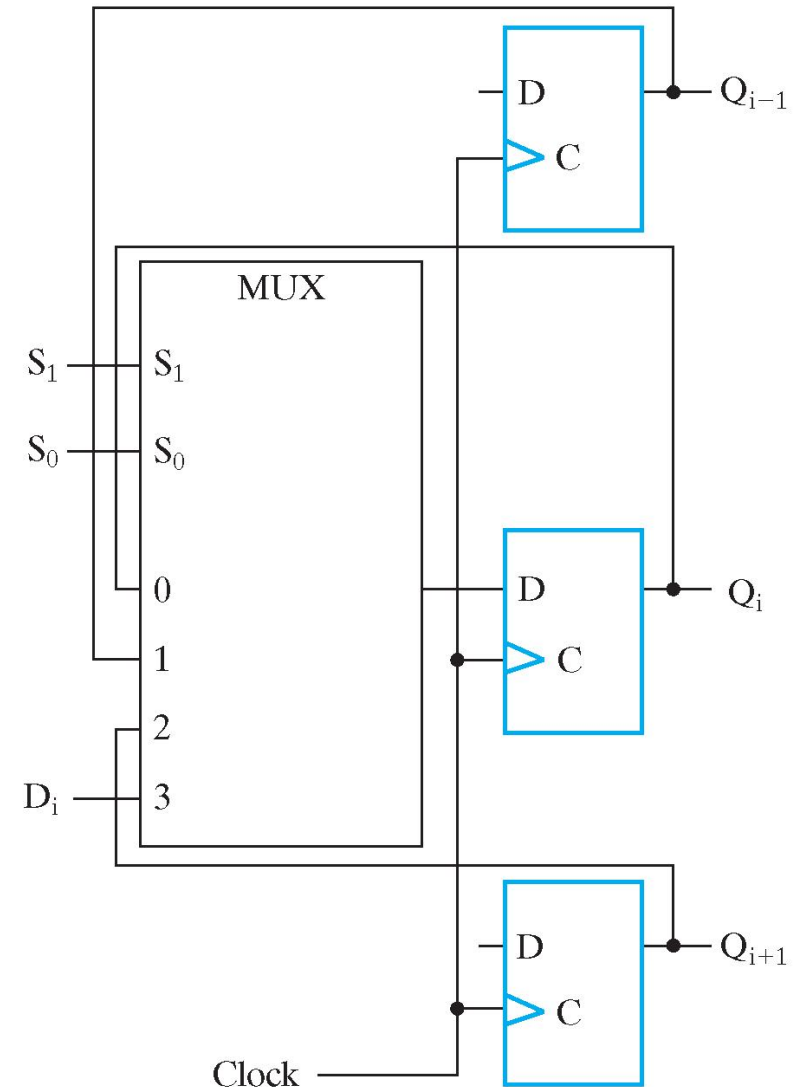


# Shift Register with Parallel Load



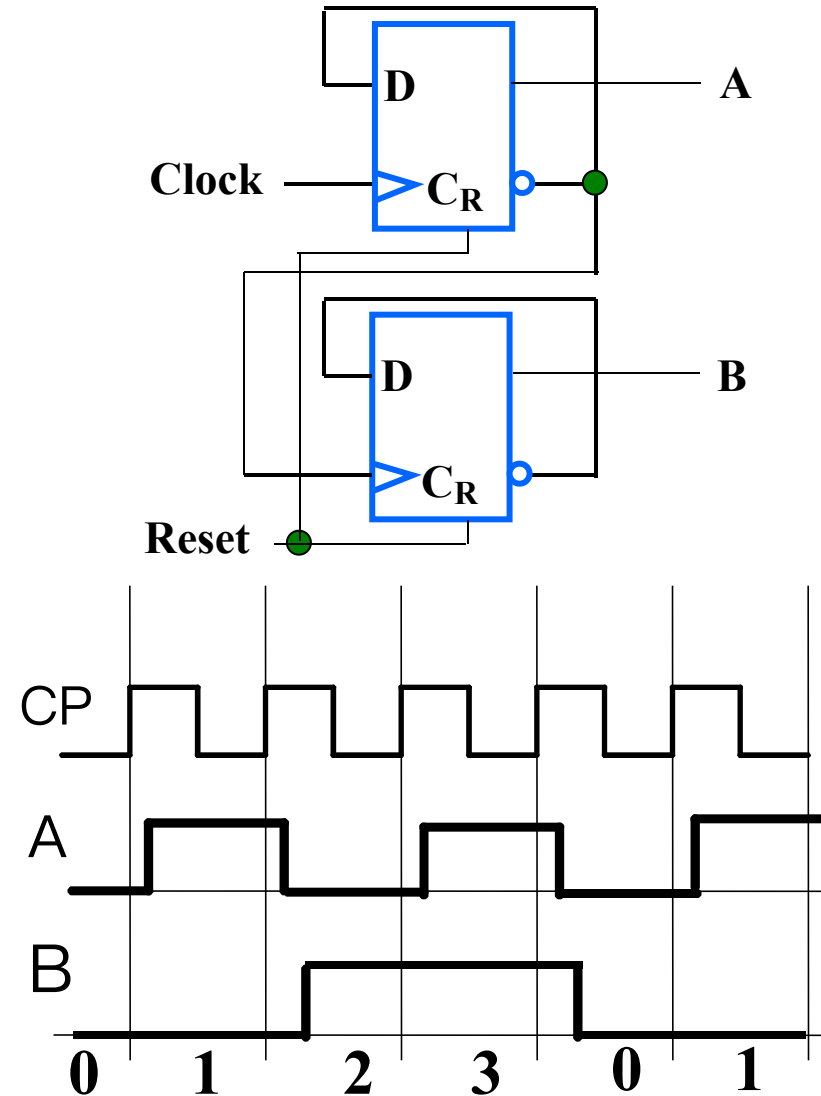
# Shift Registers with Additional Functions

- By placing a 4-input multiplexer in front of each D flip-flop in a shift register, we can implement a circuit with shifts right, shifts left, parallel load, hold.
- Shift registers can also be designed to shift more than a single bit position right or left
- Shift registers can be designed to shift a variable number of bit positions specified by a variable called a *shift amount*.



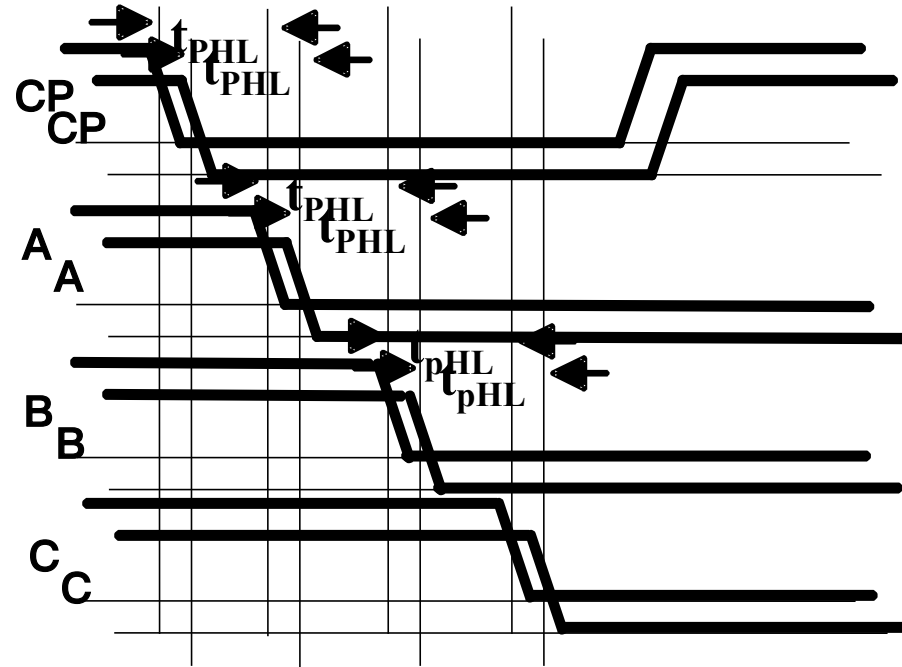
# Ripple Counters

- When there is a positive edge on the clock input of A, A complements
- The clock input for flip-flop B is the complemented output of flip-flop A
- When flip A changes from 1 to 0, there is a positive edge on the clock input of B causing B to complement



# Ripple Counters (cont'd)

- Starting with  $C = B = A = 1$ , equivalent to  $(C,B,A) = 7$  base 10, the next clock increments the count to  $(C,B,A) = 0$  base 10. In fine timing detail:
  - The clock to output delay  $t_{PHL}$  causes an increasing delay from clock edge for each stage transition.
  - Thus, the count “ripples” from least to most significant bit.
  - For  $n$  bits, total worst case delay is  $n \cdot t_{PHL}$

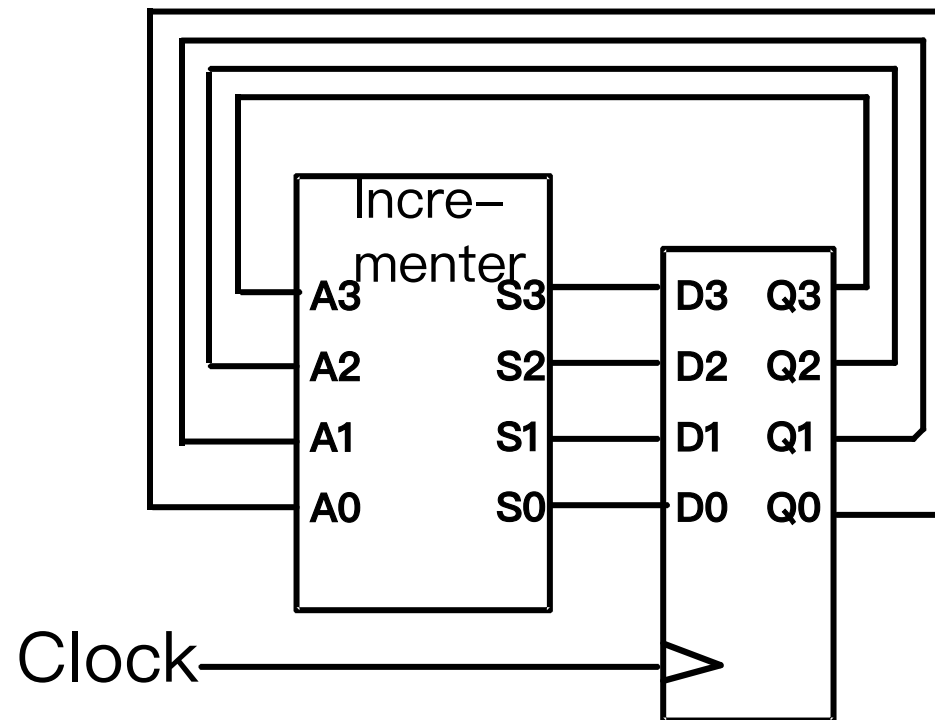


# Synchronous Counters

- To eliminate the "ripple" effects, use a common clock for each flip-flop and a combinational circuit to generate the next state.
- For an up-counter, use an incrementer

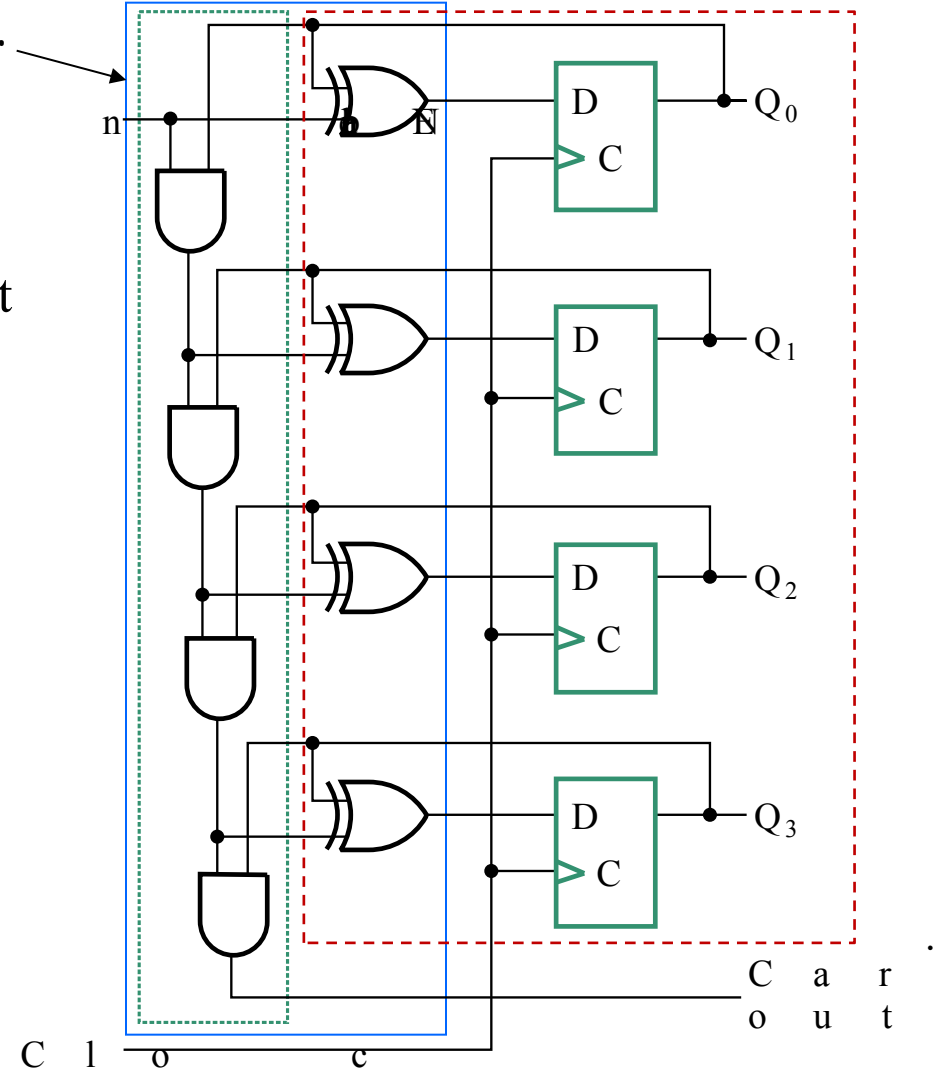
Upward Counting Sequence

| $Q_3$ | $Q_2$ | $Q_1$ | $Q_0$ |
|-------|-------|-------|-------|
| 0     | 0     | 0     | 0     |
| 0     | 0     | 0     | 1     |
| 0     | 0     | 1     | 0     |
| 0     | 0     | 1     | 1     |
| 0     | 1     | 0     | 0     |
| 0     | 1     | 0     | 1     |
| 0     | 1     | 1     | 0     |
| 0     | 1     | 1     | 1     |
| 1     | 0     | 0     | 0     |
| 1     | 0     | 0     | 1     |
| 1     | 0     | 1     | 0     |
| 1     | 0     | 1     | 1     |
| 1     | 1     | 0     | 0     |
| 1     | 1     | 0     | 1     |
| 1     | 1     | 1     | 0     |
| 1     | 1     | 1     | 1     |



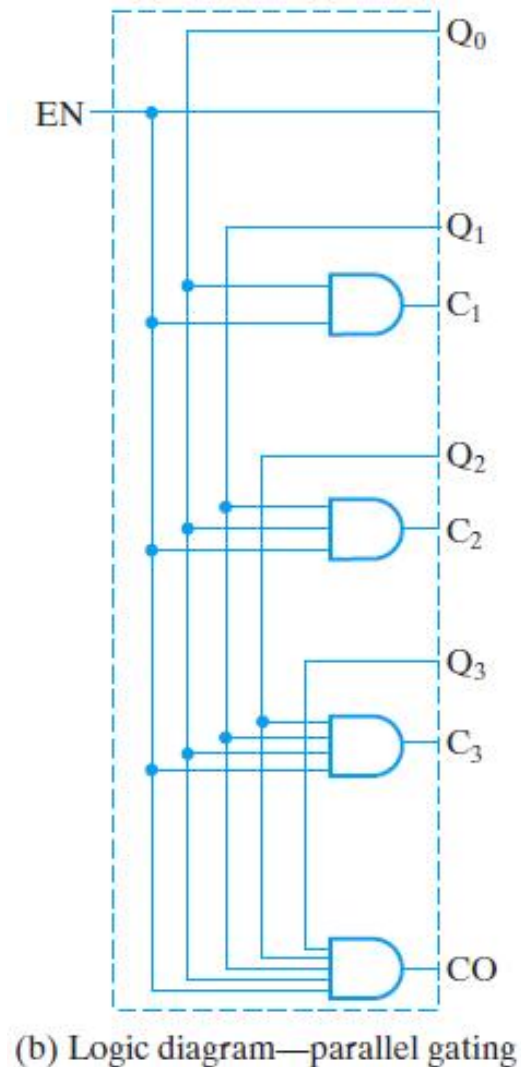
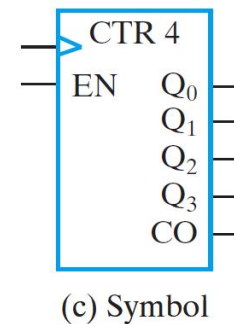
# Synchronous Counters (cont'd)

- Internal details => Incrementer
- Internal Logic
  - XOR complements each bit
  - AND chain causes complement of a bit if all bits toward LSB from it equal 1
- Count Enable
  - Forces all outputs of AND chain to 0 to “hold” the state
- Carry Out
  - Added as part of incrementer
  - Connect to Count Enable of additional 4-bit counters to form larger counters



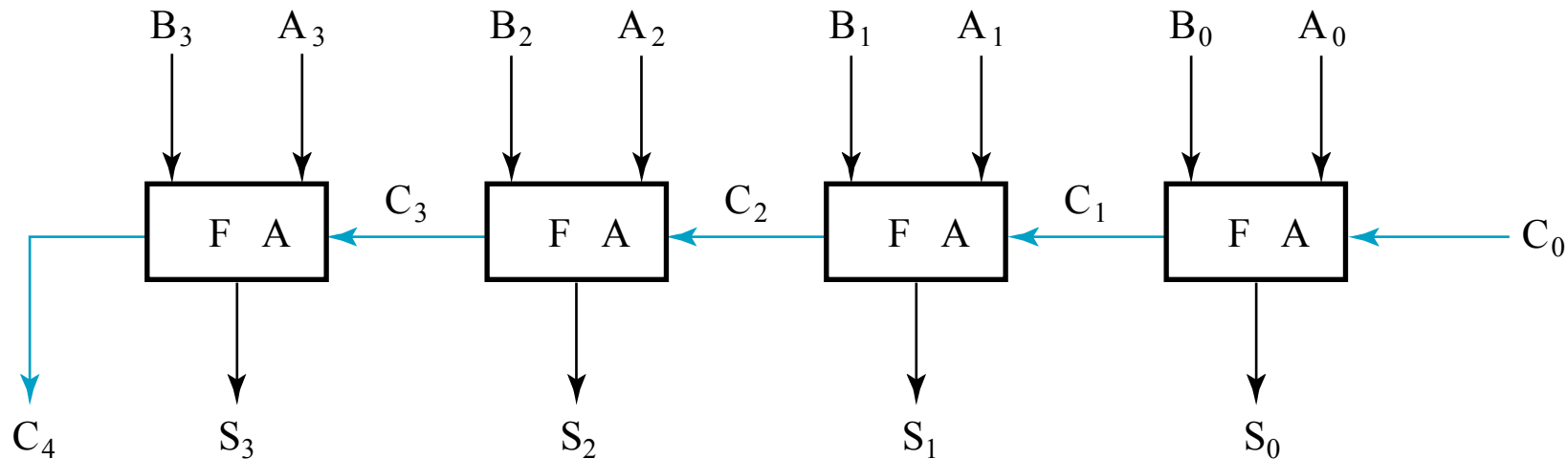
# Synchronous Counters (cont'd)

- Carry chain
  - Series of AND gates through which the carry “ripples”
  - Yields long path delays
  - Called serial gating
- Replace AND carry chain with ANDs => in parallel
  - Reduces path delays
  - Called parallel gating
  - Like carry lookahead
  - Lookahead can be used on COs and ENs to prevent long paths in large counters
- Symbol for synchronous counter



# Ripple-Carry Adder (RCA)

- Chain 1-bit adders together
- Carry ripples through entire chain
- Disadvantage: **slow**





# Carry-Lookahead Adder (CLA)

---

- E.g., a 4-bit adder

$$C_1 = A_0B_0 + (A_0 \oplus B_0) C_0$$

$$\begin{aligned} C_2 &= A_1B_1 + (A_1 \oplus B_1) C_1 \\ &= A_1B_1 + (A_1 \oplus B_1) A_0B_0 + (A_1 \oplus B_1)(A_0 \oplus B_0) C_0 \end{aligned}$$

$$\begin{aligned} C_3 &= A_2B_2 + (A_2 \oplus B_2) C_2 \\ &= A_2B_2 + (A_2 \oplus B_2) A_1B_1 + (A_2 \oplus B_2)(A_1 \oplus B_1)A_0B_0 + (A_2 \oplus B_2)(A_1 \oplus B_1)(A_0 \oplus B_0) C_0 \end{aligned}$$

$$\begin{aligned} C_4 &= A_3B_3 + (A_3 \oplus B_3) C_3 \\ &= A_3B_3 + (A_3 \oplus B_3) A_2B_2 + (A_3 \oplus B_3)(A_2 \oplus B_2)A_1B_1 + (A_3 \oplus B_3) (A_2 \oplus B_2)(A_1 \oplus B_1)A_0B_0 + (A_3 \oplus B_3)(A_2 \oplus B_2)(A_1 \oplus B_1)(A_0 \oplus B_0) C_0 \end{aligned}$$

# Carry-Lookahead Adder: Bit Level

---

- Column  $i$  produces a carry out by either *generating* a carry out or *propagating* a carry in to the carry out
- Generate ( $G_i$ ) and propagate ( $P_i$ ) signals for each column:
  - Column  $i$  will generate a carry out if  $A_i$  AND  $B_i$  are both 1
$$G_i = A_i B_i$$
  - Column  $i$  will propagate a carry in to the carry out if  $A_i$  OR  $B_i$  is 1
$$P_i = A_i \oplus B_i$$
  - The carry out of column  $i+1$  ( $C_{i+1}$ ) is:
$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$
$$= G_i + P_i C_i$$
  - The sum of column  $i$  ( $S_i$ ) is:
$$S_i = A_i \oplus B_i \oplus C_i = P_i \oplus C_i$$

# Revisit the 4-Bit Adder

---

- Compute carry out ( $C_{out}$ ) for 4-bit blocks using *generate* and *propagate* signals

$$C_1 = G_0 + P_0 C_0$$

$$\begin{aligned} C_2 &= G_1 + P_1 C_1 \\ &= G_1 + P_1 (G_0 + P_0 C_0) \\ &= G_1 + P_1 G_0 + P_1 P_0 C_0 \end{aligned}$$

$$\begin{aligned} C_3 &= G_2 + P_2 C_2 \\ &= G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0) \\ &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \end{aligned}$$

$$\begin{aligned} C_4 &= G_3 + P_3 C_3 \\ &= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) \\ &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \end{aligned}$$

# Carry-Lookahead Addition: Block/Group Level (cont'd)

---

- **Step 1:** compute  $G_i$  and  $P_i$  for all columns
- **Step 2:** compute  $G$  and  $P$  for  $k$ -bit blocks
- **Step 3:**  $C_{in}$  propagates through each  $k$ -bit propagate/generate block

- **Generally, for 4-bit blocks**  $i=j+3$

$$G_{i:j} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} G_j))$$

$$P_{i:j} = P_i P_{i-1} P_{i-2} P_j$$

$$C_{j+1} = G_{i:j} + P_{i:j} C_j$$

- $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 = \mathbf{G_{3:0} + P_{3:0} C_0}$
- $C_8 = G_7 + P_7 G_6 + P_7 P_6 G_5 + P_7 P_6 P_5 G_4 + P_7 P_6 P_5 P_4 C_4 = \mathbf{G_{7:4} + P_{7:4} C_4}$
- $C_{12} = G_{11} + P_{11} G_{10} + P_{11} P_{10} G_9 + P_{11} P_{10} P_9 G_8 + P_{11} P_{10} P_9 P_8 C_8 = \mathbf{G_{11:8} + P_{11:8} C_8}$
- $C_{16} = G_{15} + P_{15} G_{14} + P_{15} P_{14} G_{13} + P_{15} P_{14} P_{13} G_{12} + P_{15} P_{14} P_{13} P_{12} C_{12} = \mathbf{G_{15:12} + P_{15:12} C_{12}}$

# 4-Bit Adder vs. 16-Bit Adder

---

## 4-bit Adder

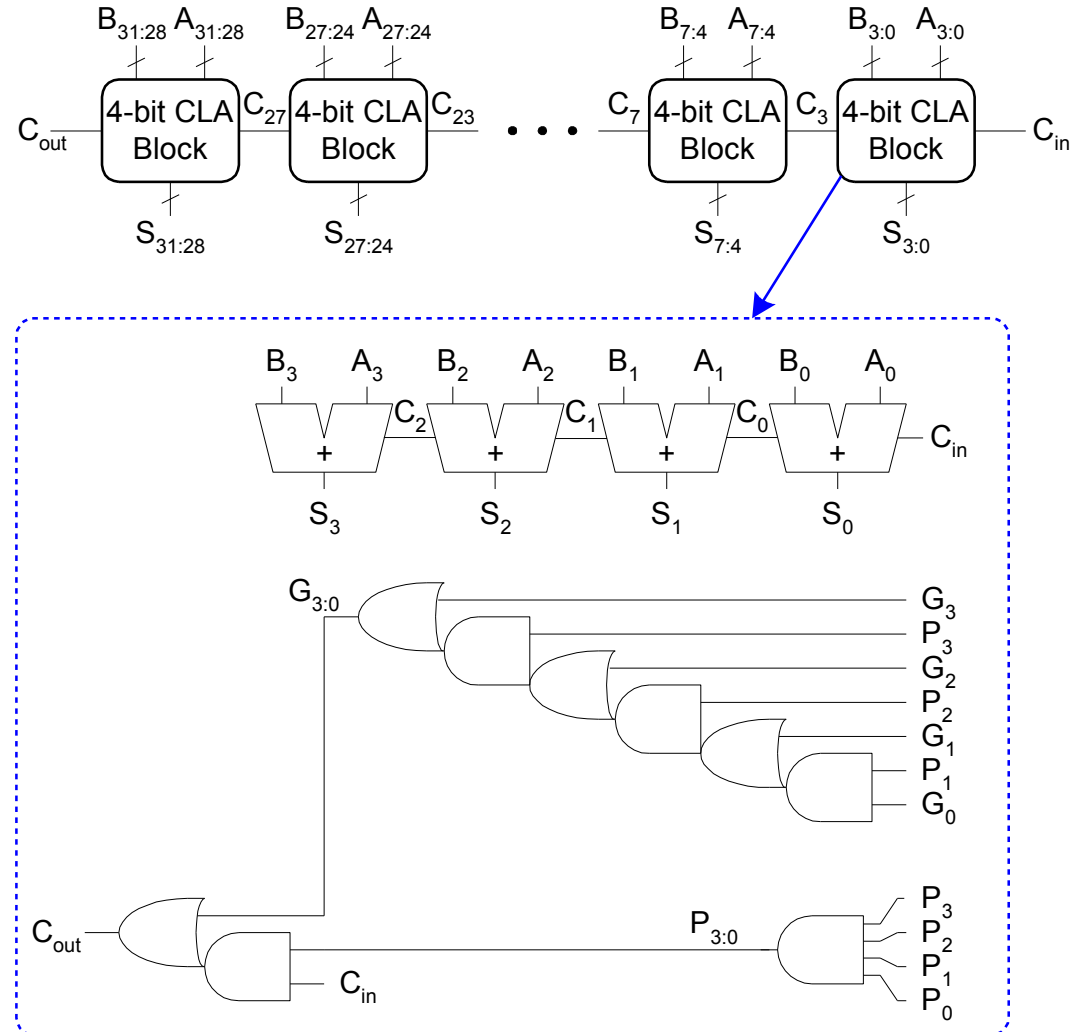
- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1 C_1$
- $C_3 = G_2 + P_2 C_2$
- $C_4 = G_3 + P_3 C_3$

## 16-bit Adder

- $C_4 = G_{3:0} + P_{3:0} C_0$
- $C_8 = G_{7:4} + P_{7:4} C_4$
- $C_{12} = G_{11:8} + P_{11:8} C_8$
- $C_{16} = G_{15:12} + P_{15:12} C_{12}$

**Exactly the same structure. So CLA could be used to generate Group Carry.**

# 32-bit CLA with 4-bit Blocks



# Carry-Lookahead Adder Delay

---

$$t_{CLA} = t_{pg} + t_{pg\_block} + (N/k - 1) t_{AND\_OR} + kt_{FA}$$

$t_{pg}$ : delay to generate all  $P_i G_i$

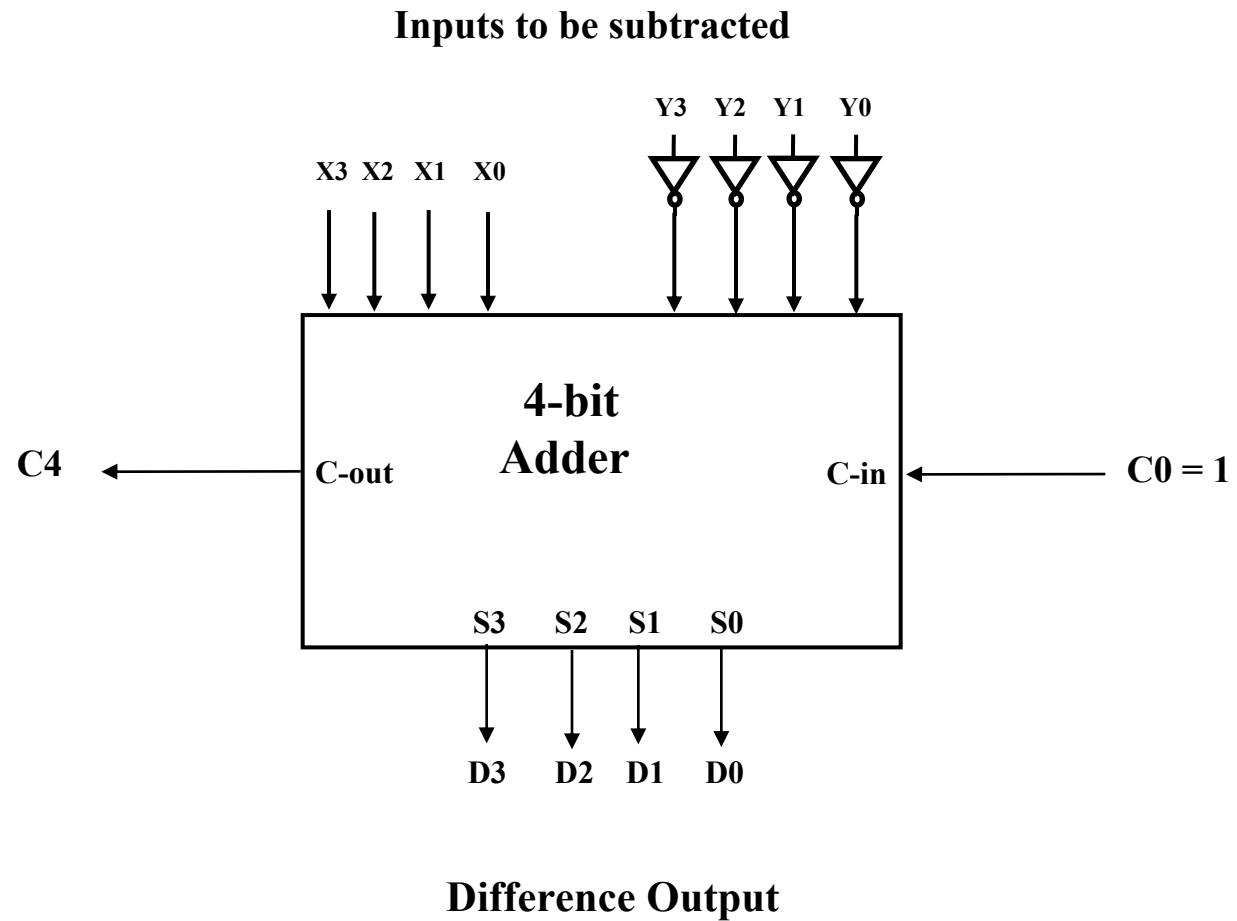
$t_{pg\_block}$ : delay of generate all  $P_{i:j}, G_{i:j}$

$t_{AND\_OR}$ : delay from  $C_{in}$  to  $C_{out}$  of final AND/OR gate in k-bit CLA

block

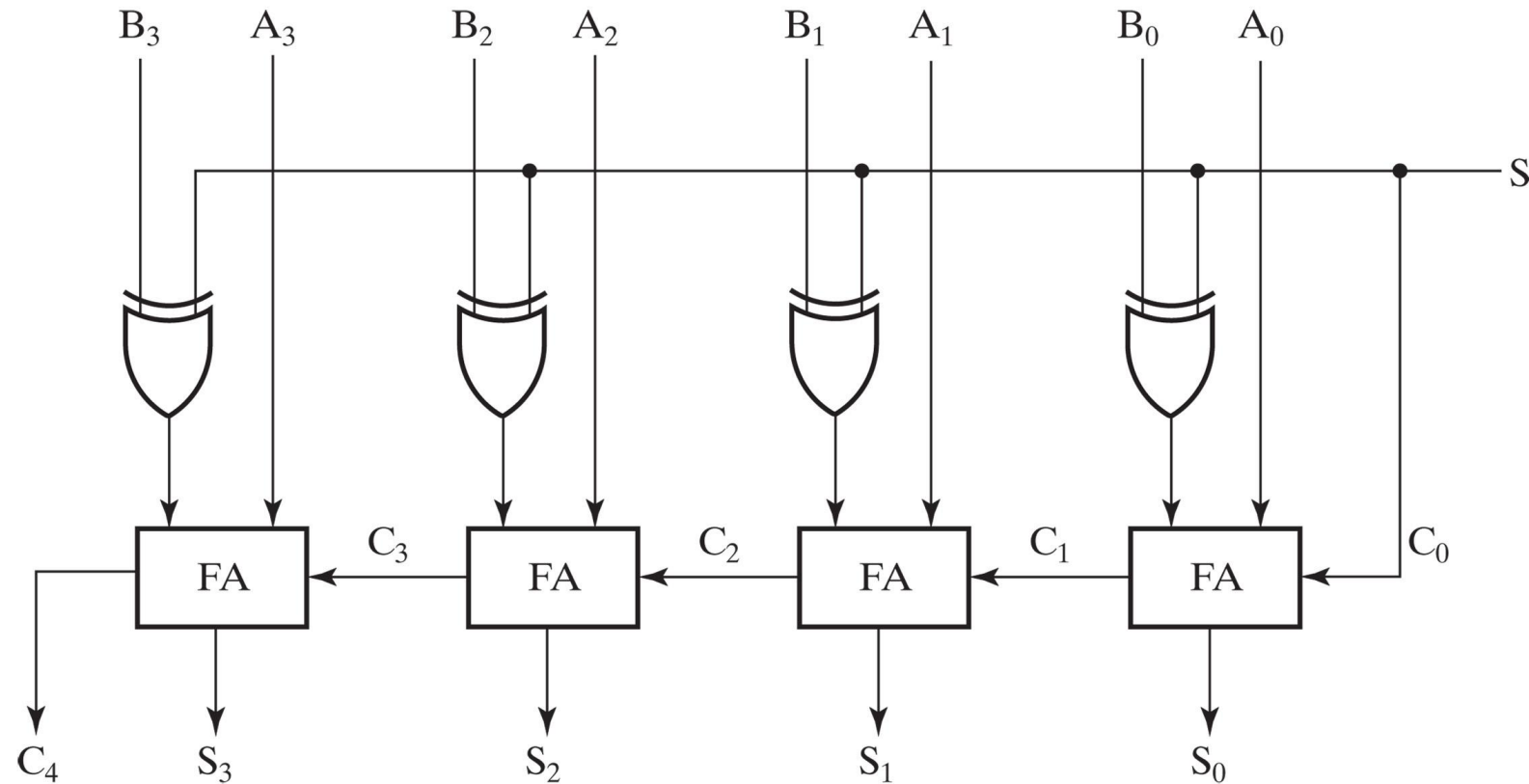
# 4-bit Subtractor Using 4-bit Adder

---





# 4-Bit Binary Adder-Subtractors



Copyright ©2016 Pearson Education, All Rights Reserved

- When  $S=0$ : Addition ( $A+B$ )
- When  $S=1$ : Subtraction ( $A+2$ 's complement of  $B$ )
- Can be used to add/subtract unsigned numbers and signed 2's complement numbers

# Important Flags

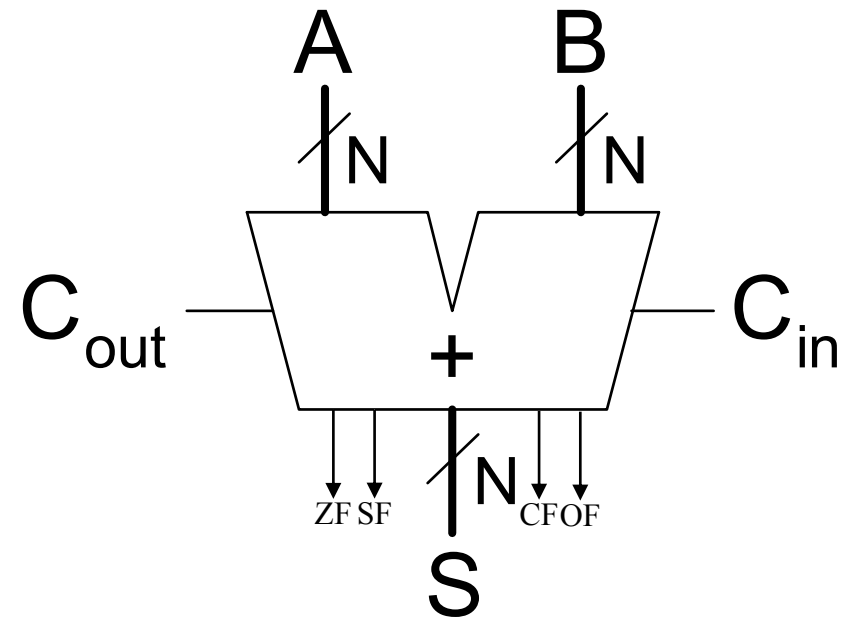
---

- Zero flag (ZF)
  - $ZF = 1$  means the result is 0
  - Valid for both unsigned and signed operations
- Sign flag (SF/NF)
  - The sign of the result, i.e.,  $S_{n-1}$
  - Valid for signed operations
- Carry/borrow flag (CF)
  - If  $CF = 1$ 
    - Carry for addition, i.e.,  $C_{out}$
    - Borrow for subtraction, i.e.,  $\sim C_{out}$
  - Valid for unsigned operations
- Overflow flag (OF)
  - Valid for signed operations

# Adders with Flags

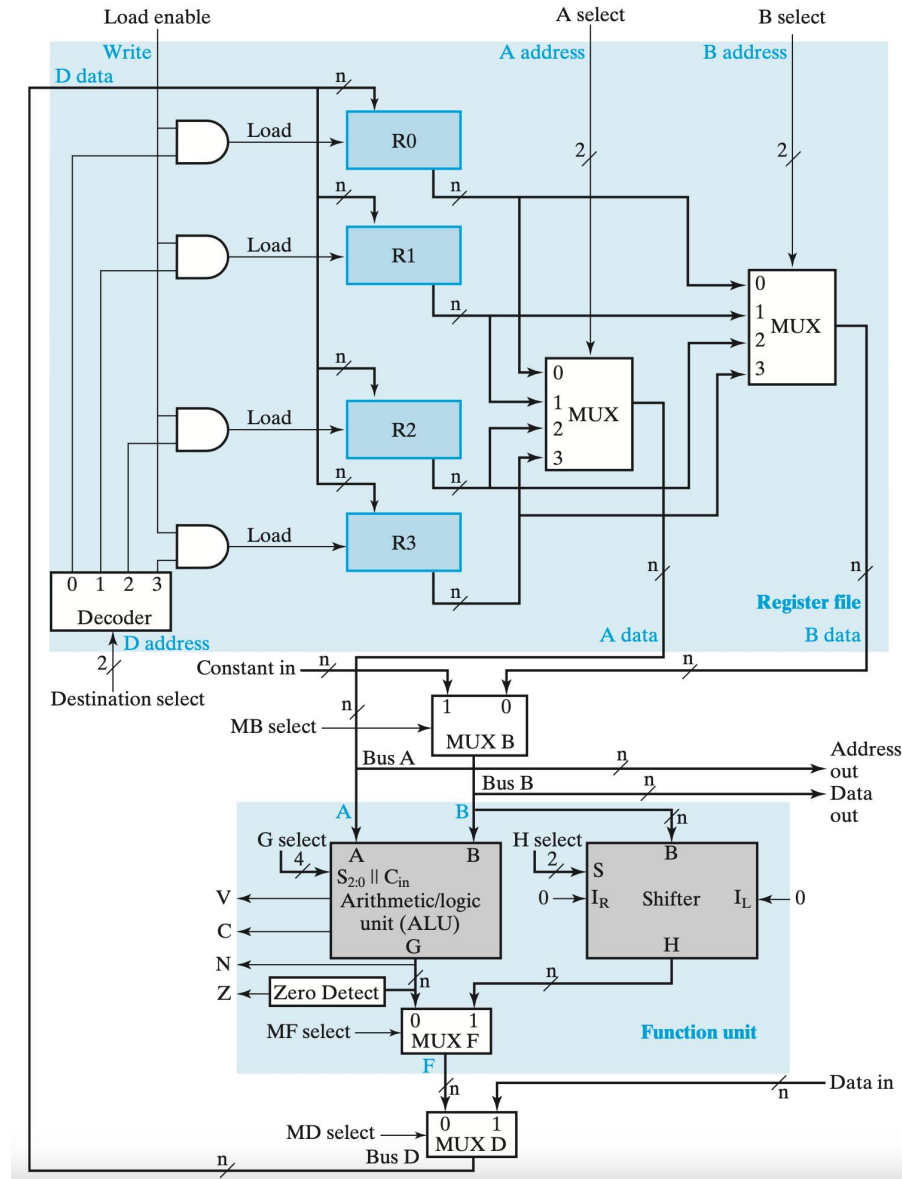
- ZF, SF, CF and OF

Symbol



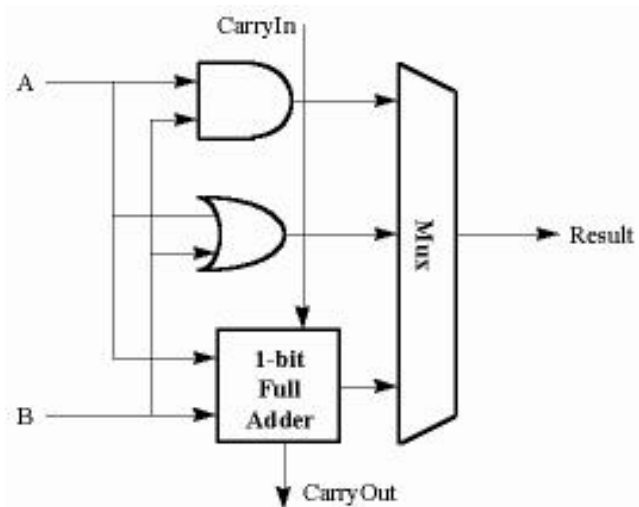
# Datapath = ALU + Registers

- The combination of a set of registers with a shared ALU and interconnecting paths is the datapath for the system.
- The control unit for the datapath directs the information flow through the buses, the ALU, the shifter, and the registers by applying signals to the select inputs.



# A Simple Example: 1-Bit ALU with 3 OPs

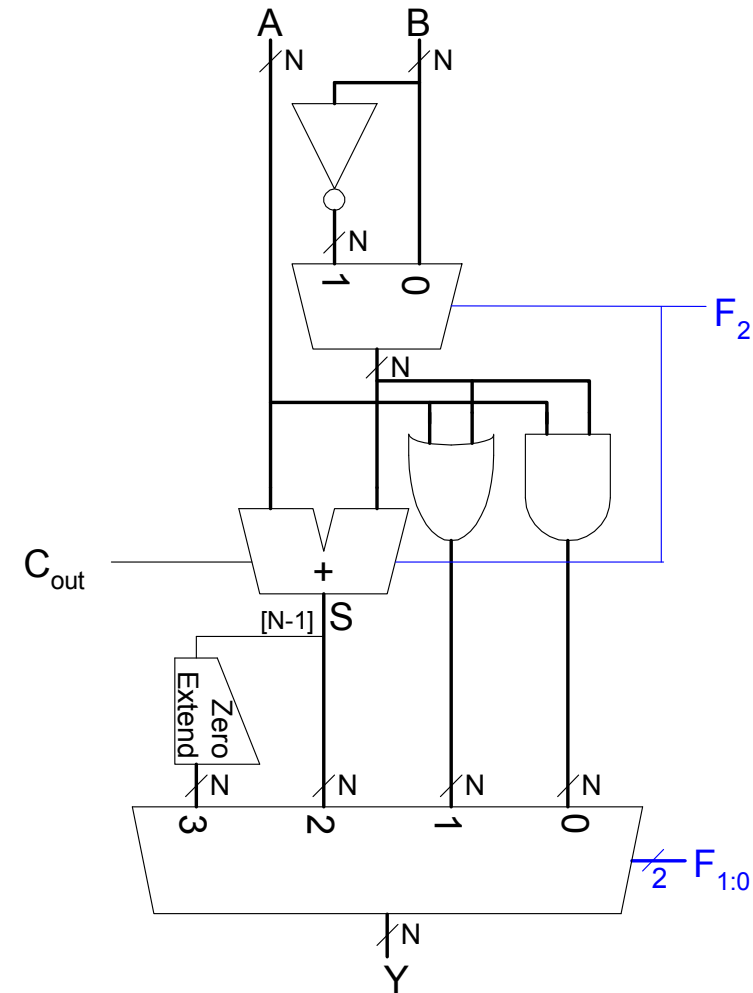
| $F_{1:0}$ | Function |
|-----------|----------|
| 00        | $A \& B$ |
| 01        | $A   B$  |
| 10        | $A + B$  |
| 11        | not used |



```
module ALU(  
    input [3:0] a, b,  
    input [1:0] aluop,  
    input cin,  
    output [3:0] f,  
    output OF, SF, ZF, CF,  
    output cout  
);  
    wire [3:0] sum;  
    CLA_FLAGS(a, b, cin, sum,  
OF, SF, ZF, CF, cout);  
    always @(*) begin  
        case(aluop)  
            2'b00: f = a & b;  
            2'b01: f = a | b;
```

# Set Less Than (SLT) Example

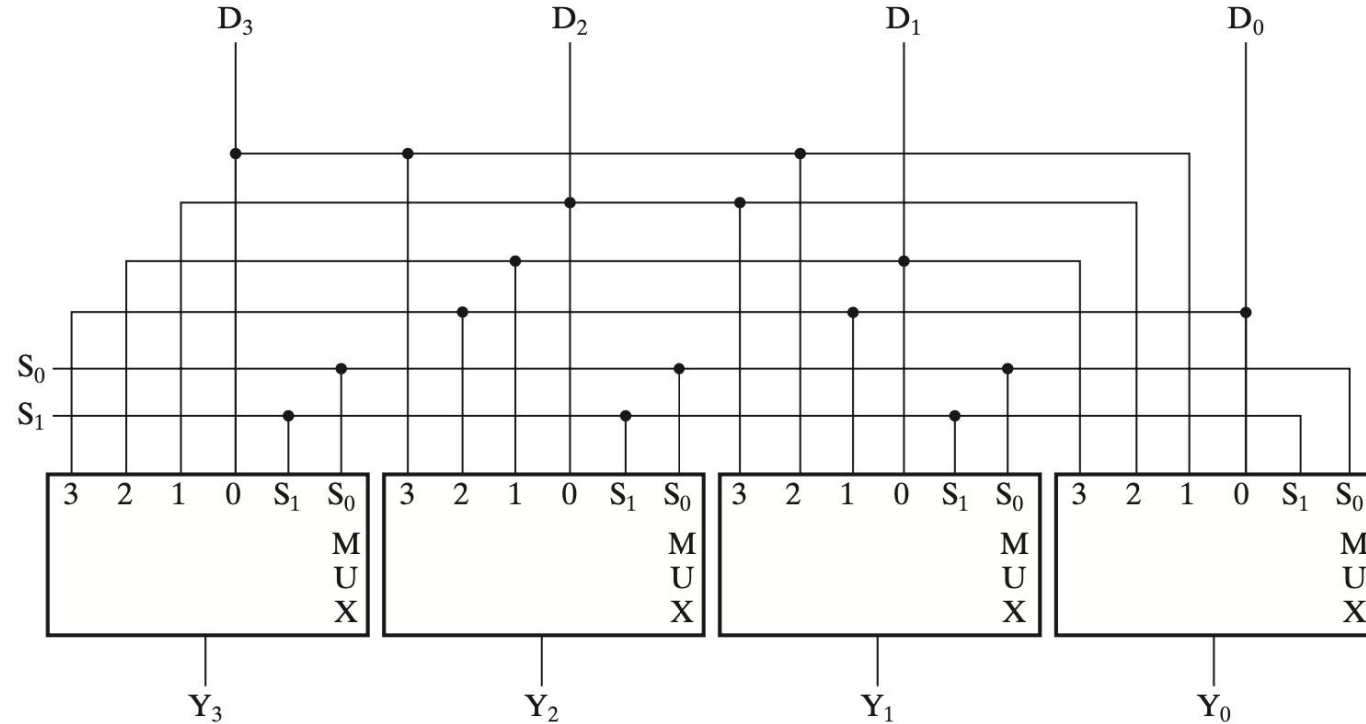
- Configure 32-bit ALU for SLT operation:  $A = 25$  and  $B = 32$ 
  - $A < B$ , so  $Y$  should be 32-bit representation of 1 (0x00000001)
  - $F_{2:0} = 111$ 
    - $F_2 = 1$  (adder acts as subtractor), so  $25 - 32 = -7$
    - $-7$  has 1 in the most significant bit ( $S_{31} = 1$ )
    - $F_{1:0} = 11$  multiplexer selects  $Y = S_{31}$  (zero extended) = 0x00000001.



# 4-Bit Barrel Shifter (ROL)

Function Table for 4-Bit Barrel Shifter

| Select |       | Output |       |       |       | Operation              |
|--------|-------|--------|-------|-------|-------|------------------------|
| $S_1$  | $S_0$ | $Y_3$  | $Y_2$ | $Y_1$ | $Y_0$ |                        |
| 0      | 0     | $D_3$  | $D_2$ | $D_1$ | $D_0$ | No rotation            |
| 0      | 1     | $D_2$  | $D_1$ | $D_0$ | $D_3$ | Rotate one position    |
| 1      | 0     | $D_1$  | $D_0$ | $D_3$ | $D_2$ | Rotate two positions   |
| 1      | 1     | $D_0$  | $D_3$ | $D_2$ | $D_1$ | Rotate three positions |



# Example of Implementation 2

- Let's do  $0010 \times 0110$  (2 x 6), unsigned

| Iteration | <u>Implementation 2</u>                        |            |              |                       |
|-----------|------------------------------------------------|------------|--------------|-----------------------|
|           | Step                                           | Multiplier | Multiplicand | Product               |
| 0         | initial values                                 | 0110       | 0010         | 0000 <sub>xxxx</sub>  |
| 1         | 1: 0 -> no op                                  | 0110       | 0010         | 0000 <sub>xxxx</sub>  |
|           | 2: Multiplier shift right/ Product shift right | x011       | 0010         | 0000 0 <sub>xxx</sub> |
| 2         | 1: 1 -> product = product + multiplicand       | x011       | 0010         | 0010 0 <sub>xxx</sub> |
|           | 2: Multiplier shift right/ Product shift right | xx01       | 0010         | 0001 00 <sub>xx</sub> |
| 3         | 1: 1 -> product = product + multiplicand       | xx01       | 0010         | 0011 00 <sub>xx</sub> |
|           | 2: Multiplier shift right/ Product shift right | xxx0       | 0010         | 0001 100 <sub>x</sub> |
| 4         | 1: 0 -> no op                                  | xxx0       | 0010         | 0001 100 <sub>x</sub> |
|           | 2: Multiplier shift right/ Product shift right | xxxx       | 0010         | 0000 1100             |



# Example of Implementation 3

- Let's do 0010 x 0110 (2 x 6), unsigned

| Iteration | Implementation 3                               |            |              |                    |
|-----------|------------------------------------------------|------------|--------------|--------------------|
|           | Step                                           | Multiplier | Multiplicand | Product Multiplier |
| 0         | initial values                                 | 0110       | 0010         | 0000 0110          |
| 1         | 1: 0 -> no op                                  | 0110       | 0010         | 0000 0110          |
|           | 2: Multiplier shift right/ Product shift right | x011       | 0010         | 0000 0011          |
| 2         | 1: 1 -> product = product + multiplicand       | x011       | 0010         | 0010 0011          |
|           | 2: Multiplier shift right/ Product shift right | xx01       | 0010         | 0001 0001          |
| 3         | 1: 1 -> product = product + multiplicand       | xx01       | 0010         | 0011 0001          |
|           | 2: Multiplier shift right/ Product shift right | xxx00      | 0010         | 0001 1000          |
| 4         | 1: 0 -> no op                                  | xxx0       | 0010         | 0001 1000          |
|           | 2: Multiplier shift right/ Product shift right | xxxx       | 0010         | 0000 1100          |

# Booth's Algorithm

- 检测连续的1
- $111\dots 1 = 2^n - 1$ , 类比9, 999

| Current bit | Bit to right | Explanation            | Example              | Operation |
|-------------|--------------|------------------------|----------------------|-----------|
| 1           | 0            | Begins run of '1'      | 0000111 <b>1</b> 000 | Subtract  |
| 1           | 1            | Middle of run of '1'   | 000011 <b>11</b> 000 | Nothing   |
| 0           | 1            | End of a run of '1'    | 000 <b>01</b> 111000 | Add       |
| 0           | 0            | Middle of a run of '0' | 0 <b>00</b> 01111000 | Nothing   |

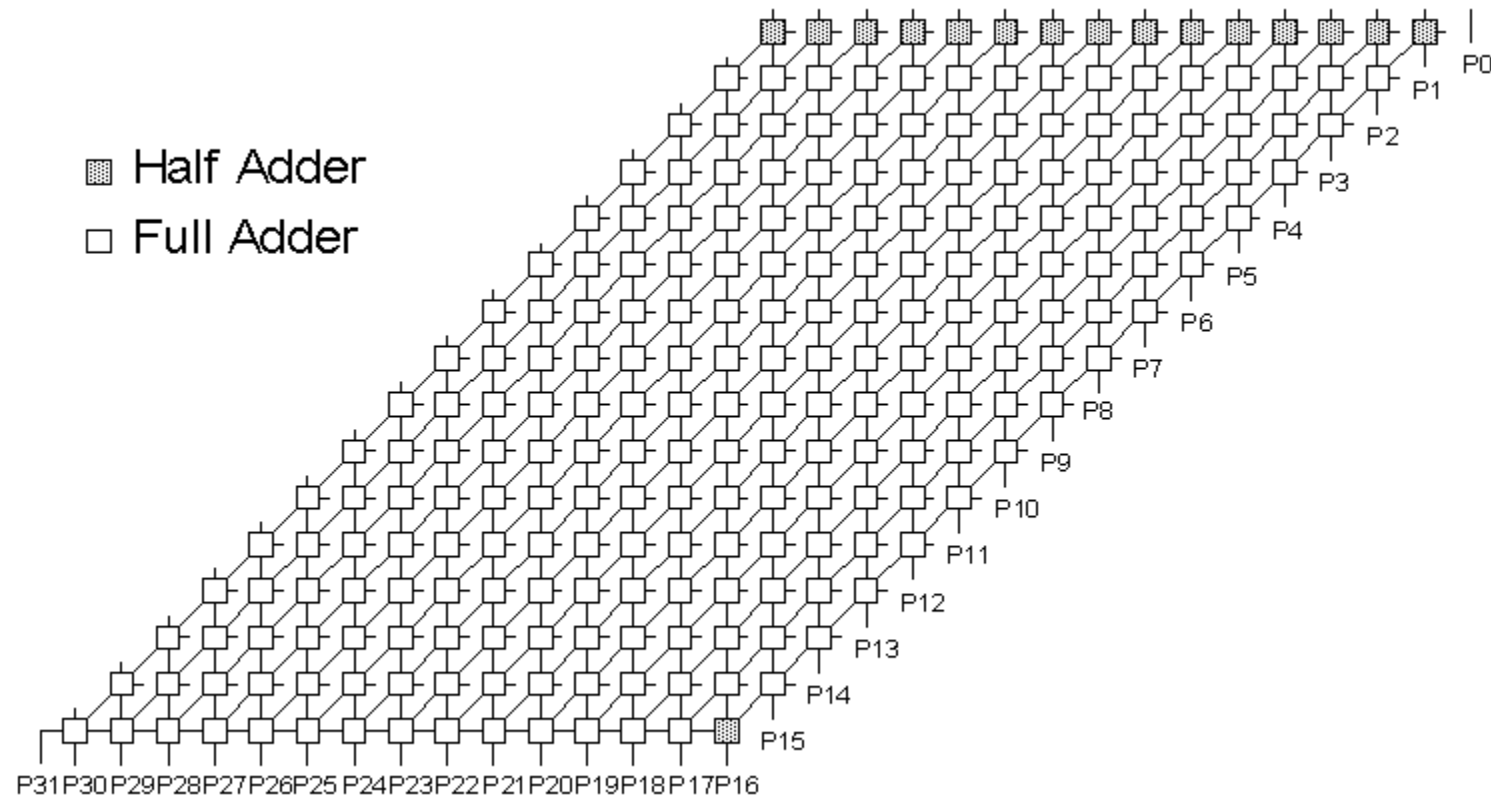
# Example 1 of Booth's Algorithm

- Let's do  $0010 \times 1101$  ( $2 \times -3$ )

| Iteration | Implementation 3                              |              |                     |
|-----------|-----------------------------------------------|--------------|---------------------|
|           | Step                                          | Multiplicand | Product             |
| 0         | initial values                                | 0010         | 0000 110 <b>1 0</b> |
| 1         | <b>10</b> -> product = product – multiplicand | 0010         | 1110 1101 0         |
|           | shift right                                   |              | 1111 011 <b>0 1</b> |
| 2         | <b>01</b> -> product = product + multiplicand | 0010         | 0001 0110 1         |
|           | shift right                                   |              | 0000 101 <b>1 0</b> |
| 3         | <b>10</b> -> product = product – multiplicand | 0010         | 1110 1011 0         |
|           | shift right                                   |              | 1111 010 <b>1 1</b> |
| 4         | <b>11</b> -> no op                            | 0010         | 1111 0101 1         |
|           | shift right                                   |              | 1111 1010 1         |



# 16-bit Array Multiplier



Conceptually straightforward

Fairly expensive hardware, integer multiplies relatively rare

Most used in array address calc: replace with shifts

# Example of Implementation 1

- Let's do  $7 \div 2$

| Iteration | Step                                      | Quotient | Divisor   | Remainder |
|-----------|-------------------------------------------|----------|-----------|-----------|
| 0         | Initial values                            | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem=Rem-Div                            | 0000     | 0010 0000 | 1110 0111 |
|           | 2b: Rem<0=>+Div, sll Q, Q <sub>0</sub> =0 | 0000     | 0010 0000 | 0000 0111 |
|           | 3: Shift Div right                        | 0000     | 0001 0000 | 0000 0111 |
| 2         | 1: Rem=Rem-Div                            | 0000     | 0001 0000 | 1111 0111 |
|           | 2b: Rem<0=>+Div, sll Q, Q <sub>0</sub> =0 | 0000     | 0001 0000 | 0000 0111 |
|           | 3: Shift Div right                        | 0000     | 0000 1000 | 0000 0111 |
| 3         | 1: Rem=Rem-Div                            | 0000     | 0000 1000 | 1111 1111 |
|           | 2b: Rem<0=>+Div, sll Q, Q <sub>0</sub> =0 | 0000     | 0000 1000 | 0000 0111 |
|           | 3: Shift Div right                        | 0000     | 0000 0100 | 0000 0111 |
| 4         | 1: Rem=Rem-Div                            | 0000     | 0000 0100 | 0000 0011 |
|           | 2a: Rem≥0=> sll Q, Q <sub>0</sub> =1      | 0001     | 0000 0100 | 0000 0011 |
|           | 3: Shift Div right                        | 0001     | 0000 0010 | 0000 0011 |
| 5         | 1: Rem=Rem-Div                            | 0001     | 0000 0010 | 0000 0001 |
|           | 2a: Rem≥0=> sll Q, Q <sub>0</sub> =1      | 0011     | 0000 0010 | 0000 0001 |
|           | 3: Shift Div right                        | 0011     | 0000 0001 | 0000 0001 |

# Restoring Division

| Iteration | Divisor | Divide algorithm                               |           |
|-----------|---------|------------------------------------------------|-----------|
|           |         | Step                                           | Remainder |
| 0         | 0010    | Initial values                                 | 0000 0111 |
|           | 0010    | Shift Rem left 1                               | 0000 1110 |
| 1         | 0010    | 2: Rem = Rem - Div                             | 1110 1110 |
|           | 0010    | 3b: Rem < 0 $\Rightarrow$ + Div, sll R, R0 = 0 | 0001 1100 |
| 2         | 0010    | 2: Rem = Rem - Div                             | 1111 1100 |
|           | 0010    | 3b: Rem < 0 $\Rightarrow$ + Div, sll R, R0 = 0 | 0011 1000 |
| 3         | 0010    | 2: Rem = Rem - Div                             | 0001 1000 |
|           | 0010    | 3a: Rem $\geq$ 0 $\Rightarrow$ sll R, R0 = 1   | 0011 0001 |
| 4         | 0010    | 2: Rem = Rem - Div                             | 0001 0001 |
|           | 0010    | 3a: Rem $\geq$ 0 $\Rightarrow$ sll R, R0 = 1   | 0010 0011 |
| Done      | 0010    | shift left half of Rem right 1                 | 0001 0011 |

# Non-Restoring Division

| Iteration | Step                          | Divisor | Remainder |
|-----------|-------------------------------|---------|-----------|
| 0         | Initial values                | 0010    | 0000 0111 |
|           | Shift rem left 1              | 0010    | 0000 1110 |
| 1         | 2: Rem = Rem - Div            | 0010    | 1110 1110 |
|           | 3b: Rem < 0 (add next), sll 0 | 0010    | 1101 1100 |
| 2         | 2: Rem = Rem + Div            | 0010    | 1111 1100 |
|           | 3b: Rem < 0 (add next), sll 0 | 0010    | 1111 1000 |
| 3         | 2: Rem = Rem + Div            | 0010    | 0001 1000 |
|           | 3a: Rem > 0 (sub next), sll 1 | 0010    | 0011 0001 |
| 4         | Rem = Rem - Div               | 0010    | 0001 0001 |
|           | Rem > 0 (sub next), sll 1     | 0010    | 0010 0011 |
|           | Shift Rem right by 1          | 0010    | 0001 0011 |



# Two-Operand Instructions

---

- Here is a possible sequence of instructions for our equation (say the operands are registers):

$$Y = (A-B) / (C + (D * E))$$

- SUB A, B ; Register A  $\leftarrow$  A - B
  - MUL D, E ; Register D  $\leftarrow$  D \* E
  - ADD D, C ; Register D  $\leftarrow$  D + C
  - DIV A, D ; Register A  $\leftarrow$  A / D
- Get same end result as before, but we changed the contents of registers A and D
  - If we had some later processing that wanted to use the original contents of those registers, we must make a copy of them before performing the computation
    - MOV R1, A ; Copy A to register R1
    - MOV R2, D ; Copy D to register R2
    - SUB R1, B ; Register R1  $\leftarrow$  R1 - B
    - MUL R2, E ; Register R2  $\leftarrow$  R2 \* E
    - ADD R2, C ; Register R2  $\leftarrow$  R2 + C
    - DIV R1, R2 ; Register R1  $\leftarrow$  R1 / R2
  - Now the original registers for A-E remain the same as before, but at the cost of some extra instructions to save the results.

# One-Operand Instructions

---

- Can use the same idea to get rid of the second operand, leaving only one operand
- The second operand is left implicit; e.g., could assume that the second operand will always be in a register such as the Accumulator:

$$Y = (A-B) / (C + (D * E))$$

- LDA D ; Load ACC with D
  - MUL E ; Acc ← Acc \* E
  - ADD C ; Acc ← Acc + C
  - STO R1 ; Store Acc to R1
  - LDA A ; Acc ← A
  - SUB B ; Acc ← A-B
  - DIV R1 ; Acc ← Acc / R1
- Many early computers relied heavily on one-address based instructions, as it makes the CPU much simpler to design. As you can see, it does become somewhat more unwieldy to program.

# Stack-Based Instructions

---

$$Y = (A-B) / (C + (D * E))$$

| ■ Instruction | Stack Contents (stack increase ->) |
|---------------|------------------------------------|
| ■ PUSH B      | ; B                                |
| ■ PUSH A      | ; B, A                             |
| ■ SUB         | ; (A-B)                            |
| ■ PUSH E      | ; (A-B), E                         |
| ■ PUSH D      | ; (A-B), E, D                      |
| ■ MUL         | ; (A-B), (E*D)                     |
| ■ PUSH C      | ; (A-B), (E*D), C                  |
| ■ ADD         | ; (A-B), (E*D+C)                   |
| ■ DIV         | ; (A-B) / (E*D+C)                  |

# Summary - Addressing Modes

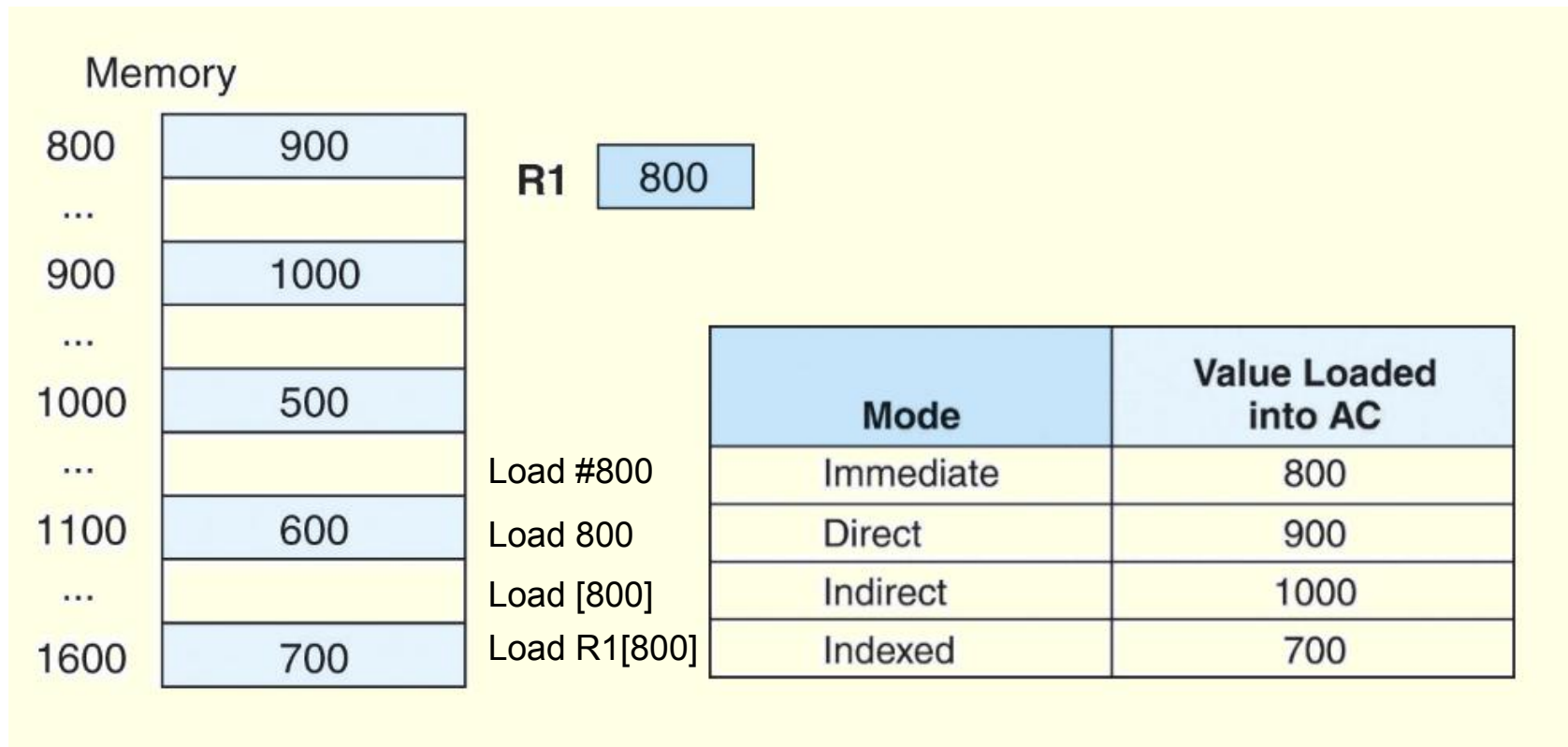
---

| Addressing Mode        | Syntax                                | Meaning                                |
|------------------------|---------------------------------------|----------------------------------------|
| Immediate              | #K                                    | K                                      |
| Direct                 | K                                     | M[K]                                   |
| Indirect               | (K)                                   | M[M[K]]                                |
| Register               | (R <sub>n</sub> )                     | M[R <sub>n</sub> ]                     |
| Register Indexed       | (R <sub>m</sub> + R <sub>n</sub> )    | M[R <sub>m</sub> + R <sub>n</sub> ]    |
| Register Based         | (R <sub>m</sub> + X)                  | M[R <sub>m</sub> + X]                  |
| Register Based Indexed | (R <sub>m</sub> + R <sub>n</sub> + X) | M[R <sub>m</sub> + R <sub>n</sub> + X] |

**Four ways of computing the address of a value in memory: (1) a constant value known at assembly time, (2) the contents of a register, (3) the sum of two registers, (4) the sum of a register and a constant. The table gives names to these and other addressing modes.**

# Addressing Example

- These are the values loaded into the accumulator for each addressing mode.



Load R1    Using Indirect Addressing?

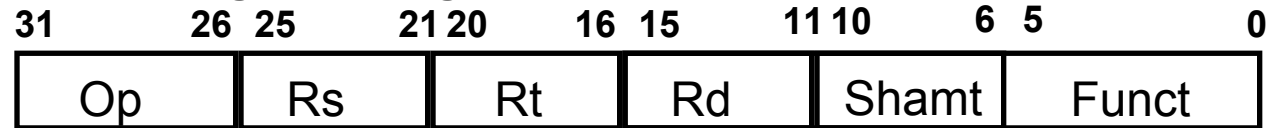
# Types of Operations

---

- Arithmetic and Logic
- Shift
- Data Transfer
  - E.g., MOV/LOAD/STORE
- String
- Control
  - BRANCH/JMP/CALL/RET/...
- System
  - HALT/INTERRUPT ON/INTERRUPT OFF/SWITCH...
- Input/Output
- ...

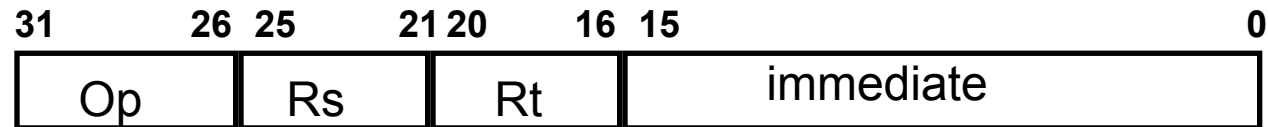
# RISC Example: MIPS

## R-format (Register-Register)



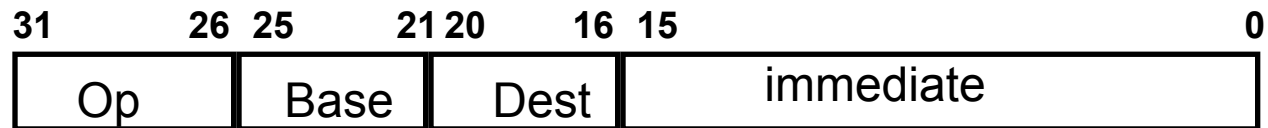
add \$1, \$2, \$3

## I-format (Register-Immediate)



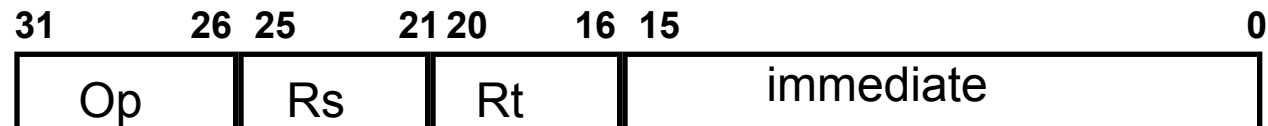
addi \$1, \$2, -5

## I-format (Load/Store)



lw \$1, 24(\$9)

## I-format (Branch)



beq L1, \$4, \$0

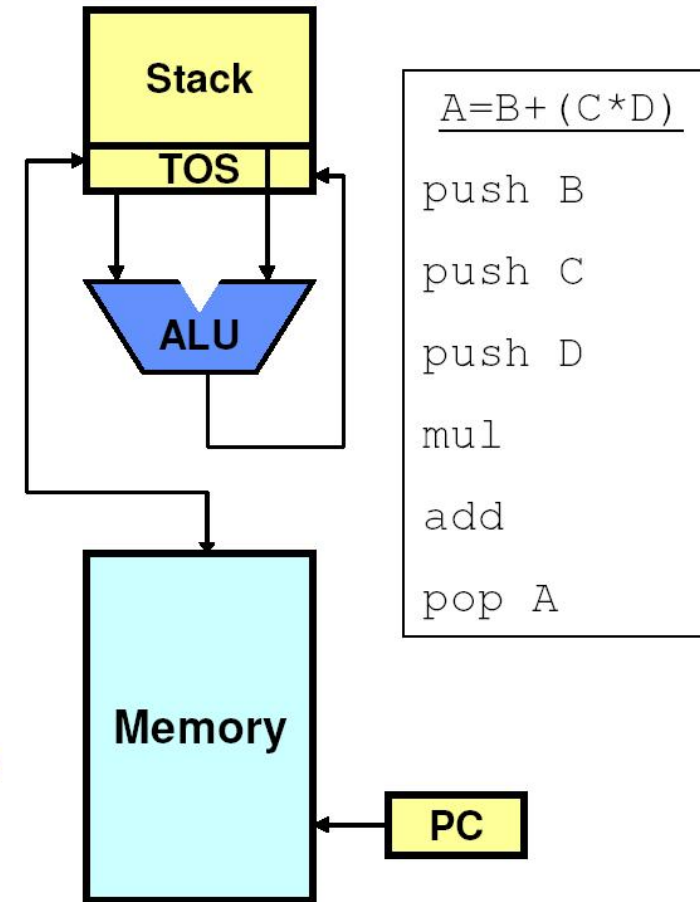
## J-format (Jump / Call)



j L2

# Stack Architectures

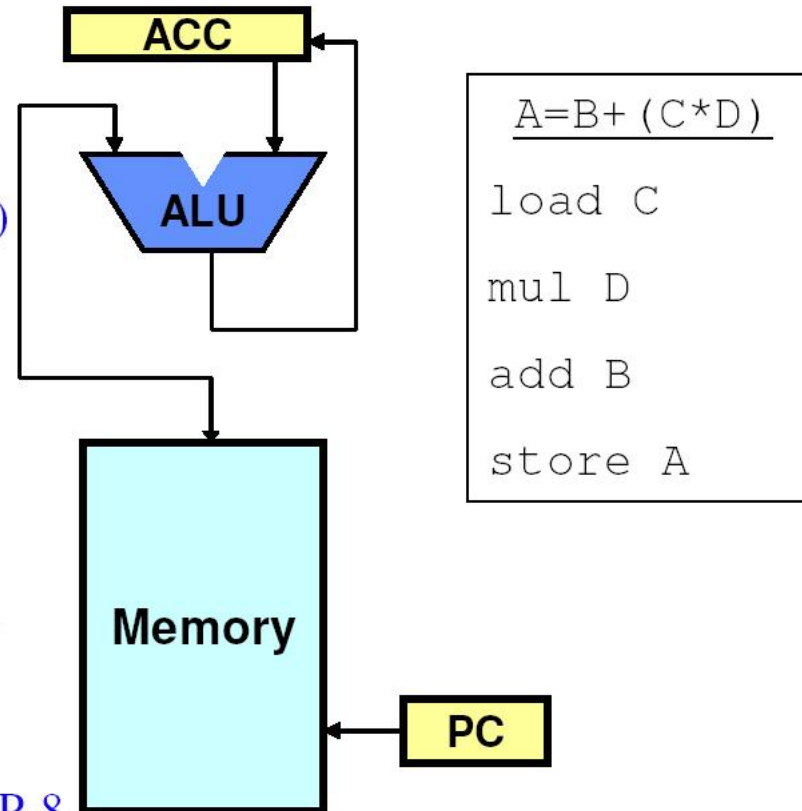
- Stack: First-In Last-Out data structure (FILO)
- Instruction operands
  - None for ALU operations
  - One for push/pop
- Advantages:
  - Short instructions
  - Compiler is easy to write
- Disadvantages
  - Code is inefficient
    - Fix: random access to stacked values
  - Stack size & access latency
    - Fix : register file or cache for top entries
- Examples
  - 60s: Burroughs B5500/6500, HP 3000/70
  - Today: Java VM





# Accumulator Architectures

- Single register (accumulator)
- Instructions
  - $ALU (Acc \leftarrow Acc + *M)$
  - Load to accumulator ( $Acc \leftarrow *M$ )
  - Store from accumulator ( $*M \leftarrow Acc$ )
- Instruction operands
  - One explicit (memory address)
  - One implicit (accumulator)
- Attributes:
  - Short instructions
  - Minimal internal state; simple design
  - Many loads and stores
- Examples:
  - Early machines: IBM 7090, DEC PDP-8
  - Today: DSP architectures



# Memory-Memory Architectures

---

- All ALU operands from memory addresses
- Advantages
  - No register wastage
  - Lowest instruction count
- Disadvantages
  - Large variation in instruction length
  - Large variation in clocks per instructions
  - Huge memory traffic
- Examples
  - VAX

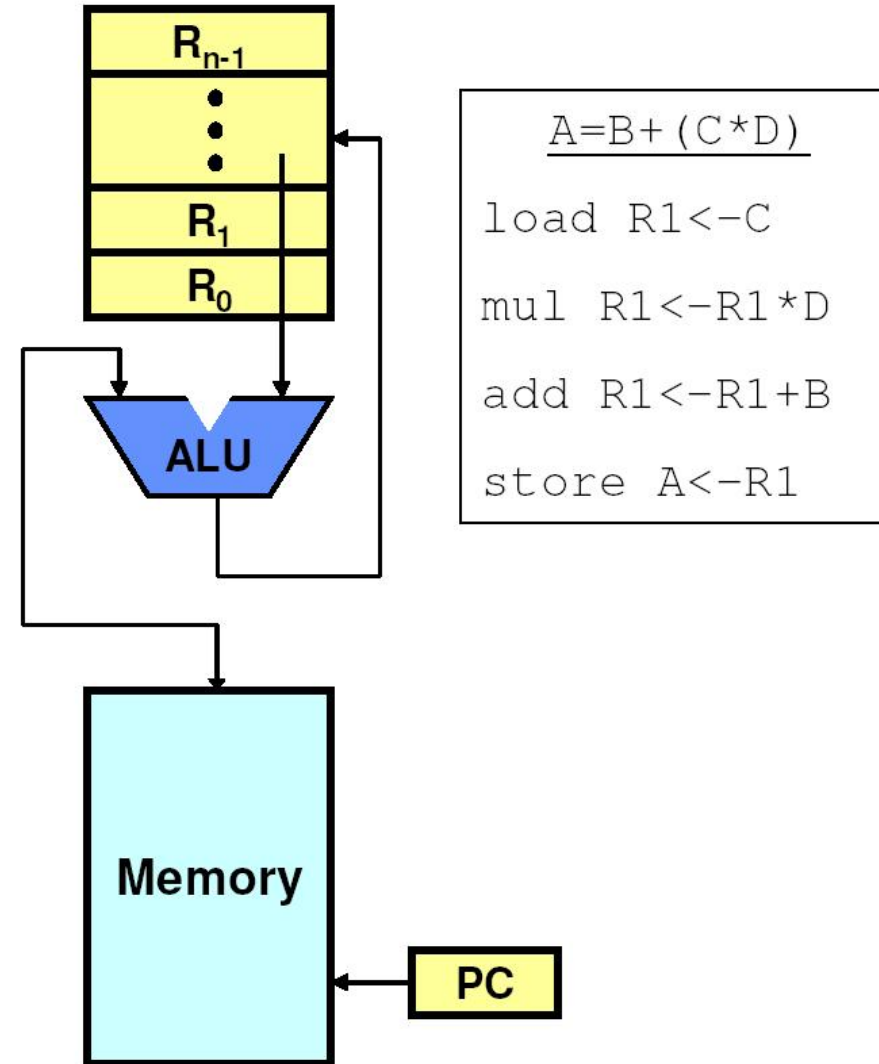
$$\underline{D = B + (C * D)}$$

mul D ← C \* D

add D ← D + B

# Register-Memory Architectures

- One memory address in ALU ops
- Typically 2-operand ALU ops
- Advantages
  - Small instruction count
  - Dense encoding
- Disadvantages
  - Result destroys an operand
  - Instruction length varies
  - Clocks per instruction varies
  - Harder to pipeline
- Examples
  - IBM 360/370, VAX



# Load-Store Architectures

- No memory addresses in ALU ops
- Typically 3-operand ALU ops
  - Bigger encoding, but simplifies register allocation
- Advantages
  - Simple fixed-length instructions
  - Easily pipelined
- Disadvantages
  - Higher instruction count
- Examples
  - CDC6600, CRAY-1, most RISCs

